



算法设计与分析

Lecture 8: Graph Algorithms (图算法)

曹刘娟

厦门大学信息学院计算机系

caoliujuan@xmu.edu.cn



图的用途

■ 图无处不在

■ Google

- BFS/DFS: 爬虫遍历互联网上尽可能多的网页
- 最大流: 爬虫最快速度下载网页，但是又不影响正常使用

■ 社交网络（朋友圈，公众号）

- 推荐/匹配
- 连接性（抓黑产）

■ 神经网络

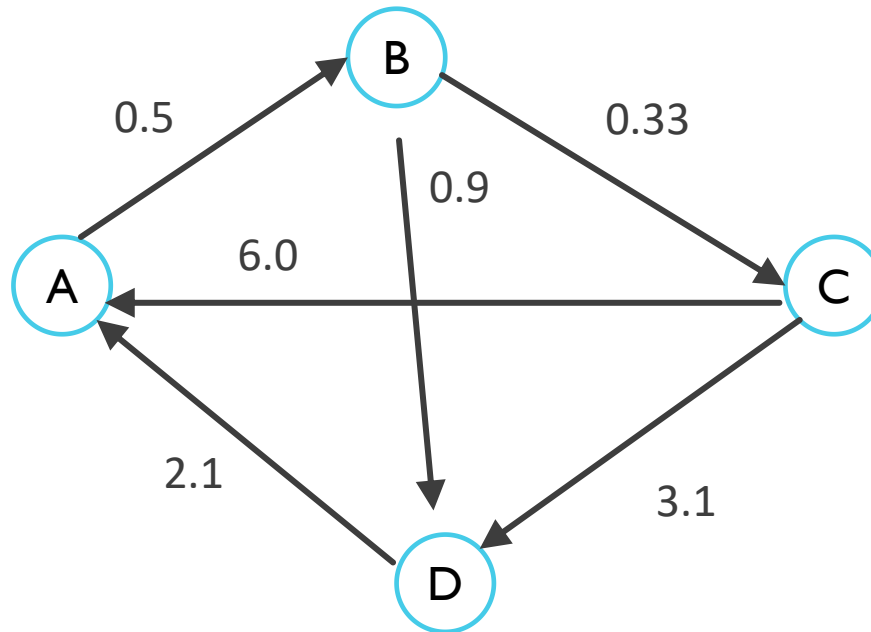
■ 下围棋（博弈树）

■ 金融（资金流向，比特币兑换最佳路径）

如何把实际问题转换为图的问题，是艺术



如何将图的知识用在经济领域





Representations of Graphs

Two standard ways to represent a graph $G = (V, E)$:

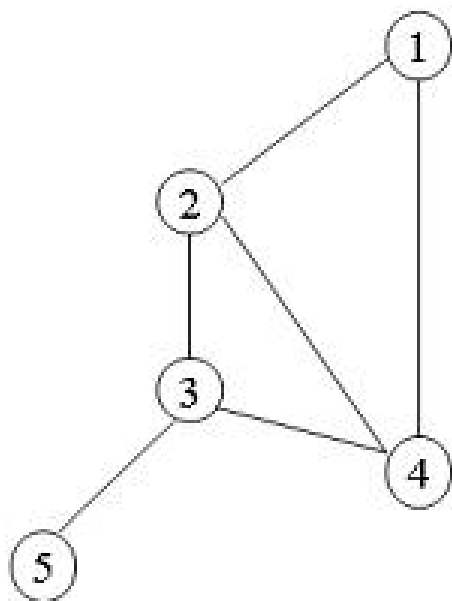
- Adjacency matrix (邻接矩阵): a $|V| \times |V|$ matrix $A = (a_{ij})$:

$$a_{ij} = \begin{cases} w(i, j) & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

- Adjacency lists (邻接表): each vertex u has a linked list $Adj[u]$, constructed by u 's neighbors.



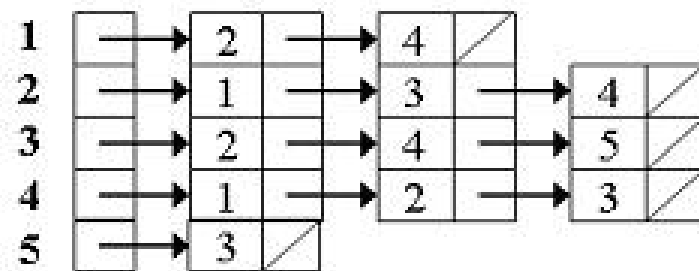
Representations of Graphs 无向图



(a)

	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	1	0
3	0	1	0	1	1
4	1	1	1	0	0
5	0	0	1	0	0

(b)

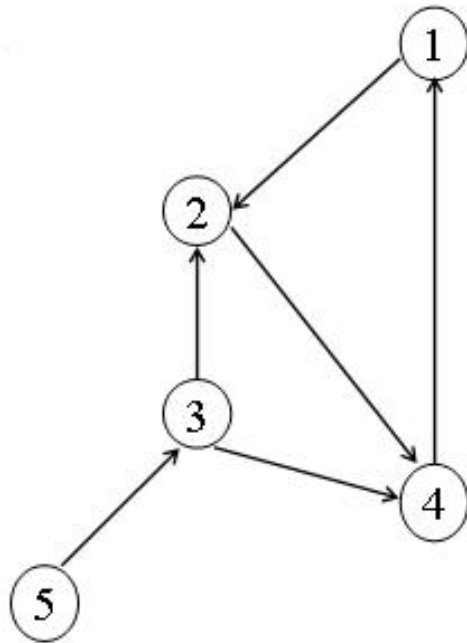


(c)

Adjacency matrix and adjacency lists for undirected graph.



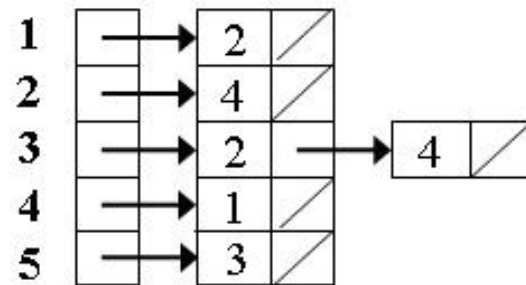
Representations of Graphs 有向图



(a)

	1	2	3	4	5
1	0	1	0	0	0
2	0	0	0	1	0
3	0	1	0	1	0
4	1	0	0	0	0
5	0	0	1	0	0

(b)



(c)

Adjacency matrix and adjacency lists for directed graph.



Representations of Graphs

The usage of these two representations is different:

- Adjacency matrix is suitable for representing **dense graphs** - those for which $|E|$ is close to $|V|^2$. 邻接矩阵更加适合边比较多的场景，即稠密图
 - Graphs close to complete graph (完全图).
- Adjacency lists represent **sparse graphs** - those for which $|E|$ is much less than $|V|^2$. 邻接表方式更加适合于边比较稀疏的场景
 - Graphs close to null graph (零图).



GRAPH SEARCH

图的搜索问题



Graph Search

- Graph search aims to compute the distance (smallest number of edges) from a vertex s to each reachable vertex.
- 图搜索问题主要是计算从一个节点到另外一个可以到达的节点的最短距离
- There are two general searching strategies: Breadth-first search (BFS) (宽度优先搜索) and depth-first search (DFS) (深度优先搜索).



GRAPH SEARCH

BREADTH-FIRST SEARCH 宽度优先搜索



Breadth-First Search 宽度优先搜索

- Given a **graph** $G = (V, E)$ and a distinguished source vertex s , **BFS** systematically explores **the edges** of G to "discover" every vertex that is reachable from s .
 - It computes the distance (**smallest number of edges**) from s to each reachable vertex. 计算从 s 到每个节点的最短距离（最少边数）
 - It also produces a "**breadth-first tree**" with **root** s that contains all **reachable vertices**. 构建了一颗宽度优先搜索树



BFS(G, s)

1 **for** each vertex $u \in V - \{s\}$ **do**

2 $color[u] \leftarrow \text{White}$

3 $d[u] \leftarrow \infty$

4 $\pi[u] \leftarrow \text{NIL}$

Initialization for all
unvisited vertices.

5 $color[s] \leftarrow \text{Gray}$

6 $d[s] \leftarrow 0$; $\pi[s] \leftarrow \text{NIL}$; $Q \leftarrow \emptyset$

7 Enqueue(Q, s)

8 **while** $Q \neq \emptyset$ **do**

Iterate over all
Gray vertices.

9 $u \leftarrow \text{Dequeue}(Q)$

10 **for** each $v \in \text{Adj}[u]$ **do**

11 **if** $color[v] = \text{White}$

Iterate over all
neighbors of u .

then

12 $color[v] \leftarrow \text{Gray}$

13 $d[v] \leftarrow d[u] + 1$

14 $\pi[v] \leftarrow u$

15 Enqueue(Q, v)

16 $color[u] \leftarrow \text{DarkGray}$

- $color[u]$: record the color of each vertex u .
- **White** 白色: Not visited. 还没有访问
- **Gray** 灰色: Searched. 访问
- **DarkGray** 深灰: All its neighbors have been searched. 所有邻居已访问
- $d[u]$: the number of edges on the path from s to u . 从 s 到 u 需要经过多少边
- $\pi[u]$: parent vertex of u . u 的父亲节点
- Q : a queue to store Gray vertices. 存储灰色节点的队列

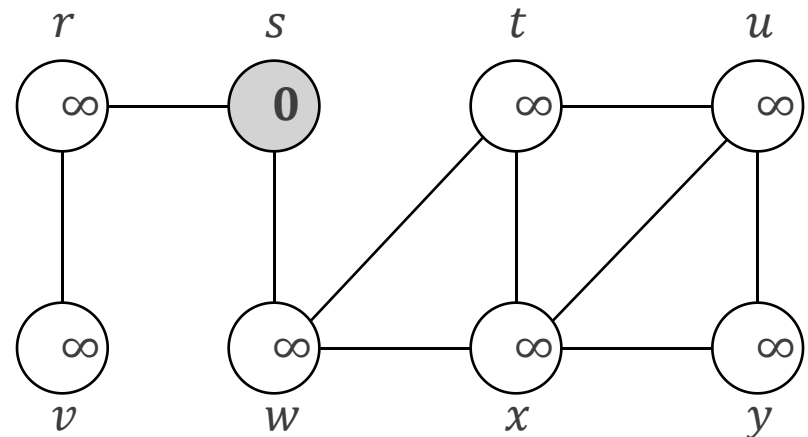


BFS(G, s)

```
1 for each vertex  $u \in V - \{s\}$  do
2    $color[u] \leftarrow \text{White}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{Gray}$ 
6  $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7 Enqueue( $Q, s$ )
8 while  $Q \neq \emptyset$  do
9    $u \leftarrow \text{Dequeue}(Q)$ 
10  for each  $v \in \text{Adj}[u]$  do
11    if  $color[v] = \text{White}$  then
12       $color[v] \leftarrow \text{Gray}$ 
13       $d[v] \leftarrow d[u] + 1$ 
14       $\pi[v] \leftarrow u$ 
15      Enqueue( $Q, v$ )
16   $color[u] \leftarrow \text{DarkGray}$ 
```

Q

s



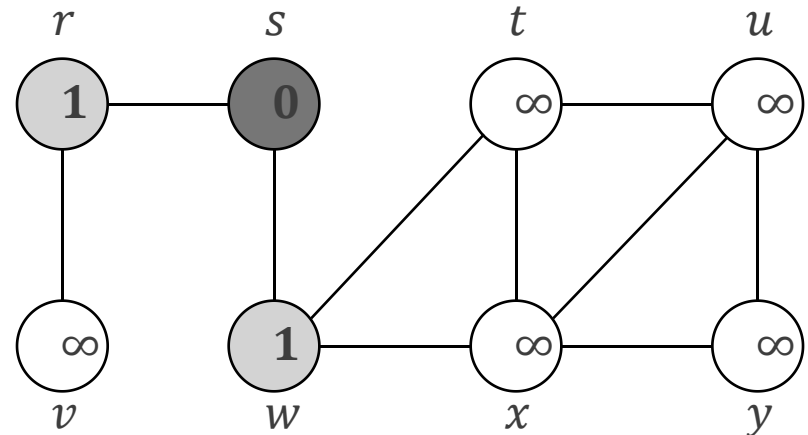


BFS(G, s)

```
1 for each vertex  $u \in V - \{s\}$  do
2    $color[u] \leftarrow \text{White}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{Gray}$ 
6  $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7 Enqueue( $Q, s$ )
8 while  $Q \neq \emptyset$  do
9    $u \leftarrow \text{Dequeue}(Q)$ 
10  for each  $v \in \text{Adj}[u]$  do
11    if  $color[v] = \text{White}$  then
12       $color[v] \leftarrow \text{Gray}$ 
13       $d[v] \leftarrow d[u] + 1$ 
14       $\pi[v] \leftarrow u$ 
15      Enqueue( $Q, v$ )
16   $color[u] \leftarrow \text{DarkGray}$ 
```

Dequeue(Q) = s

Q	
r	w
1	1





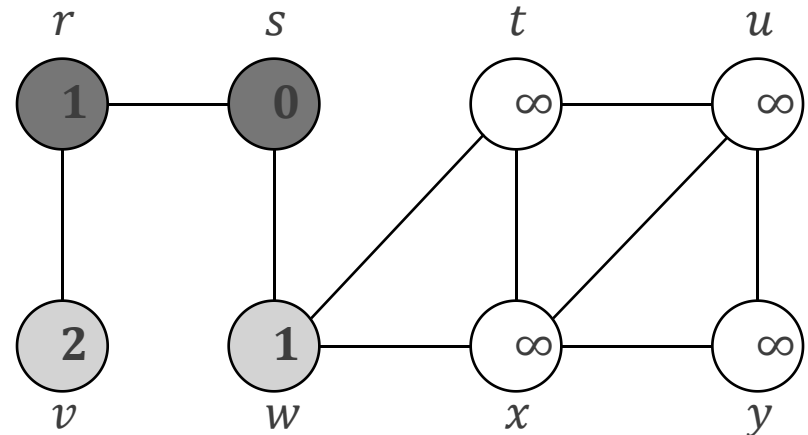
BFS(G, s)

```

1  for each vertex  $u \in V - \{s\}$  do
2       $color[u] \leftarrow \text{White}$ 
3       $d[u] \leftarrow \infty$ 
4       $\pi[u] \leftarrow \text{NIL}$ 
5   $color[s] \leftarrow \text{Gray}$ 
6   $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7  Enqueue( $Q, s$ )
8  while  $Q \neq \emptyset$  do
9       $u \leftarrow \text{Dequeue}(Q)$ 
10     for each  $v \in \text{Adj}[u]$  do
11         if  $color[v] = \text{White}$  then
12              $color[v] \leftarrow \text{Gray}$ 
13              $d[v] \leftarrow d[u] + 1$ 
14              $\pi[v] \leftarrow u$ 
15             Enqueue( $Q, v$ )
16      $color[u] \leftarrow \text{DarkGray}$ 
  
```

Dequeue(Q) = r

Q	w	v
	1	2





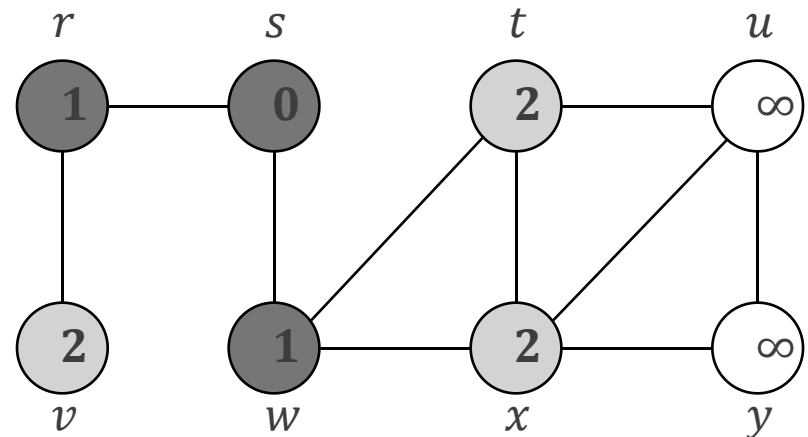
BFS(G, s)

```

1  for each vertex  $u \in V - \{s\}$  do
2       $color[u] \leftarrow \text{White}$ 
3       $d[u] \leftarrow \infty$ 
4       $\pi[u] \leftarrow \text{NIL}$ 
5   $color[s] \leftarrow \text{Gray}$ 
6   $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7  Enqueue( $Q, s$ )
8  while  $Q \neq \emptyset$  do
9       $u \leftarrow \text{Dequeue}(Q)$ 
10     for each  $v \in \text{Adj}[u]$  do
11         if  $color[v] = \text{White}$  then
12              $color[v] \leftarrow \text{Gray}$ 
13              $d[v] \leftarrow d[u] + 1$ 
14              $\pi[v] \leftarrow u$ 
15             Enqueue( $Q, v$ )
16      $color[u] \leftarrow \text{DarkGray}$ 
  
```

Dequeue(Q) = w

Q	v	t	x
	2	2	2





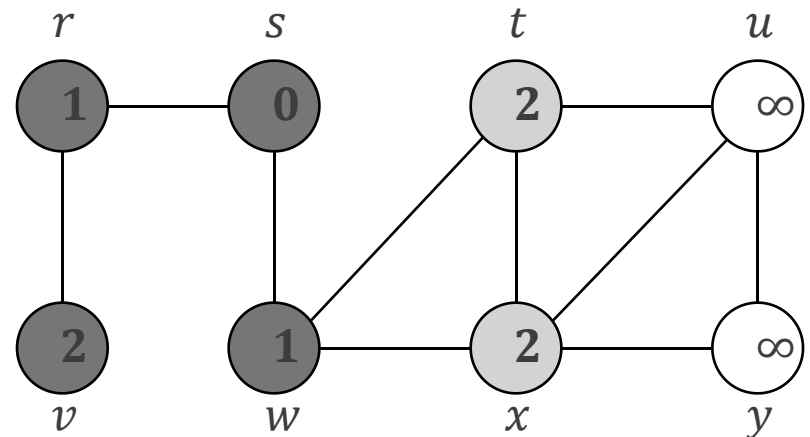
BFS(G, s)

```

1  for each vertex  $u \in V - \{s\}$  do
2       $color[u] \leftarrow \text{White}$ 
3       $d[u] \leftarrow \infty$ 
4       $\pi[u] \leftarrow \text{NIL}$ 
5   $color[s] \leftarrow \text{Gray}$ 
6   $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7  Enqueue( $Q, s$ )
8  while  $Q \neq \emptyset$  do
9       $u \leftarrow \text{Dequeue}(Q)$ 
10     for each  $v \in \text{Adj}[u]$  do
11         if  $color[v] = \text{White}$  then
12              $color[v] \leftarrow \text{Gray}$ 
13              $d[v] \leftarrow d[u] + 1$ 
14              $\pi[v] \leftarrow u$ 
15             Enqueue( $Q, v$ )
16      $color[u] \leftarrow \text{DarkGray}$ 
  
```

Dequeue(Q) = v

Q	t	x
	2	2



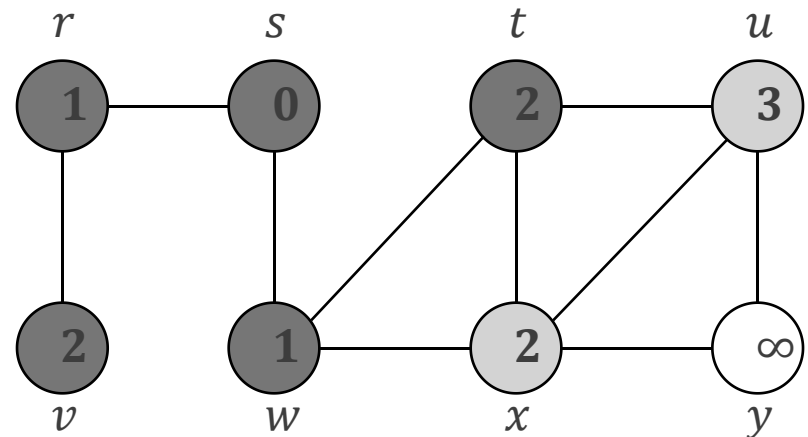


BFS(G, s)

```
1 for each vertex  $u \in V - \{s\}$  do
2    $color[u] \leftarrow \text{White}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{Gray}$ 
6  $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7 Enqueue( $Q, s$ )
8 while  $Q \neq \emptyset$  do
9    $u \leftarrow \text{Dequeue}(Q)$ 
10  for each  $v \in \text{Adj}[u]$  do
11    if  $color[v] = \text{White}$  then
12       $color[v] \leftarrow \text{Gray}$ 
13       $d[v] \leftarrow d[u] + 1$ 
14       $\pi[v] \leftarrow u$ 
15      Enqueue( $Q, v$ )
16   $color[u] \leftarrow \text{DarkGray}$ 
```

Dequeue(Q) = t

Q	x	u
	2	3



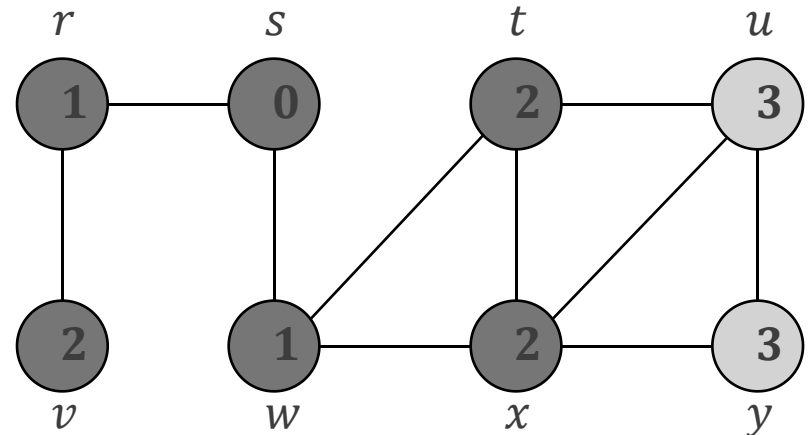


BFS(G, s)

```
1 for each vertex  $u \in V - \{s\}$  do
2    $color[u] \leftarrow \text{White}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{Gray}$ 
6  $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7 Enqueue( $Q, s$ )
8 while  $Q \neq \emptyset$  do
9    $u \leftarrow \text{Dequeue}(Q)$ 
10  for each  $v \in \text{Adj}[u]$  do
11    if  $color[v] = \text{White}$  then
12       $color[v] \leftarrow \text{Gray}$ 
13       $d[v] \leftarrow d[u] + 1$ 
14       $\pi[v] \leftarrow u$ 
15      Enqueue( $Q, v$ )
16   $color[u] \leftarrow \text{DarkGray}$ 
```

Dequeue(Q) = x

Q	u	y
	3	3





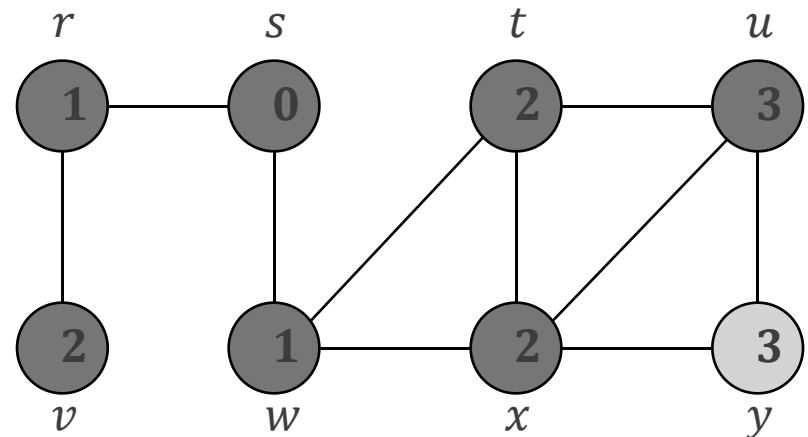
BFS(G, s)

```
1 for each vertex  $u \in V - \{s\}$  do
2    $color[u] \leftarrow \text{White}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{Gray}$ 
6  $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7 Enqueue( $Q, s$ )
8 while  $Q \neq \emptyset$  do
9    $u \leftarrow \text{Dequeue}(Q)$ 
10  for each  $v \in \text{Adj}[u]$  do
11    if  $color[v] = \text{White}$  then
12       $color[v] \leftarrow \text{Gray}$ 
13       $d[v] \leftarrow d[u] + 1$ 
14       $\pi[v] \leftarrow u$ 
15      Enqueue( $Q, v$ )
16   $color[u] \leftarrow \text{DarkGray}$ 
```

Dequeue(Q) = u

Q

y
3



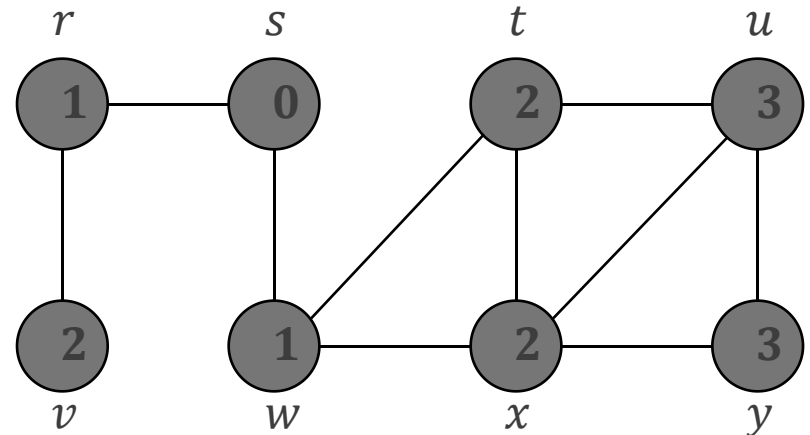


BFS(G, s)

```
1 for each vertex  $u \in V - \{s\}$  do
2    $color[u] \leftarrow \text{White}$ 
3    $d[u] \leftarrow \infty$ 
4    $\pi[u] \leftarrow \text{NIL}$ 
5  $color[s] \leftarrow \text{Gray}$ 
6  $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7 Enqueue( $Q, s$ )
8 while  $Q \neq \emptyset$  do
9    $u \leftarrow \text{Dequeue}(Q)$ 
10  for each  $v \in \text{Adj}[u]$  do
11    if  $color[v] = \text{White}$  then
12       $color[v] \leftarrow \text{Gray}$ 
13       $d[v] \leftarrow d[u] + 1$ 
14       $\pi[v] \leftarrow u$ 
15      Enqueue( $Q, v$ )
16   $color[u] \leftarrow \text{DarkGray}$ 
```

Dequeue(Q) = y

Q





Correctness of BFS

- Now, we are going to **prove the correctness of BFS**: When BFS terminates, **for all $v \in V$, $d[v]$ is the shortest path** from s to v .

证明对于所有的 $v \in V$, $d[v]$ 是 s 到 v 的最短距离的长度

Definition 8.1

The shortest path distance $\delta(s, v)$ from s to v is the minimum number of edges in any path from vertex s to vertex v ; if there is no path from s to v , then $\delta(s, v) = \infty$.

定义 $\delta(s, v)$ 是 s 到 v 最短距离，从 s 到 v 的最少边数



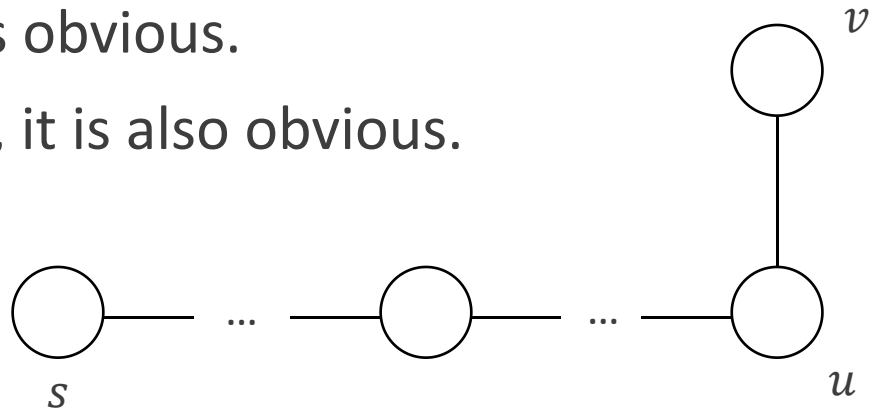
Correctness of BFS

Lemma 8.1 引理

Let $G = (V, E)$ be a directed or undirected graph, and let $s \in V$ be an arbitrary vertex. Then, **for any edge $(u, v) \in E$** ,
$$\delta(s, v) \leq \delta(s, u) + 1.$$

Proof:

- If u is **reachable** from s , it is obvious.
- If u is **not reachable** from s , it is also obvious.





Correctness of BFS

Lemma 8.2引理

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$. Then before termination, for each vertex $v \in V$, the value $d[v]$ computed by BFS satisfies $d[v] \geq \delta(s, v)$. 在BFS算法终止之前，对于任意的节点 v ， $d[v]$ 大于等于最短路径长度 $\delta(s, v)$

为什么要引入这个引理： We want to show that BFS properly computes $d[v] = \delta(s, v)$ for each vertex v . We first show that $d[v]$ bounds $\delta(s, v)$ from above.

我们要知道BFS正确的算出了 $d[v] = \delta(s, v)$ ，在这之前要证明 $d[v] \geq \delta(s, v)$

- What does this Lemma say? $d[v] \geq \delta(s, v)$: The number of edges on the path from s to v is **not less than** the shortest path from s to v , **before termination of BFS**.
- Only two cases:
 - If v is not visited, $d[v] = \infty > \delta(s, v)$.
 - If v is visited, $d[v] = \delta(s, v)$.

Easy to guess, but
how to prove?



Correctness of BFS

Proof:

We use induction on the number of Enqueue operations n . 用执行 Enqueue 运算的次数 n 进行归纳证明

- $n = 1$, $d[s] = 0 = \delta(s, s)$ and $d[v] = \infty \geq \delta(s, v)$ for all $v \in V - \{s\}$.
- $n = k$, before enqueueing v , we assume $d[u] \geq \delta(s, u)$, where v is a White neighbor of u . 依据假设
- $n = k + 1$, after enqueueing v , $d[v] = d[u] + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$. Lemma 8.1



Correctness of BFS

Lemma 8.3 引理

Suppose that **during the execution** of BFS on a graph $G = (V, E)$, the queue Q contains the vertices $v_1, v_2, \dots, v_r$, where v_1 is the head of Q and v_r is the tail. Then, $d[v_r] \leq d[v_1] + 1$ and $d[v_i] \leq d[v_{i+1}]$ for $i = 1, 2, \dots, r - 1$.

- What does this Lemma say? at all times, there are at most **two distinct d values** in the queue.
- $d[v_r] \leq d[v_1] + 1$: the difference of $d[v]$ between head v_1 and tail v_r is not greater than 1 in Q . 队列尾 $d[v_r]$ 和队列头的 $d[v_1]$ 最多多1
- $d[v_i] \leq d[v_{i+1}]$: $d[v]$ is not greater than its successor in Q . $d[v]$ 队列是按照非递减顺序的队列。



Correctness of BFS

Goal of proof:

$$(1) d[v_r] \leq d[v_1] + 1$$

$$(2) d[v_i] \leq d[v_{i+1}]$$

Q	v_1	v_2	$\dots$	v_i	v_{i+1}	$\dots$	v_r
	$d[v_1]$	$d[v_2]$		$d[v_i]$	$d[v_{i+1}]$		$d[v_r]$

Proof:

We use induction on the number of **queue operations n** . 数学归纳法，基于队列操作的次数 n

- $n = 1$, the only vertex in Q is s . Obviously $d[v_r] \leq d[v_1] + 1$, because $v_1 = v_r = s$.
- $n = k$, we assume $d[v_r] \leq d[v_1] + 1$ and $d[v_i] \leq d[v_{i+1}]$.

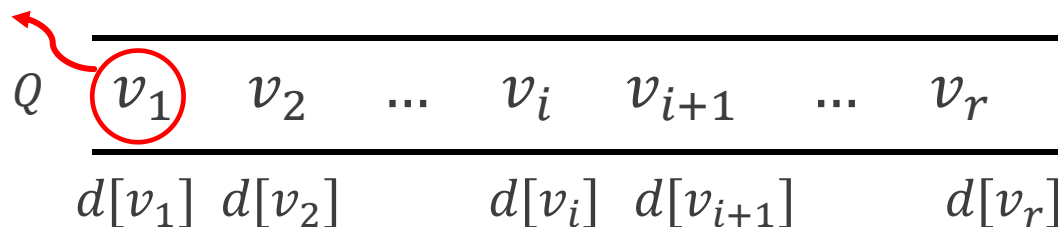


Correctness of BFS

Goal of proof:

$$(1) d[v_r] \leq d[v_1] + 1$$

$$(2) d[v_i] \leq d[v_{i+1}]$$



Proof (cont'd):

■ $n = k + 1$, consider dequeuing v_1 :

■ If the head v_1 of the queue is dequeued, v_2 becomes the new head.

■ By the inductive hypothesis, $d[v_1] \leq d[v_2]$, we have

$$d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1.$$

■ Thus, (1) is maintained after dequeue. (2) is also maintained because there is no new vertex added into Q .

For the inductive step, we must prove that the lemma holds after both dequeuing and enqueueing a vertex



Correctness of BFS

Goal of proof:

$$(1) d[v_r] \leq d[v_1] + 1$$

$$(2) d[v_i] \leq d[v_{i+1}]$$

Q	v_2	$\dots$	v_i	v_{i+1}	$\dots$	v_r	v_{r+1}
	$d[v_2]$		$d[v_i]$	$d[v_{i+1}]$		$d[v_r]$	$d[v_{r+1}]$

Proof (cont'd):

- $n = k + 1$, after dequeuing v_1 , consider enqueueing v_{r+1} :

- By BFS, we have $d[v_{r+1}] = d[v_1] + 1$.

- By the inductive hypothesis, $d[v_1] \leq d[v_2]$, we have

$$d[v_{r+1}] = d[v_1] + 1 \leq d[v_2] + 1.$$

Thus, (1) is proved because v_2 and v_{r+1} are the new head and tail of Q .

- By the inductive hypothesis, $d[v_r] \leq d[v_1] + 1$, we have

$$d[v_r] \leq d[v_1] + 1 = d[v_{r+1}].$$

Thus, (2) is maintained when v_{r+1} is enqueued.



Introduction to Algorithms对引理8.3的证明

Proof The proof is by induction on the number of queue operations. Initially, when the queue contains only s , the lemma certainly holds.

For the inductive step, we must prove that the lemma holds after both dequeuing and enqueueing a vertex. If the head v_1 of the queue is dequeued, v_2 becomes the new head. (If the queue becomes empty, then the lemma holds vacuously.) By the inductive hypothesis, $d[v_1] \leq d[v_2]$. But then we have $d[v_r] \leq d[v_1] + 1 \leq d[v_2] + 1$, and the remaining inequalities are unaffected. Thus, the lemma follows with v_2 as the head.

Enqueueing a vertex requires closer examination of the code. When we enqueue a vertex v in line 17 of BFS, it becomes v_{r+1} . At that time, we have already removed vertex u , whose adjacency list is currently being scanned, from the queue Q , and by the inductive hypothesis, the new head v_1 has $d[v_1] \geq d[u]$. Thus, $d[v_{r+1}] = d[v] = d[u] + 1 \leq d[v_1] + 1$. From the inductive hypothesis, we also have $d[v_r] \leq d[u] + 1$, and so $d[v_r] \leq d[u] + 1 = d[v] = d[v_{r+1}]$, and the remaining inequalities are unaffected. Thus, the lemma follows when v is enqueued.



Correctness of BFS

Corollary 8.1 推论

Suppose that vertices v_i and v_j are enqueued during the execution of BFS, and that v_i is enqueued before v_j . Then $d[v_i] \leq d[v_j]$ at the time that v_j is enqueued. 单调递增

- What does this Corollary say? We have $d[v_i] \leq d[v_j]$, if v_i is in front of v_j in Q .
 - It can be easily proved by $d[v_i] \leq d[v_{i+1}]$ for $i = 1, 2, \dots, r - 1$ in Lemma 8.3.



Correctness of BFS

Theorem 8.1 (Correctness of BFS)

Let $G = (V, E)$ be a directed or undirected graph, and suppose that BFS is run on G from a given source vertex $s \in V$.

- (1) When BFS terminates, $d[v] = \delta(s, v)$ for all $v \in V$.
- (2) For any vertex $v \neq s$ that is reachable from s , one of the shortest paths from s to v is a shortest path from s to $\pi[v]$ followed by the edge $(\pi[v], v)$.

■ What does the Theorem say?

- (1) When BFS terminates, the number of edges from s to v is **the shortest path distance**.
- (2) **One of the shortest path** from s to v must **go through v 's parent $\pi[v]$** .



Correctness of BFS

假设 v 是所有 $d[v] \neq \delta(s, v)$ 里面 $\delta(s, v)$ 最短的

Proof of (1):

- We use **proof by contradiction**: v be the vertex with minimum $\delta(s, v)$ that receives such an incorrect d value; $d[v] \neq \delta(s, v)$
- By Lemma 8.2, $d[v] \geq \delta(s, v)$, and thus we have $d[v] > \delta(s, v)$.
- Let u be the vertex immediately preceding v on a shortest path from s to v , so that $\delta(s, v) = \delta(s, u) + 1$.
- By the choice of v , there must **exists a u** that $d[u] = \delta(s, u)$. 因为 v 是所有 $d[v] \neq \delta(s, v)$ 里面 $\delta(s, v)$ 最短的，所以对于 v 之前的 u ，他是正确的。
- Thus, we have

$$d[v] > \delta(s, v) = \delta(s, u) + 1 = d[u] + 1.$$

(接下来证明与这个式子矛盾)



Correctness of BFS

考虑当 u 出队列时, 3种 v 的颜色

Proof of (1) (cont'd):

- Base on the fact that $d[v] > d[u] + 1$, we consider the cases when u is dequeued:
 - If v is White, then Line 15 sets $d[v] = d[u] + 1$.
 - If v is DarkGray, then it was already removed from the queue and, by Corollary 8.1, we have $d[v] \leq d[u]$.
 - If v is Gray, then it was painted gray upon dequeuing some vertex w , which was removed from Q earlier than u and for which $d[v] = d[w] + 1$ and $d[w] \leq d[u]$ by Corollary 8.1. So we have $d[v] \leq d[u] + 1$.
- All three cases show contradiction to $d[v] > d[u] + 1$. Therefore, the assumption fails. 这三种情况都和 $d[v] > d[u] + 1$ 矛盾



Correctness of BFS

(2) For any vertex $v \neq s$ that is reachable from s , one of the shortest paths from s to v is a shortest path from s to $\pi[v]$ followed by the edge $(\pi[v], v)$.

Proof of (2):

- If $\pi[v] = u$, we have $\delta(s, u) = d[u]$ and $\delta(s, v) = d[v]$.
Therefore, we get $d[v] = d[u] + 1$.
- We can thus find a shortest path from s to u and from u to v .



Breadth-First Tree 宽度优先树

宽度优先生成树

- For a graph $G = (V, E)$ with source s , we define the **predecessor subgraph of G** as $G_\pi = (V_\pi, E_\pi)$, where
$$V_\pi = \{v \in V: \pi[v] \neq \text{NIL}\} \cup \{s\},$$
$$E_\pi = \{(\pi[v], v): v \in V_\pi - \{s\}\}.$$
- The predecessor subgraph G_π is a **breadth-first tree** if V_π consists of **the vertices reachable** from s and, for all $v \in V_\pi$, there is a **unique simple path** from s to v in G_π that is also a shortest path from s to v in G .
- Breadth-first tree is a spanning tree. (宽度优先生成树)



Computational Cost of BFS 计算复杂度

- The for loop in **Line 1** runs $|V|$ times.
- In the worst case, the while loop in **Line 8** runs $|V|$ times and the for loop in **Line 10** runs $|E|$ times.
- 粗粗的看，时间复杂度是 $O(|V||E|)$ 对不对？
- Will this worst case happen?
Think of using amortized analysis. 用合计分析法来分析这种情况对不对

```
BFS( $G, s$ )
1  for each vertex  $u \in V - \{s\}$  do
2       $color[u] \leftarrow \text{White}$ 
3       $d[u] \leftarrow \infty$ 
4       $\pi[u] \leftarrow \text{NIL}$ 
5   $color[s] \leftarrow \text{Gray}$ 
6   $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7  Enqueue( $Q, s$ )
8  while  $Q \neq \emptyset$  do
9       $u \leftarrow \text{Dequeue}(Q)$ 
10     for each  $v \in Adj[u]$  do
11         if  $color[v] = \text{White}$  then
12              $color[v] \leftarrow \text{Gray}$ 
13              $d[v] \leftarrow d[u] + 1$ 
14              $\pi[v] \leftarrow u$ 
15             Enqueue( $Q, v$ )
16      $color[u] \leftarrow \text{DarkGray}$ 
```



Computational Cost of BFS

- There are at **most $|V|$ vertices** in Q .
- For **each vertex**, for loop runs **at most $Adj[u]$ times**.
- Total cost from Line 8 to Line 16:
$$\sum_{u \in V} |Adj[u]| = O(|E|).$$
- Total cost for BFS: **$O(|V| + |E|)$** .

```
BFS( $G, s$ )
1  for each vertex  $u \in V - \{s\}$  do
2       $color[u] \leftarrow \text{White}$ 
3       $d[u] \leftarrow \infty$ 
4       $\pi[u] \leftarrow \text{NIL}$ 
5   $color[s] \leftarrow \text{Gray}$ 
6   $d[s] \leftarrow 0$ ;  $\pi[s] \leftarrow \text{NIL}$ ;  $Q \leftarrow \emptyset$ 
7  Enqueue( $Q, s$ )
8  while  $Q \neq \emptyset$  do
9       $u \leftarrow \text{Dequeue}(Q)$ 
10     for each  $v \in Adj[u]$  do
11         if  $color[v] = \text{White}$  then
12              $color[v] \leftarrow \text{Gray}$ 
13              $d[v] \leftarrow d[u] + 1$ 
14              $\pi[v] \leftarrow u$ 
15             Enqueue( $Q, v$ )
16      $color[u] \leftarrow \text{DarkGray}$ 
```



Print Path of BFS

```
PrintPath( $G, s, v$ )  
1  if  $v = s$  then  
2      print  $s$   
3  else if  $\pi[v] = \text{NIL}$  then  
4      print "no path from"  $s$  "to"  $v$  "exists"  
5      else PrintPath( $G, s, \pi[v]$ )  
6          print  $v$ 
```



GRAPH SEARCH

DEPTH-FIRST SEARCH 深度优先搜索

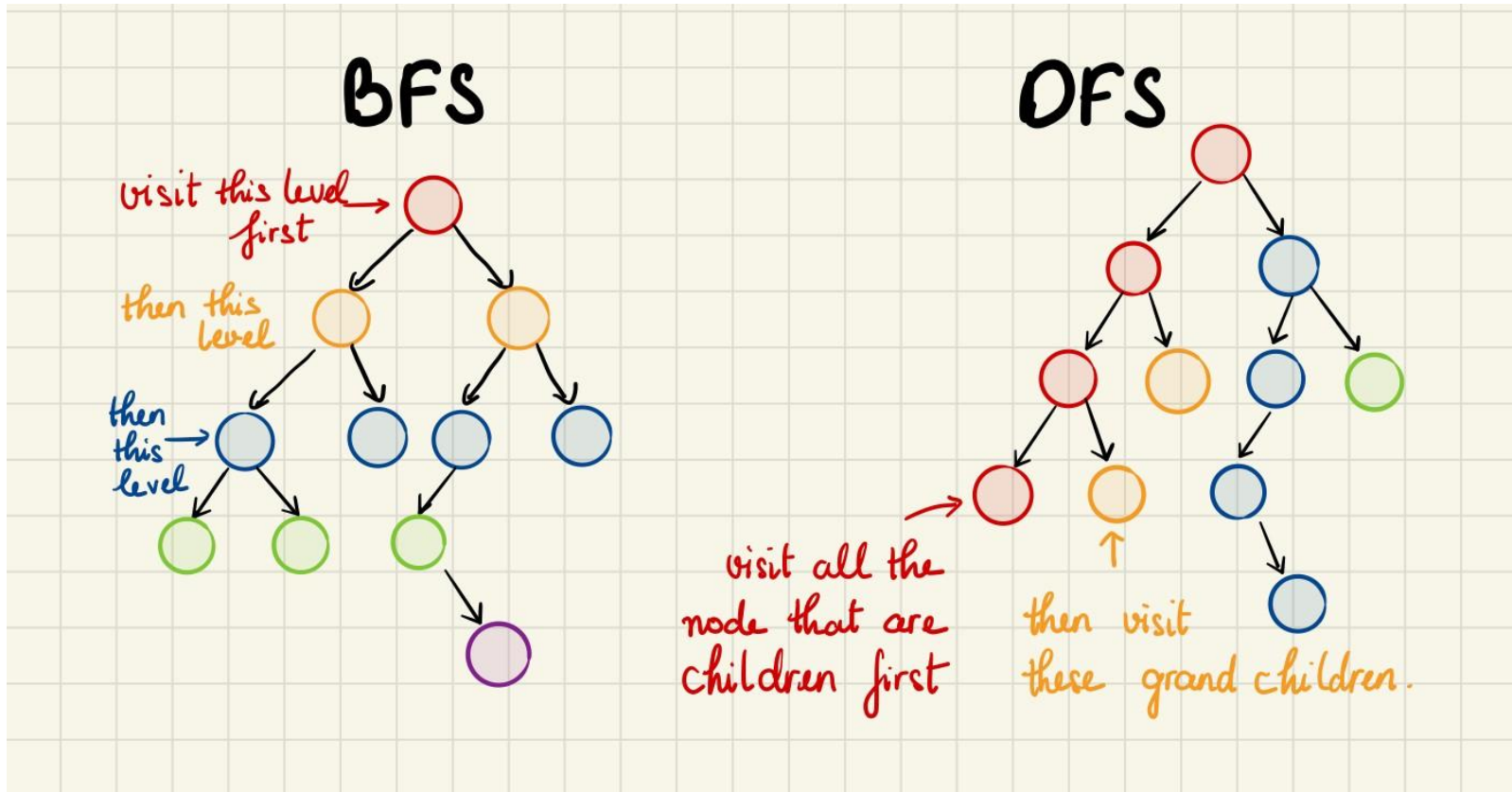


Depth-First Search

- Depth-first search (DFS) (深度优先搜索) is a recursive algorithm, which starts from a vertex v and recursively call to all v 's neighbors until all vertices are visited.
- DFS timestamps each vertex. Each vertex v has two timestamps:
 - The first timestamp $d[v]$ records when v is first discovered.
 - The second timestamp $f[v]$ records when the search finishes checking v 's neighbors.



BFS vs. DFS





Depth-First Search

DFS(G)

```
1  for each vertex  $u \in V$  do
2       $color[u] \leftarrow \text{White}$ 
3       $\pi[u] \leftarrow \text{NIL}$ 
4   $time \leftarrow 0$ 
5  for each vertex  $u \in V$  do
6      if  $color[u] = \text{White}$  then
7          DFSVisit( $u$ )
```

This DFS function search the whole graph, while the previous version of BFS only search from a source vertex s .

DFSVisit(u)

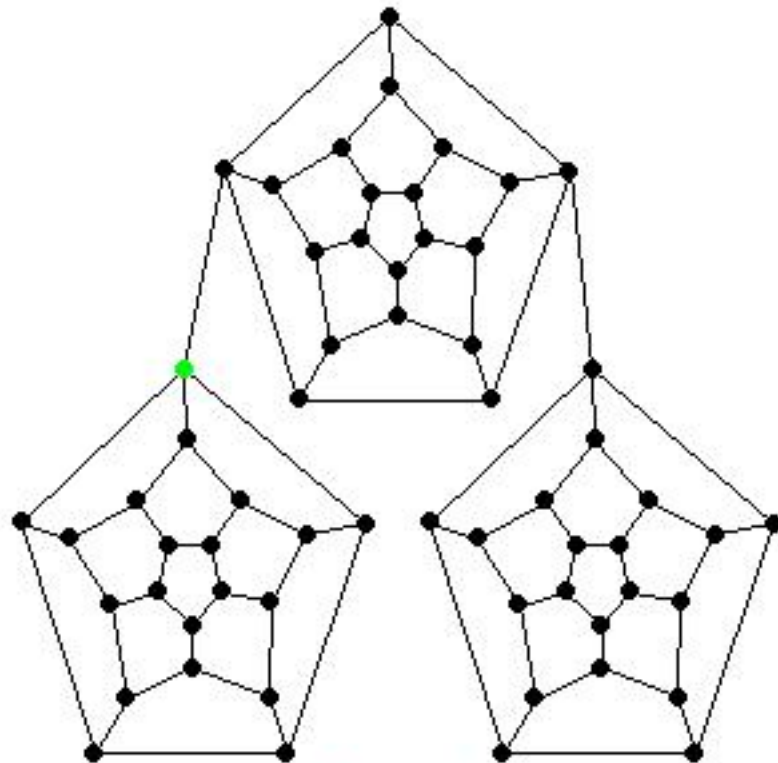
```
1   $color[u] \leftarrow \text{Gray}$ 
2   $time \leftarrow time + 1$ 
3   $d[u] \leftarrow time$ 
4  for each  $v \in Adj[u]$  do
5      if  $color[v] = \text{White}$  then
6           $\pi[v] \leftarrow u$ 
7          DFSVisit( $v$ )
8   $color[u] \leftarrow \text{DarkGray}$ 
9   $time \leftarrow time + 1$ 
10  $f[u] \leftarrow time$ 
```

Similar to the analysis of BFS, the computational complexity is $O(|V| + |E|)$.



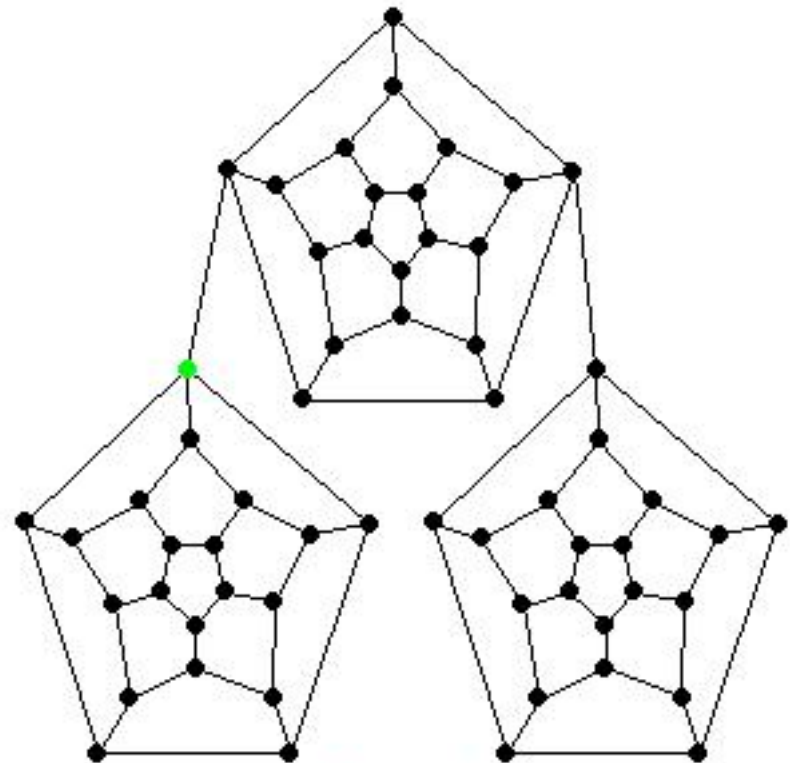
BFS vs. DFS

Breadth-First Search



www.combinatorica.com

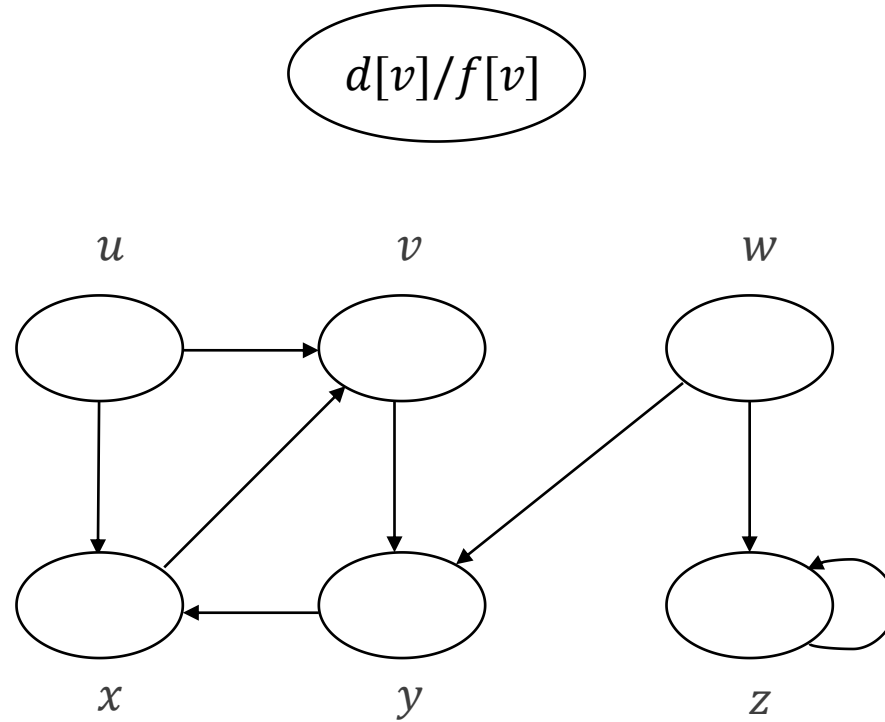
Depth-First Search



www.combinatorica.com

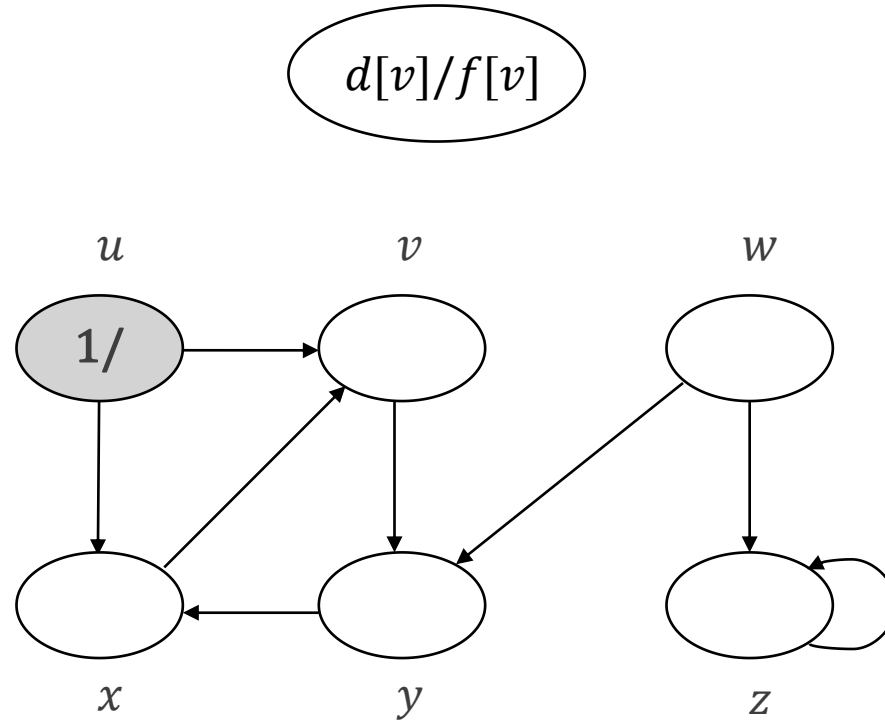


Depth-First Search



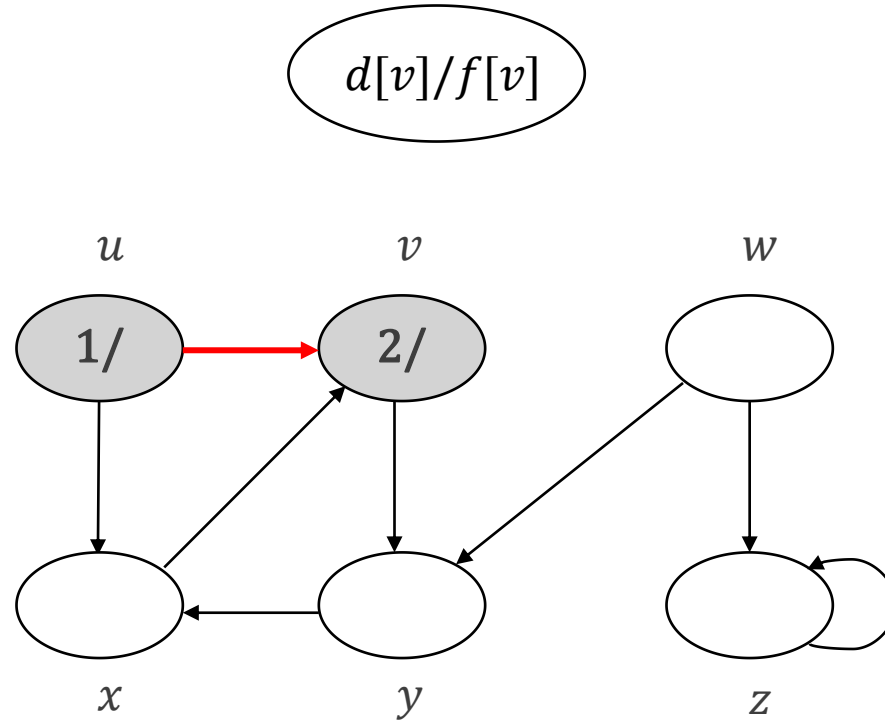


Depth-First Search



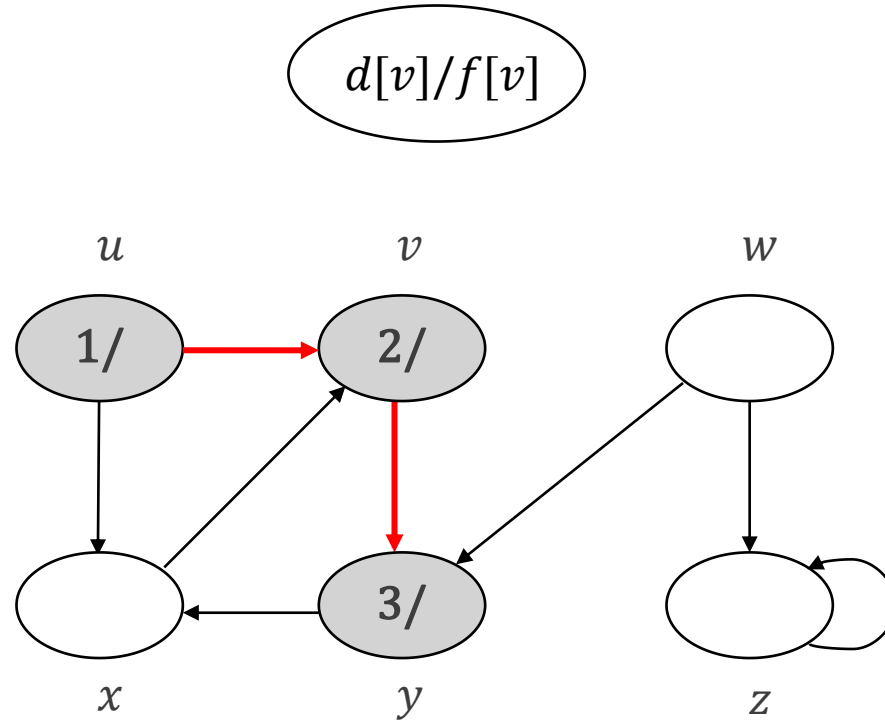


Depth-First Search



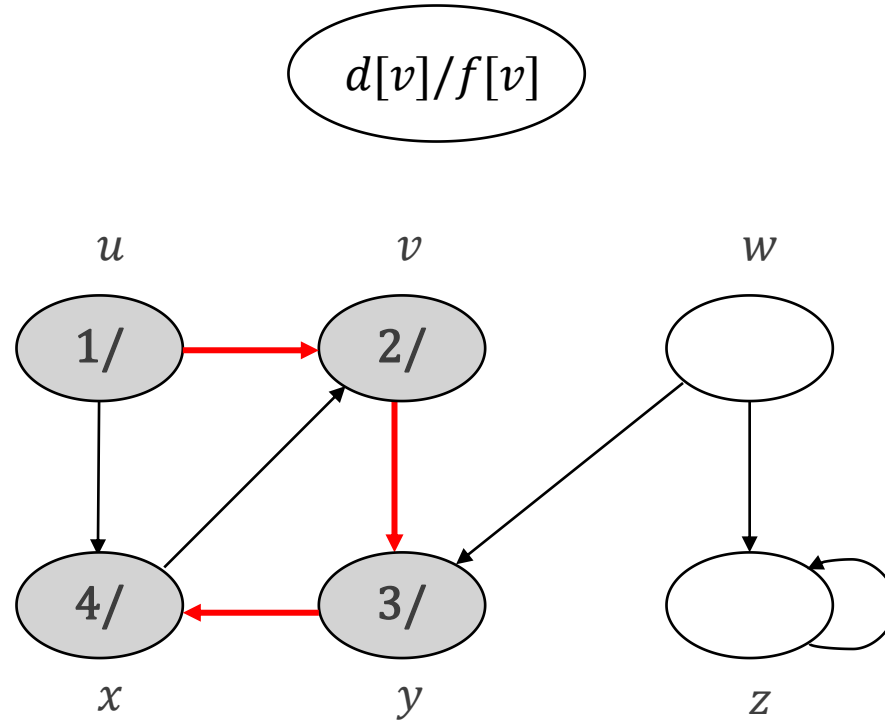


Depth-First Search



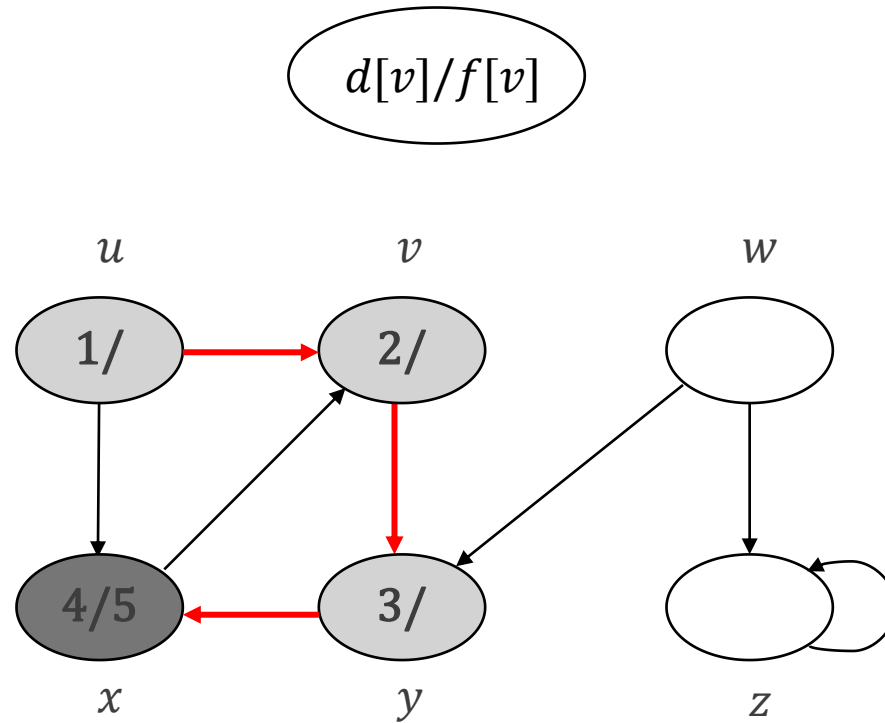


Depth-First Search



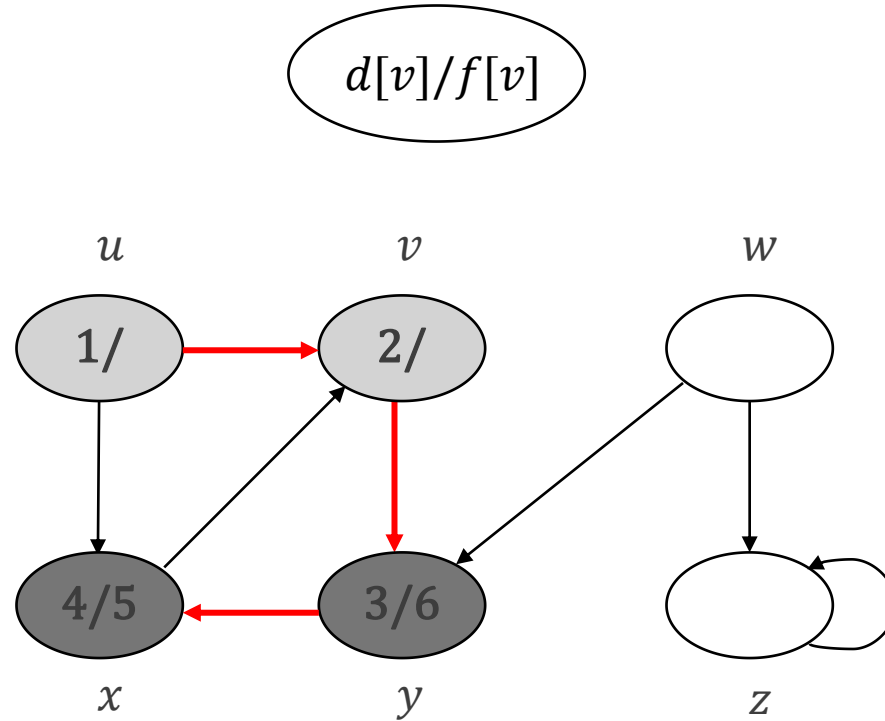


Depth-First Search



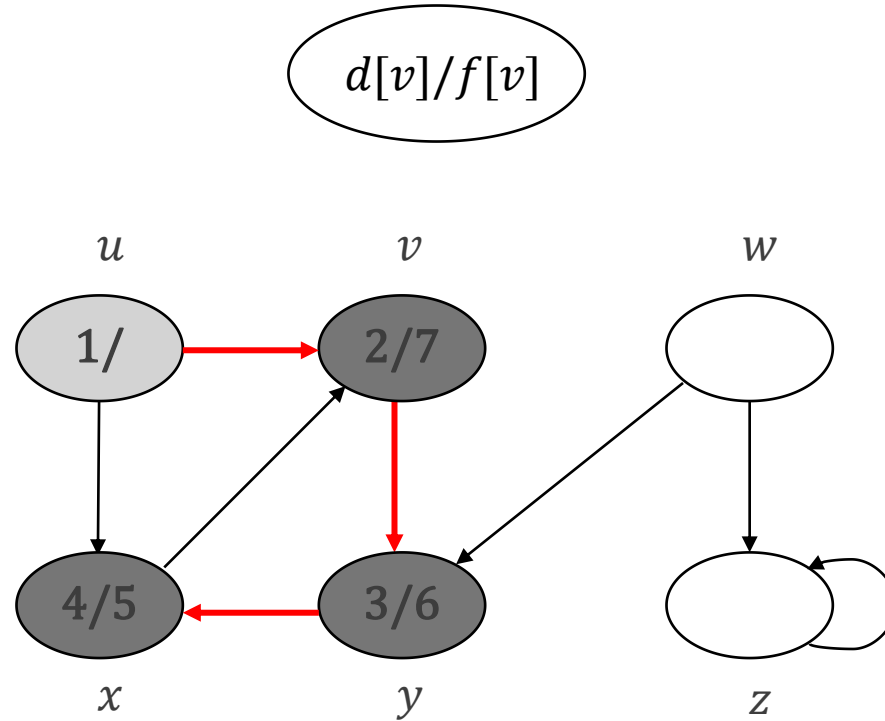


Depth-First Search



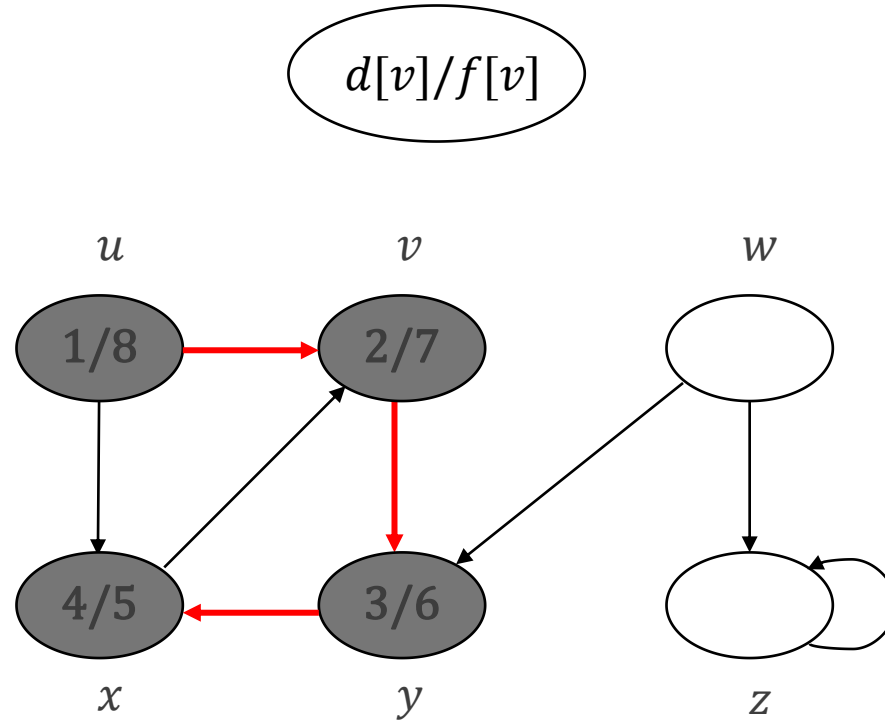


Depth-First Search



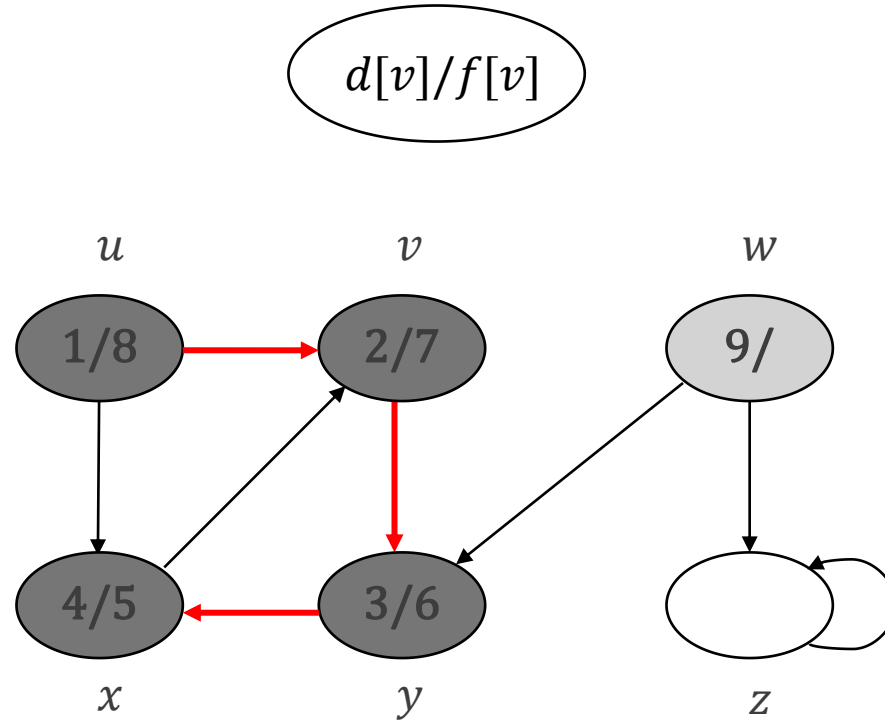


Depth-First Search



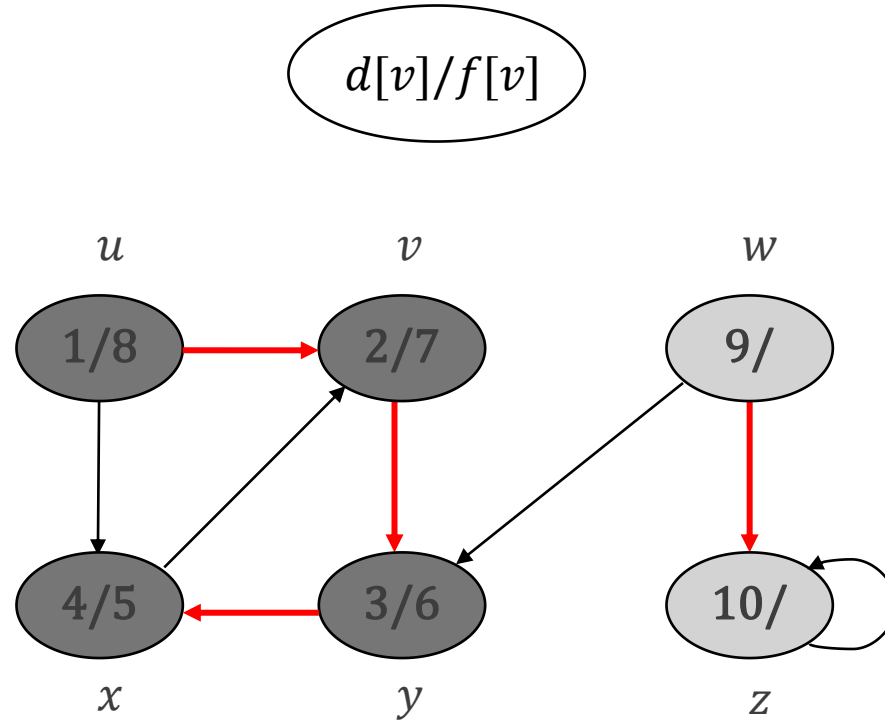


Depth-First Search



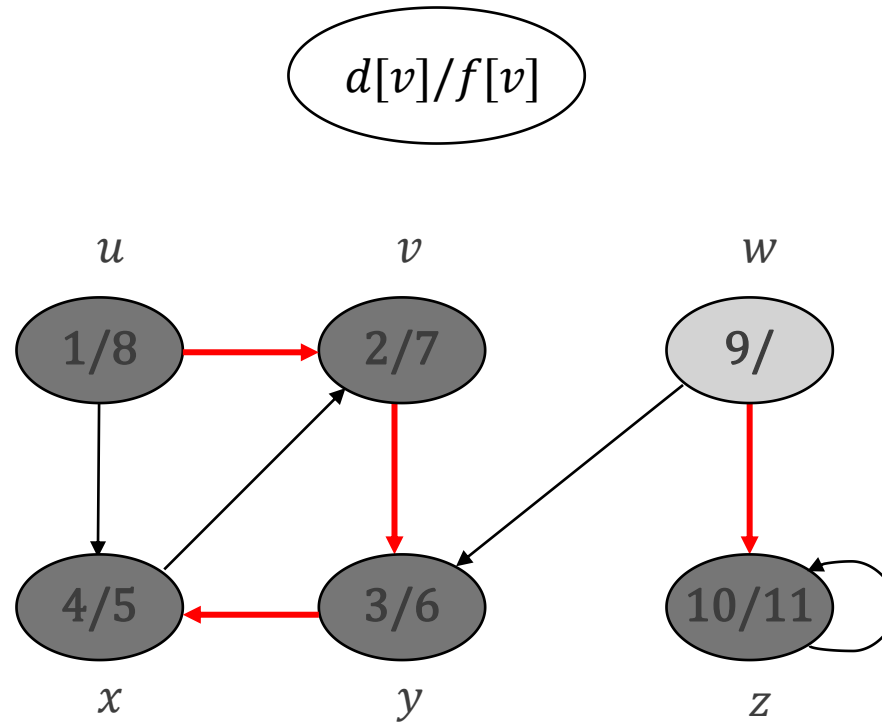


Depth-First Search



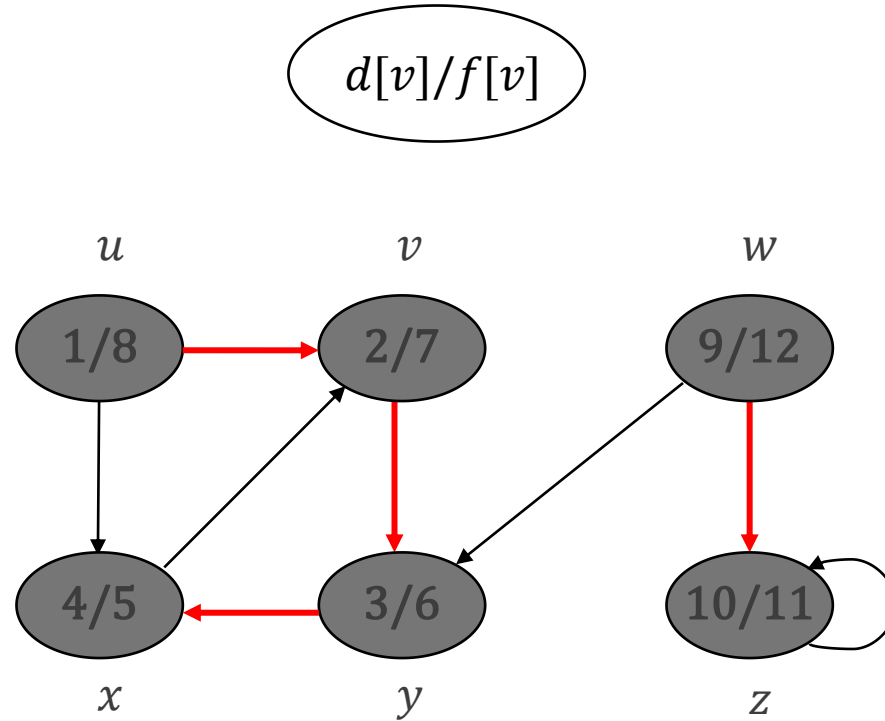


Depth-First Search





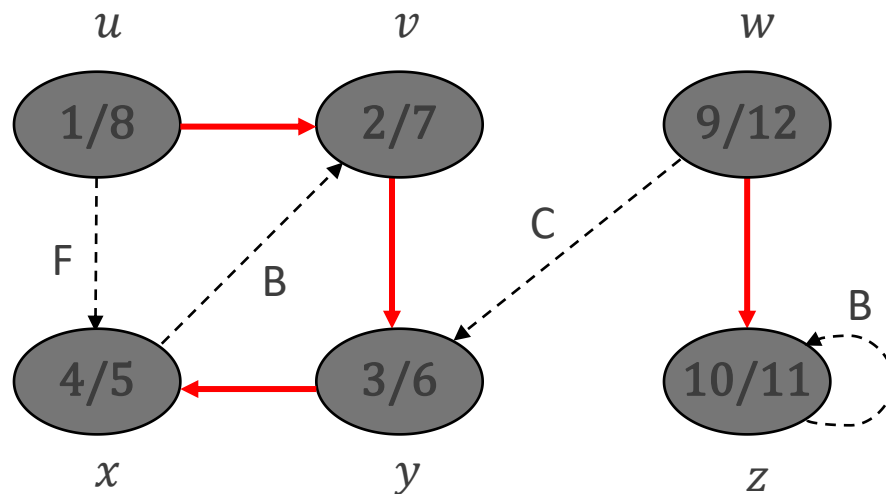
Depth-First Search





Properties of DFS

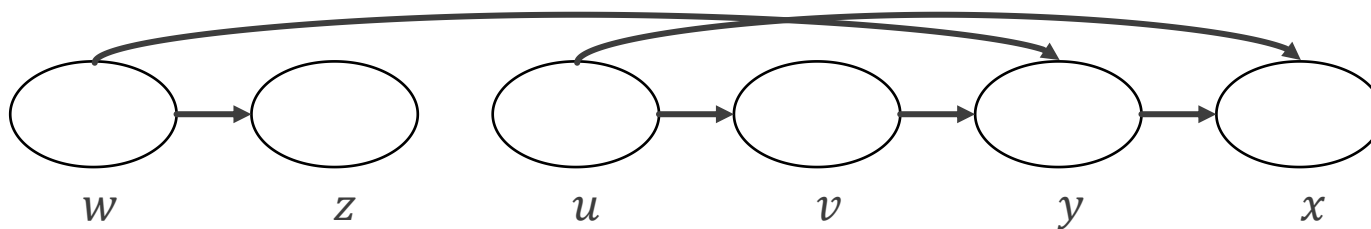
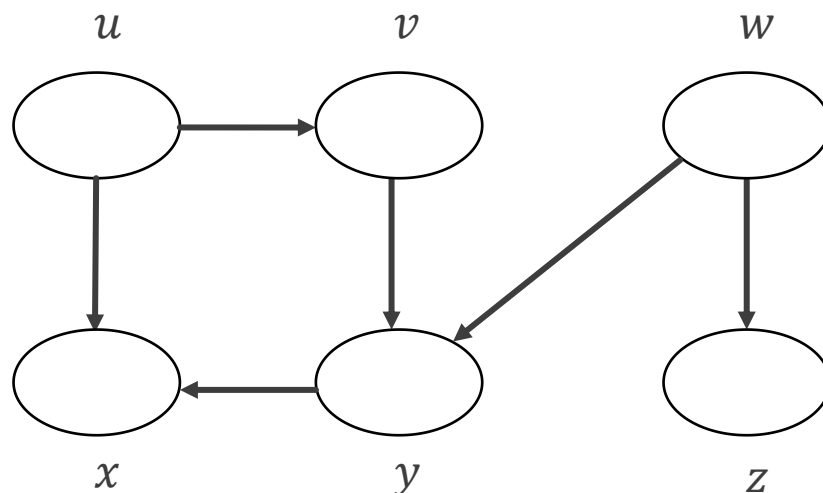
- The search can be used to classify the edges E into **four edges**:
 - **Tree edges**: Edges in depth-first trees. 在深度搜索树中的边
 - **Back edges (B)**: Not in tree edges, but points to its ancestor vertex or itself. 指向父辈节点的边
 - **Forward edges (F)**: Not in tree edges, but points to its descendant vertex. 指向子孙节点的边
 - **Cross edges (C)**: Others.





DFS应用: Topological Sort 拓扑排序

- A topological sort (拓扑排序) of a **directed acyclic graph (DAG)** (有向无环图) $G = (V, E)$ is a **linear ordering** of all its vertices such that if G contains an edge (u, v) , then u appears before v in the ordering.





Application of DFS: Topological Sort

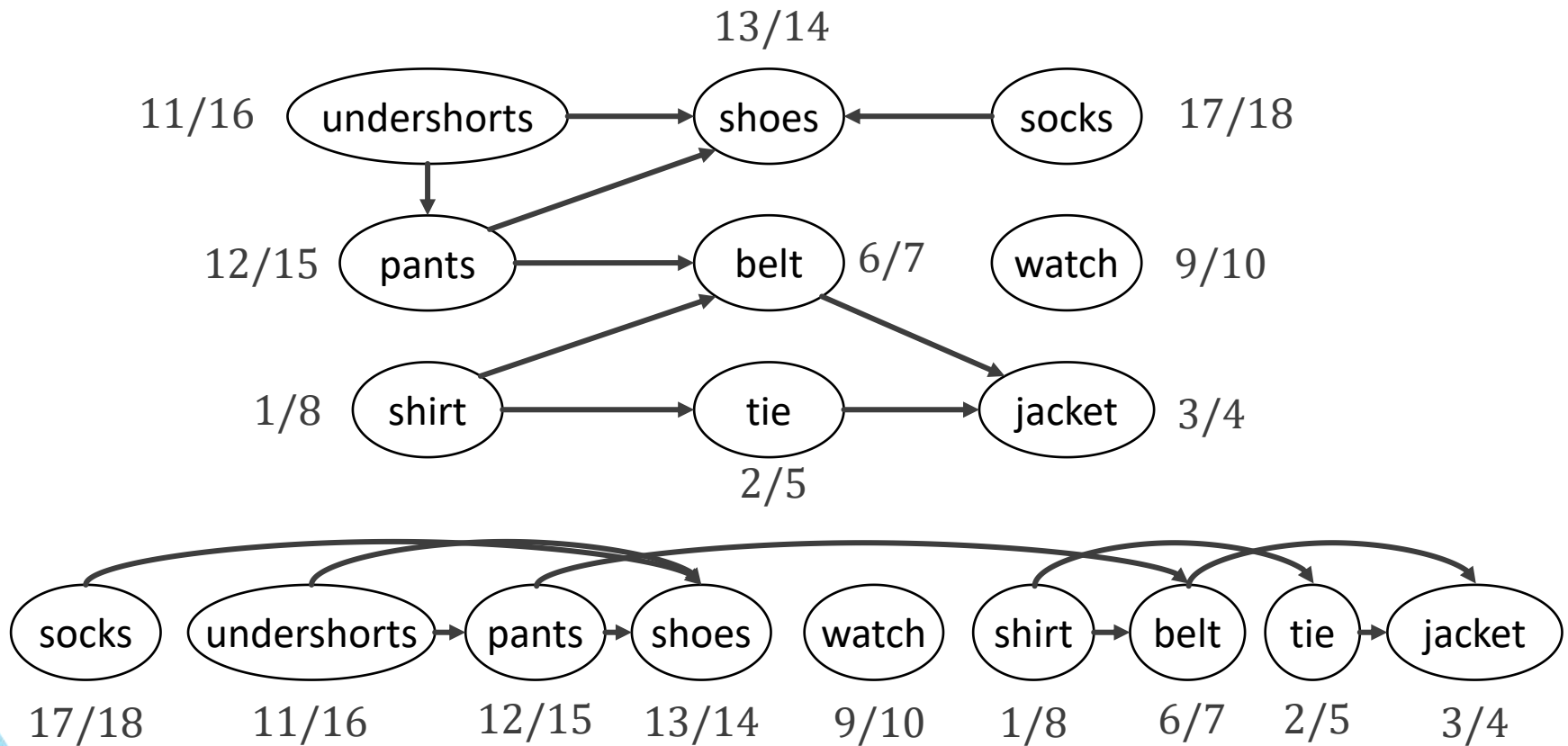
TopologicalSort(G)

- 1 调用 DFS(G) 计算每个节点 v 的完成时间 $f[v]$
- 2 每个节点遍历完后, 按照 $f[v]$ 大小, 从最小的开始, 从小到大的顺序插入到链表的前面
- 3 **return** the linked list of vertices



Application of DFS: Topological Sort

书上给了一个穿衣服的例子，也可以大学上课，各个课程之间的依赖关系建图。之后排序，求出各个课程之间的拓扑关系，肯定是大一的课程都排在前面





DFS的应用: 强连通分支

- A **connected component (连通分支)** of a directed graph $G = (V, E)$ is **a set of vertices $C \subseteq V$** such that for **every pair** of vertices **u and v** in C , u and v are **reachable** from each other.
- A **strongly connected component (强连通分支)** is **the maximal one** among all connected components.
- The **transpose graph (转置图)** of G is the graph $G^T = (V, E^T)$, where $E^T = \{(u, v): (v, u) \in E\}$. That is, E^T consists of the edges of G with their **directions reversed**.



DFS的应用: 强连通分支

StrongConnectedComponents(G)

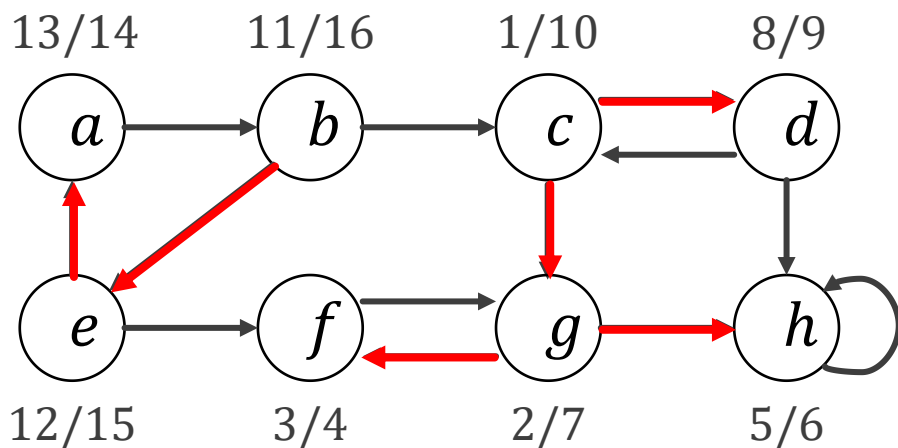
- 1 调用DFS(G)计算每个顶点 u 的结束时间 $f[u]$
- 2 计算图的倒置 G^T
- 3 call DFS(G^T), but in the main loop of DFS, 从 $f[u]$ 从大到小遍历 (as computed in line 1)
- 4 输出: 将在第三行得到的深度优先搜索树输出, 就是一个个强连通分支。第一个输出的强连通分支就是最大强连通分支

DFS(G)

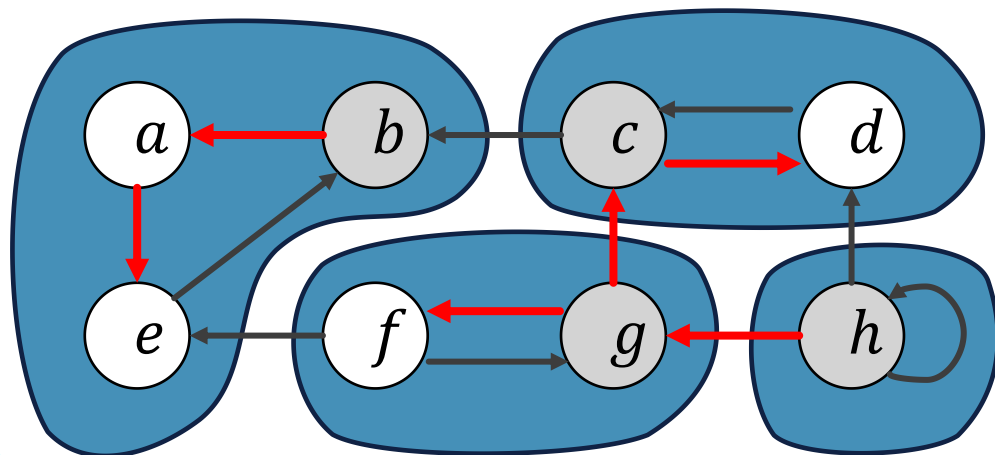
- 1 **for** each vertex $u \in V$ **do**
- 2 $color[u] \leftarrow \text{White}$
- 3 $\pi[u] \leftarrow \text{NIL}$
- 4 $time \leftarrow 0$
- 5 **for** each vertex $u \in V$ **do**
- 6 **if** $color[u] = \text{White}$ **then**
- 7 DFSVisit(u)



DFS的应用: 强连通分支



First DFS run.



Second DFS run. Each tree is a connected component with **gray vertex as the root**. The first tree is the strongly connected components.



MINIMUM SPANNING TREE

最小生成树



Minimum Spanning Tree

- Given a **connected, undirected** graph $G = (V, E)$, for each edge $(u, v) \in E$, we have a **weight** $w(u, v)$ specifying the cost to connect u and v .
- An acyclic subset $T \subseteq E$ is called **minimum spanning tree (MST)** (最小生成树). It connects **all of the vertices** and whose total **weight is minimized**:

$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

- Next, we introduce **two greedy algorithms**: Kruskal's algorithm (克鲁斯卡尔算法) and Prim's algorithm (普里姆算法).



MINIMUM SPANNING TREE

KRUSKAL'S ALGORITHM 克鲁斯卡尔算法

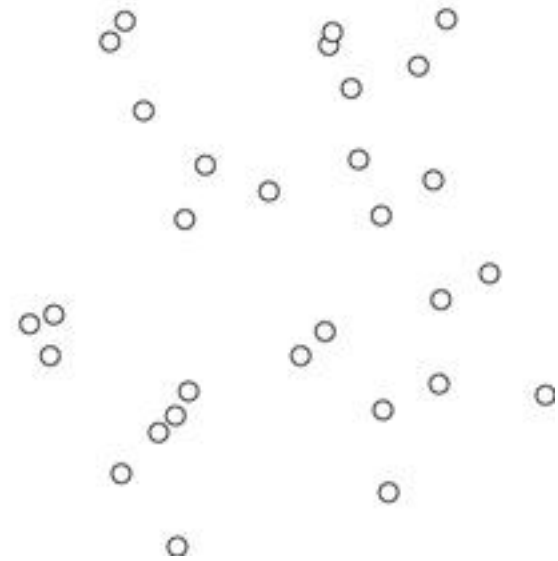


Kruskal's Algorithm

High level pseudocode:

不停的合并顶点，同时存在多个子树

- $A = \emptyset$.
- Sort the edges in E in **nondecreasing** order. (非递减)
- Iterate over each e in E .
 - If $A \cup e$ do not form a cycle, $A = A \cup e$. 如果加入 e 之后 A 不有回路，就加入 e



A demo for Kruskal's algorithm on a complete graph with weights based on Euclidean distance.



Pseudocode

KruskalMST(G, w)

```

1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V$  do
3   MakeSet( $v$ )
4 sort the edges of  $E$  by weight  $w$ 
5 for each edge  $(u, v) \in E$  do
6   if FindSet( $u$ )  $\neq$  FindSet( $v$ )
7   then
8      $A \leftarrow A \cup \{(u, v)\}$ 
9     Union( $u, v$ )
9 return  $A$ 
  
```

$O(|V|)$

Total: $O(|E| \lg |E|)$

$O(|E| \lg |E|)$

$O(|E| \lg |E|)$



FindSet and Union
are both $O(\lg |E|)$

MakeSet(v) 以 v 创建一个集合

FindSet(u) 找到 u 所在的集合的代表元素

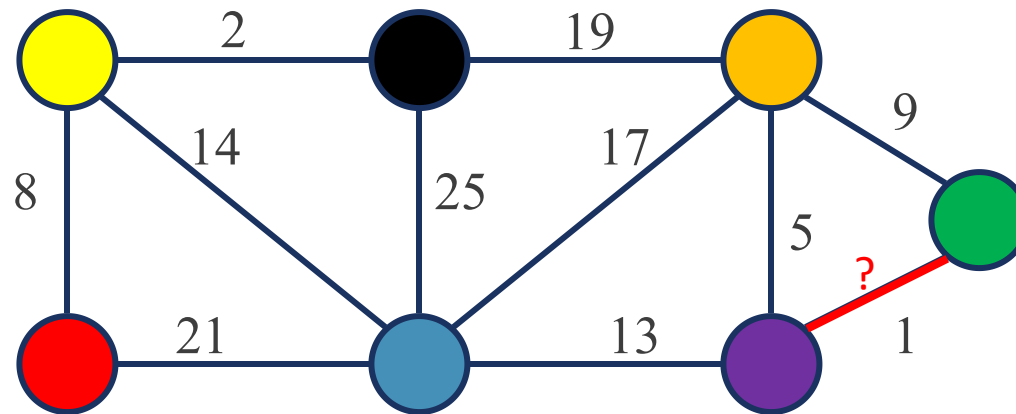
Union(u, v) 将 u, v 各自所在的两个集合合并

Disjoint-set forests 实现的速度最快 参考链接 <https://blog.csdn.net/a130737/article/details/38438531>

Disjoint-set 并查集

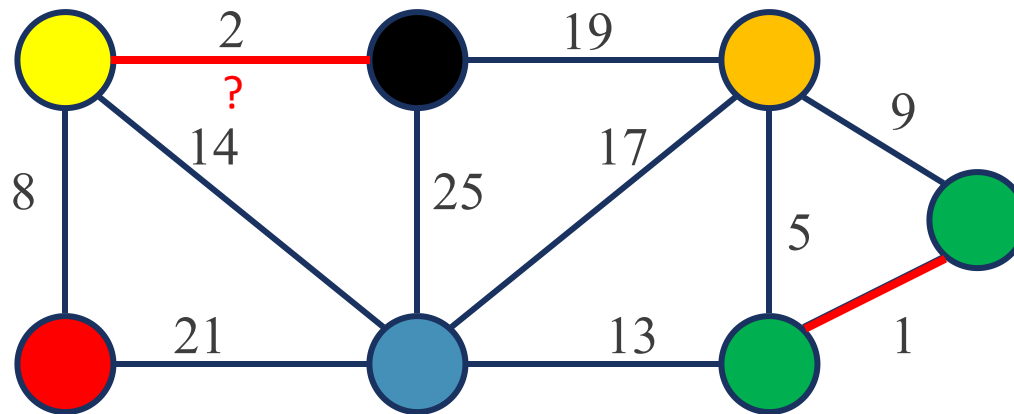


Example



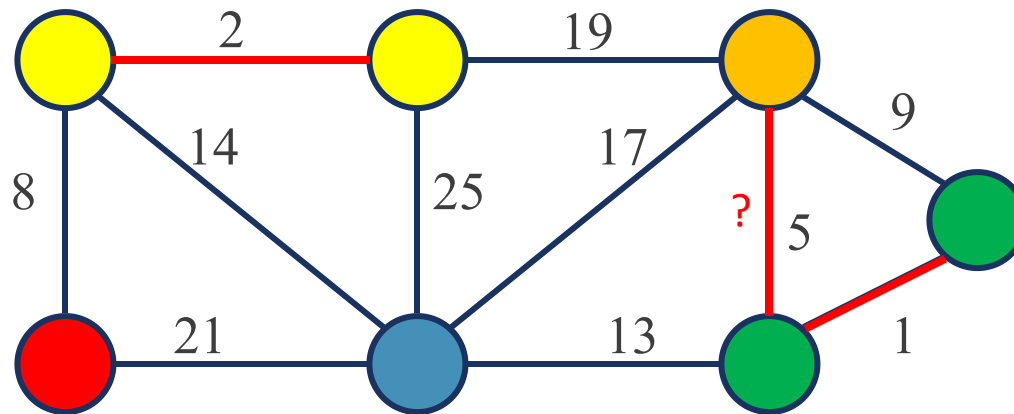


Example



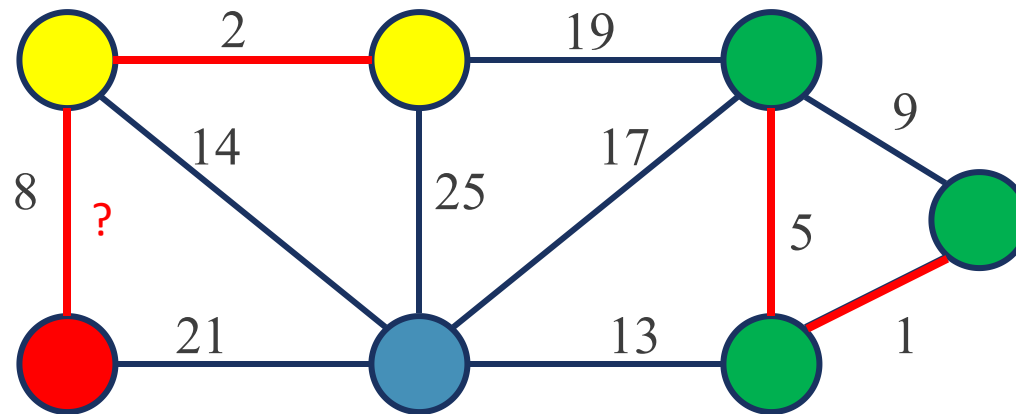


Example



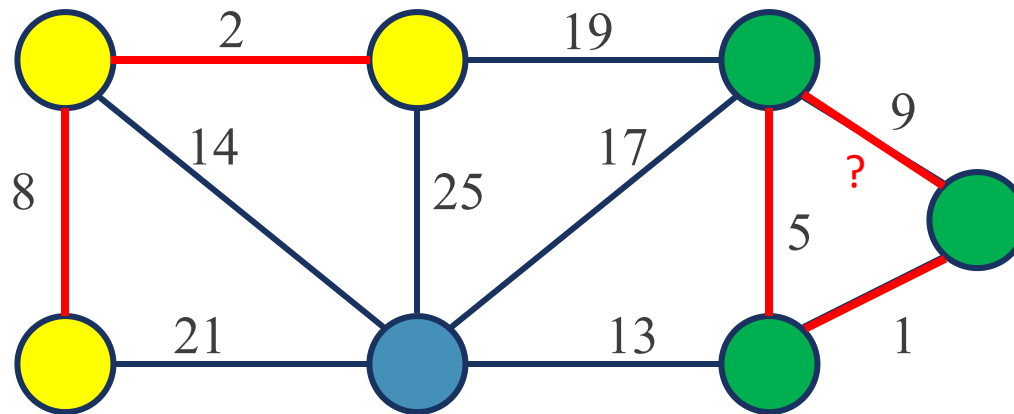


Example



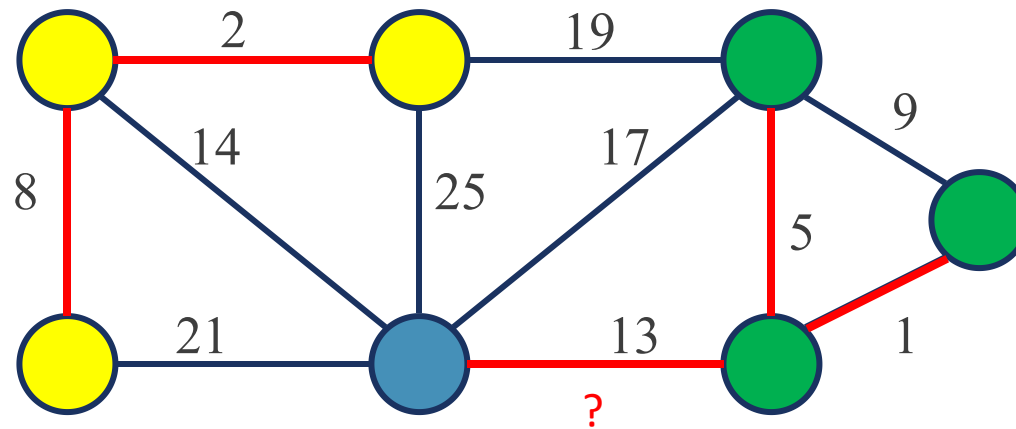


Example



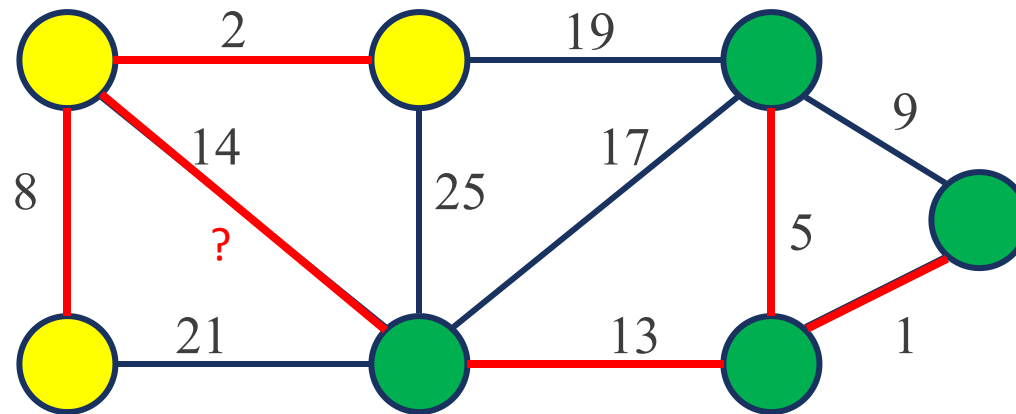


Example



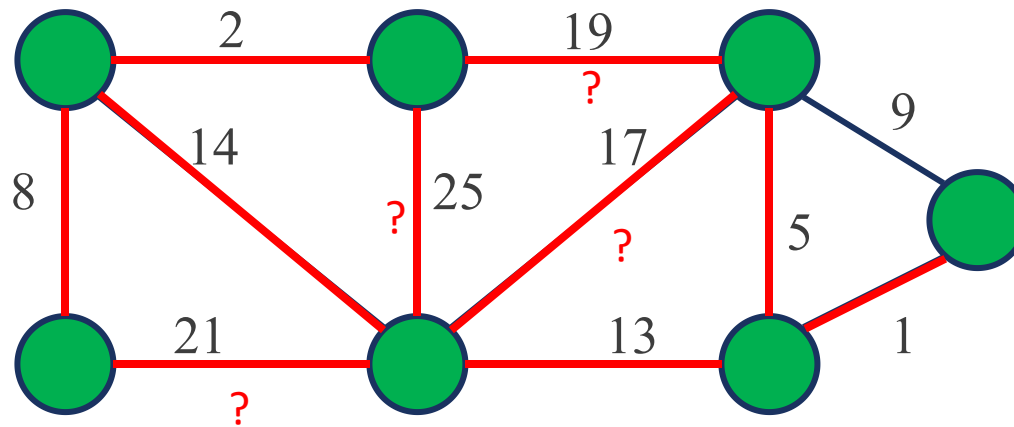


Example





Example





Correctness Proof of Kruskal's Algorithm

证明Kruskal算法正确的创建了一颗最小生成树
循环不变量，每次迭代时 A 是某最小生成树的子集

Theorem 8.4

Algorithm KruskalMST correctly finds a minimum spanning tree in a weighted connected undirected graph.

Proof:

- We prove that the edge selected by the greedy choice at each step must be in a MST.
- Let T a minimum spanning tree and A^* is the set of edges in T .
- We add an edge to set A at each step. We need to prove the loop invariant $A \subseteq A^*$ at each step.



证明循环不变量Maintenance的思路

- 0) 假设之前加入的A都是某个最小生成树的子集，要证明的是加入 (u,v) 之后，A和 (u,v) 还是某个最小生成树的子集
- 1) 如果 (u,v) 属于某个最小生成树 A^* ，那么显然得证
- 2) 对于任意一个最小生成树 A^* ，如果 (u,v) 不属于 A^* ，那么就将 A^* 改造成成为包含有 (u,v) 的最小生成树， (u,v) 合并A也属于最小生成树。
 - 2.1) 证明 $A^* \cup (u,v)$ 会有回路
 - 2.2) 找到回路上的一条边 (x,y) ，连接u所属的节点集合X和 $S-X$
 - 2.3) 证明 $w(x,y) \geq w(u,v)$
 - 2.4) 将 (x,y) 替换成 (u,v) 得到新的生成树 A^{**} ， A^{**} 是最小生成树
 - 2.5) 按照kruskal算法的构造过程 (u,v) 合并A不会造成环，并且 (u,v) 合并A属于某个最小生成树
 - 2.6) 得证



Correctness Proof of Kruskal's Algorithm

Proof (cont'd):

Initialization: (归纳基础)

- After line 1, the set $A = \emptyset$ trivially **satisfies** the loop invariant $A \subseteq A^*$.

Maintenance: (归纳步骤)

- **Induction hypothesis** : Before adding the edge (u, v) to A , $A \subseteq A^*$.
- We will show that after adding the edge (u, v) to A , e.g. $A' = A \cup (u, v)$, $A' \subseteq A^*$.
- By the induction hypothesis, $A \subseteq A^*$. If A^* contains (u, v) , then there is nothing to prove because obviously $A \cup (u, v) \subseteq A^*$.

如果 A^* 包含 (u, v) , 那么显然循环不变量 $A \cup (u, v) \subseteq A^*$ 成立



证明循环不变量Maintenance的思路

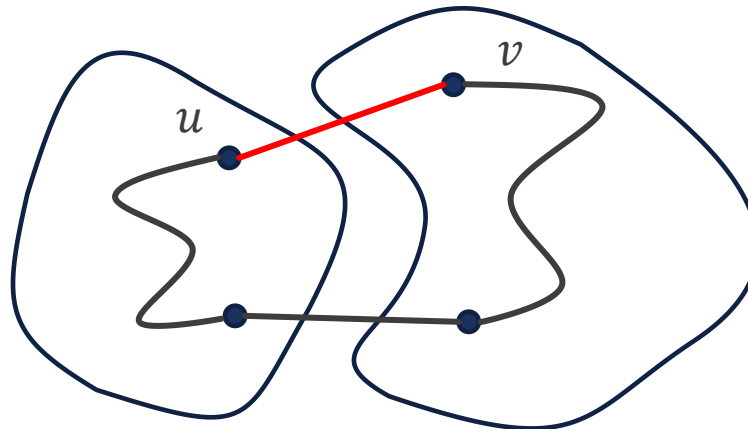
- 0) 假设之前加入的A都是某个最小生成树的子集，要证明的是加入 (u, v) 之后，A和 (u, v) 还是某个最小生成树的子集
- 1) 如果 (u, v) 属于某个最小生成树 A^* ，那么显然得证
- 2) 对于任意一个最小生成树 A^* ，如果 (u, v) 不属于 A^* ，那么就将 A^* 改造成成为包含有 (u, v) 的最小生成树， (u, v) 合并A也属于最小生成树。
 - 2.1) 证明 $A^* \cup (u, v)$ 会有回路
 - 2.2) 找到回路上一条边 (x, y) ，连接u所属的节点集合X和S-X
 - 2.3) 证明 $w(x, y) \geq w(u, v)$
 - 2.4) 将 (x, y) 替换成 (u, v) 得到新的生成树 A^{**} ， A^{**} 是最小生成树
 - 2.5) 按照kruskal算法的构造过程 (u, v) 合并A不会造成环，并且 (u, v) 合并A属于某个最小生成树
 - 2.6) 得证



Correctness Proof of Kruskal's Algorithm

Proof (cont'd):

- If A^* does not contain (u, v) , $A^* \cup (u, v)$ must have a cycle. 如果 A^* 没有包含 (u, v) , 那么 $A^* \cup (u, v)$ 肯定形成回路
- Because T is connected and thus there exists a path from u to v . Adding (u, v) provides another path from u to v that forms a cycle.
- 只要是生成树, 在任何地方多加一条边, 都会产生回路, 因为已经是连通的了, 再加一条边, 从一个点到另一个点, 就有两条路了





证明循环不变量Maintenance的思路

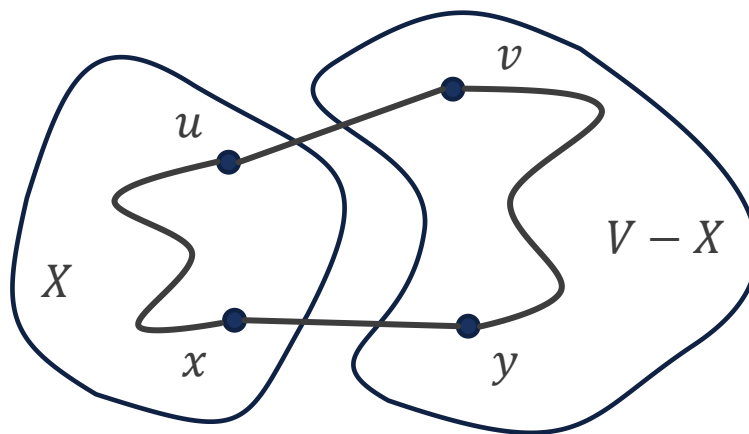
- 0) 假设之前加入的A都是某个最小生成树的子集，要证明的是加入 (u, v) 之后，A和 (u, v) 还是某个最小生成树的子集
- 1) 如果 (u, v) 属于某个最小生成树 A^* ，那么显然得证
- 2) 对于任意一个最小生成树 A^* ，如果 (u, v) 不属于 A^* ，那么就将 A^* 改造成成为包含有 (u, v) 的最小生成树， (u, v) 合并A也属于最小生成树。
 - 2.1) 证明 $A^* \cup (u, v)$ 会有回路
 - 2.2) 找到回路上的一条边 (x, y) ，连接u所属的节点集合X和S-X
 - 2.3) 证明 $w(x, y) \geq w(u, v)$
 - 2.4) 将 (x, y) 替换成 (u, v) 得到新的生成树 A^{**} ， A^{**} 是最小生成树
 - 2.5) 按照kruskal算法的构造过程 (u, v) 合并A不会造成环，并且 (u, v) 合并A属于某个最小生成树
 - 2.6) 得证



Correctness Proof of Kruskal's Algorithm

Proof (cont'd): 上一步证明了 $A^* \cup (u, v)$ 合并进去 A^* 会有回路，这里找到回路上的一条边 (x, y)

- Let X be the vertex set of the current subtree containing u . 让 X 为目前包含节点 u 的节点集合
- If A^* does not contain (u, v) , A^* must have an edge (x, y) connects X and $V - X$, where $x \in X$ and $y \in V - X$. 如果 A^* 没有包含 (u, v) ，那么由于 A^* 是生成树，肯定有另外一条边 (x, y) 连接 X 和 $V - X$





证明循环不变量Maintenance的思路

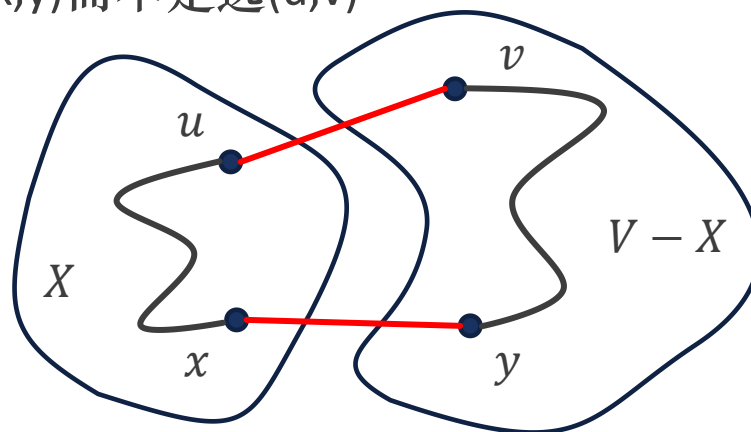
- 0) 假设之前加入的A都是某个最小生成树的子集，要证明的是加入 (u, v) 之后，A和 (u, v) 还是某个最小生成树的子集
- 1) 如果 (u, v) 属于某个最小生成树 A^* ，那么显然得证
- 2) 对于任意一个最小生成树 A^* ，如果 (u, v) 不属于 A^* ，那么就将 A^* 改造成为包含有 (u, v) 的最小生成树， (u, v) 合并A也属于最小生成树。
 - 2.1) 证明 $A^* \cup (u, v)$ 会有回路
 - 2.2) 找到回路上的一条边 (x, y) ，连接u所属的节点集合X和S-X
 - **2.3) 证明 $w(x, y) \geq w(u, v)$**
 - 2.4) 将 (x, y) 替换成 (u, v) 得到新的生成树 A^{**} ， A^{**} 是最小生成树
 - 2.5) 按照kruskal算法的构造过程 (u, v) 合并A不会造成环，并且 (u, v) 合并A属于某个最小生成树
- 2.6) 得证



Correctness Proof of Kruskal's Algorithm

Proof (cont'd):

- (u, v) is selected by KruskalMST at this step, which means X and $V - X$ is not connected. Therefore, we must have $w(u, v) \leq w(x, y)$.
KruskalMST是不断的合并两个集合的，合并的时候会判断下这条边的两个顶点是不是在同一个集合上，如果不在同一个集合上才要合并这两个顶点 u, v 对应的结合
- If $w(u, v) > w(x, y)$, KruskalMST will select (x, y) first and won't select (u, v) this time to avoid cycle. $w(u, v)$ 的权重肯定小于 $w(x, y)$ ，否则Kruskal会先选择 (x, y) 而不是选 (u, v)





证明循环不变量Maintenance的思路

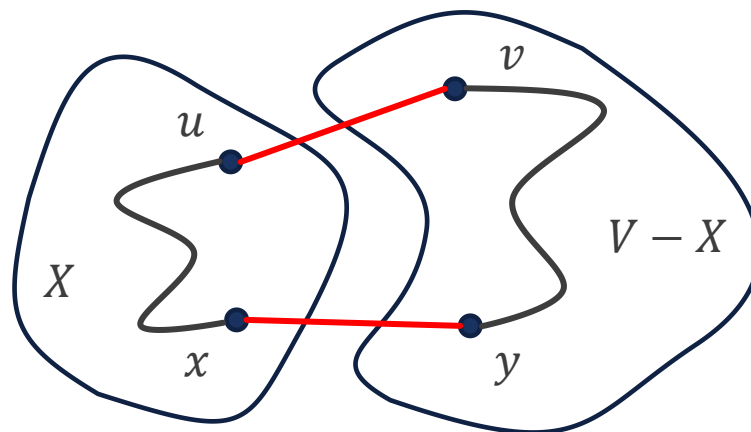
- 0) 假设之前加入的A都是某个最小生成树的子集，要证明的是加入 (u, v) 之后，A和 (u, v) 还是某个最小生成树的子集
- 1) 如果 (u, v) 属于某个最小生成树 A^* ，那么显然得证
- 2) 对于任意一个最小生成树 A^* ，如果 (u, v) 不属于 A^* ，那么就将 A^* 改造成为包含有 (u, v) 的最小生成树， (u, v) 合并A也属于最小生成树。
 - 2.1) 证明 $A^* \cup (u, v)$ 会有回路
 - 2.2) 找到回路上的一条边 (x, y) ，连接u所属的节点集合X和S-X
 - 2.3) 证明 $w(x, y) \geq w(u, v)$
 - 2.4) 将 (x, y) 替换成 (u, v) 得到新的生成树 A^{**} ， A^{**} 是最小生成树
 - 2.5) 按照kruskal算法的构造过程 $(u, v) \cup A$ 不会造成环，并且 $(u, v) \cup A$ 属于某个最小生成树
 - 2.6) 得证



Correctness Proof of Kruskal's Algorithm

Proof (cont'd):

- Now, if we construct $A^{**} = \{A^* - (x, y)\} \cup (u, v)$, then A^{**} may have smaller total weights because $w(u, v) \leq w(x, y)$. 如果我们从 A^* 将 (x, y) 替换成为 (u, v) , 那么显然得到的新集合 A^{**} 的边的权重 $\leq A^*$ 的边权重
- However, A^* is optimal which means it is impossible that $w(u, v) < w(x, y)$. The only possibility is $w(u, v) = w(x, y)$, which makes both A^* and A^{**} optimal. 但是 A^* 是最小的, 因此 A^{**} 也是最小的
- Because $(u, v) \in A^{**}$, now we have $A' \subseteq A^*$. That means adding (u, v) by KruskalMST is still a subset of a MST.



按照Kruskal算法的性质, 这里 (u, v) 肯定和 A 是没有形成回路的



Correctness Proof of Kruskal's Algorithm

Proof (cont'd):

Termination:

All edges added to A are in a minimum spanning tree, and so the set A is returned in Line 9 must be a minimum spanning tree.

结束时，所有加入到 A 的边都是属于最小生成树的边，因此结束时返回的 A 就是最小生成树

KruskalMST(G, w)

```
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v \in V$  do
3   MakeSet( $v$ )
4 sort the edges of  $E$  by weight  $w$ 
5 for each edge  $(u, v) \in E$  do
6   if FindSet( $u$ )  $\neq$  FindSet( $v$ )
7     then
8        $A \leftarrow A \cup \{(u, v)\}$ 
9   Union( $u, v$ )
9 return  $A$ 
```



MINIMUM SPANNING TREE

PRIM'S ALGORITHM



Prim's Algorithm

- Prim's algorithm has the property that the edges in the set A always form a single tree. Prim 确保在集合 A 中的边构成一颗树。
 - In Kruskal's algorithm, edges in the set A may not be a tree.
- The tree starts from an arbitrary root vertex r and grows until the tree spans all the vertices in V .

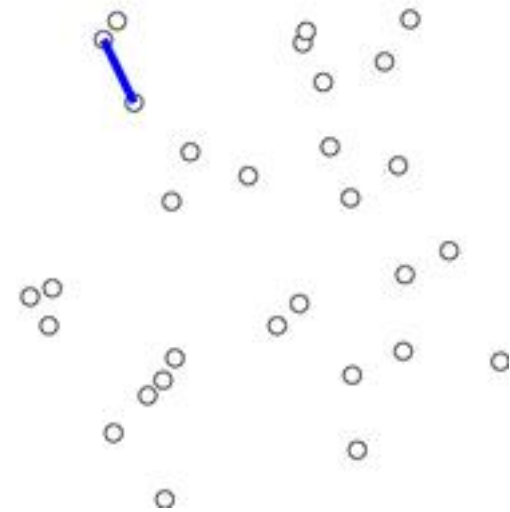


Prim's Algorithm

High level pseudocode:

一直只有一颗树，不停的将顶点合并到树中

- Initialize $A = \emptyset$ and $X = \{v_1\}$.
- Iterate when the instance is not solved:
 - Select a vertex in $V - X$ that is nearest to X .
 - Add the vertex to X .
 - Add the edge to A .



A demo for Prim's algorithm on a complete graph with weights based on Euclidean distance.



Pseudocode

PrimMST(G, w, r)

1 **for** each $u \in V$ **do**

2 $key[u] \leftarrow \infty$

3 $\pi[u] \leftarrow \text{NIL}$

4 $key[r] \leftarrow 0$

5 $Q \leftarrow V$

6 **while** $Q \neq \emptyset$ **do**

7 $u \leftarrow \text{ExtractMin}(Q)$

8 **for** each $v \in \text{Adj}[u]$ **do**

9 **if** $v \in Q$ and $w(u, v) < key[v]$ **then**

10 $\pi[v] \leftarrow u$

11 $key[v] \leftarrow w(u, v)$

$O(|V|)$

$O(\lg |V|)$

最小堆 $O(\lg |V|)$

Total: $O(|E| \lg |V|)$

用最小堆来实现最小优先队列 Q ，创建时间是 $O(|V|)$

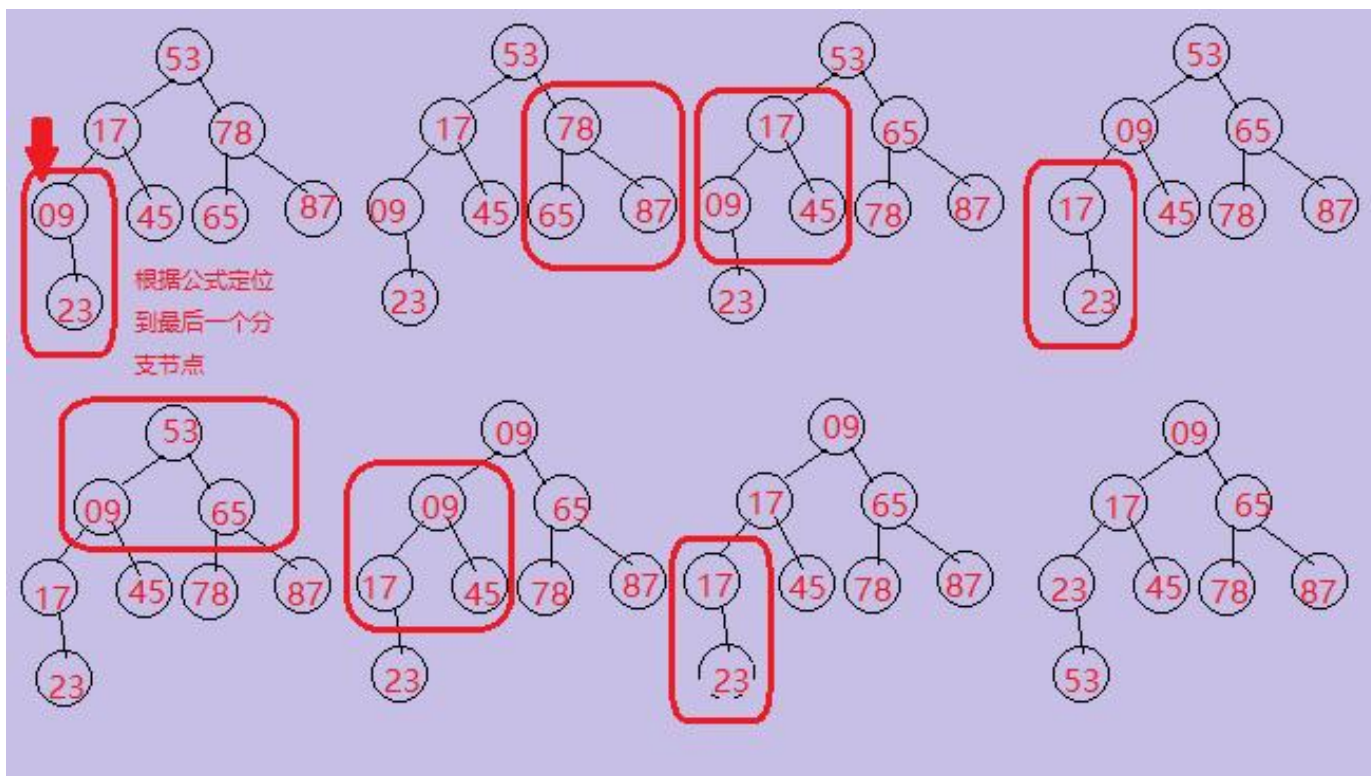
从优先队列获取最小元素，更新key的时间 $O(\lg |V|)$

$O(|E| \lg |V|)$

分摊分析，8-11最多调用 E 次，每次 $\lg V$ ，所以是 $O(|E| \lg |V|)$



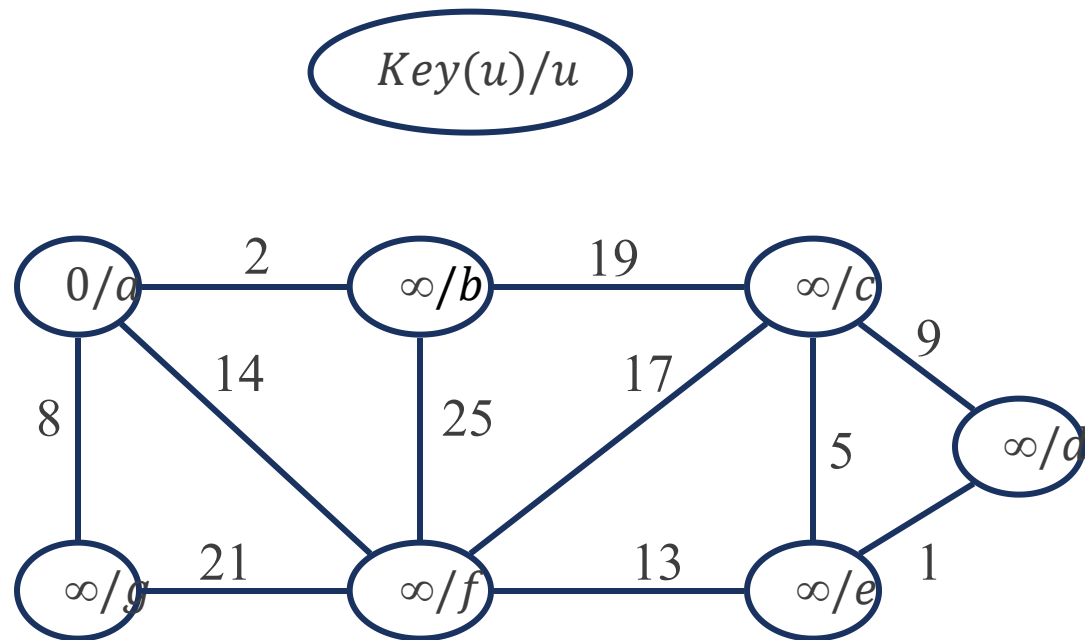
最小堆（创建时间 $O(V)$, 更新时间 $O(\log v)$ ）



- 链接https://blog.csdn.net/weixin_37818081/article/details/78685916
- <https://blog.csdn.net/hrn1216/article/details/51465270>

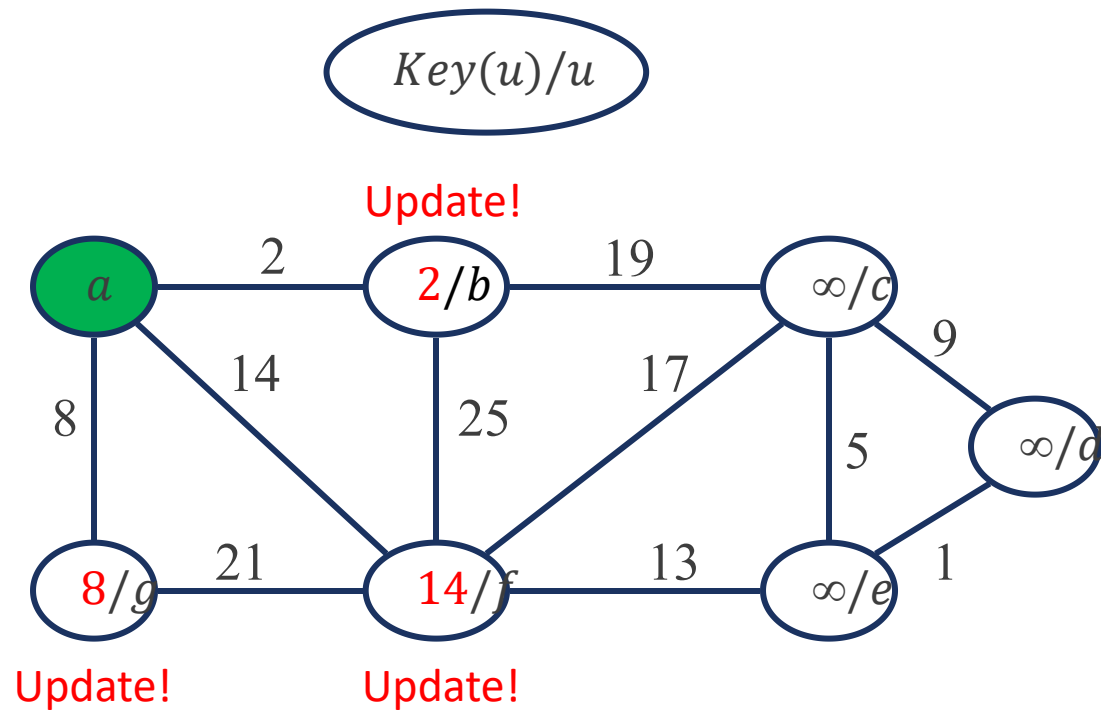


Example (Prim算法示例)



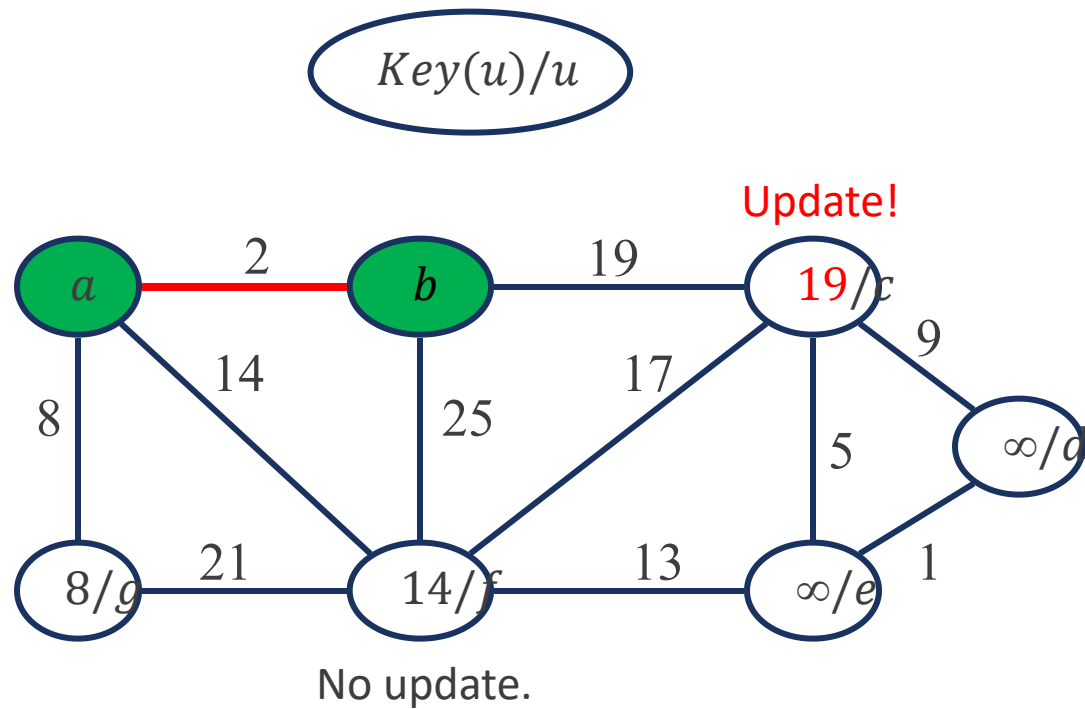


Example (Prim算法示例)



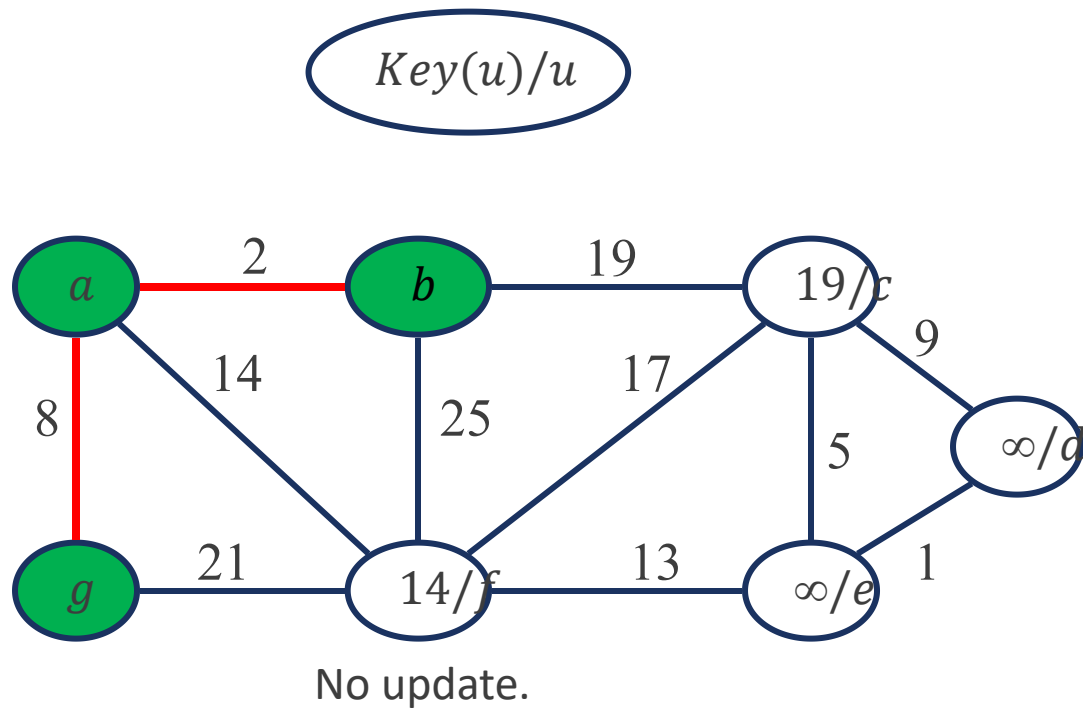


Example



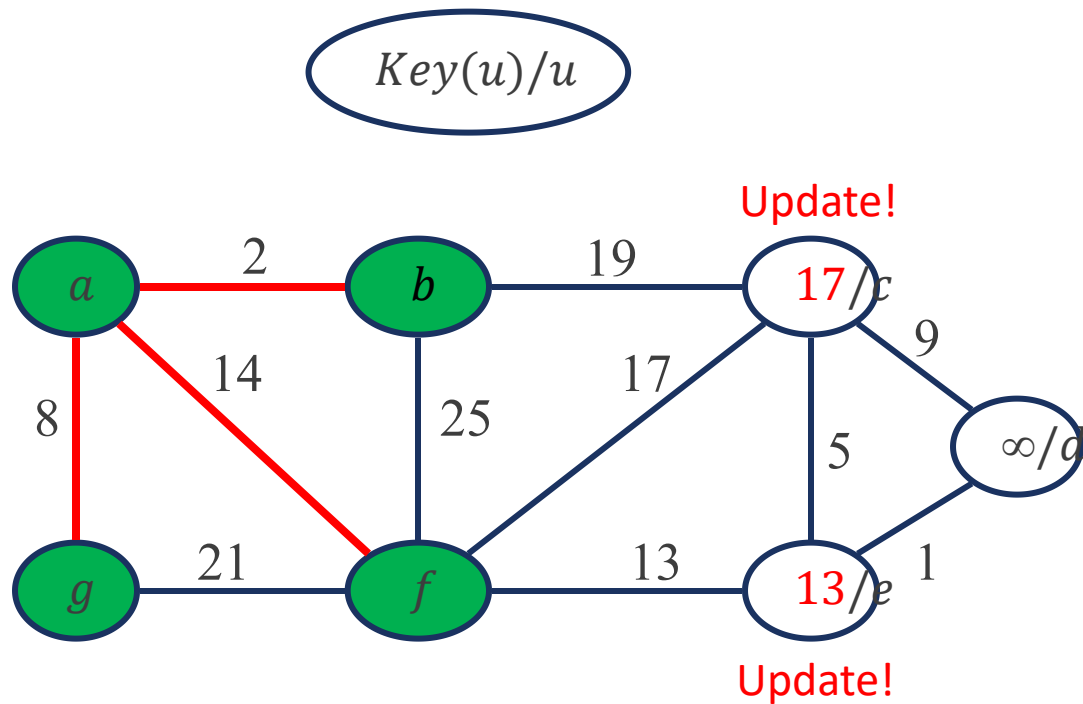


Example (Prim算法示例)



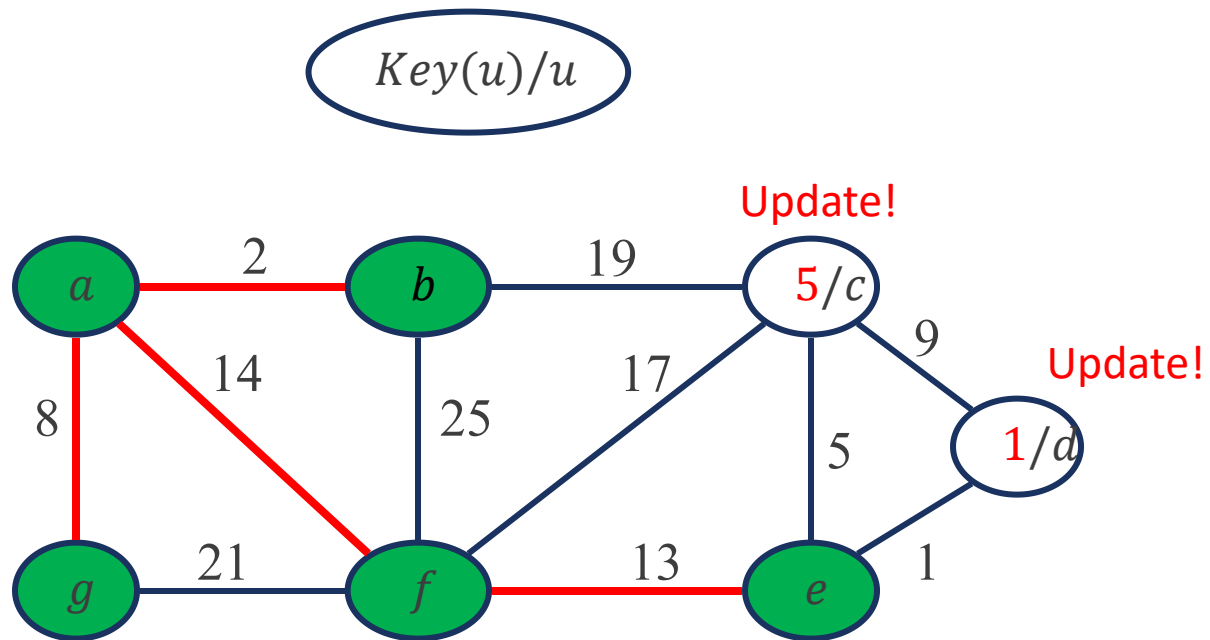


Example (Prim算法示例)



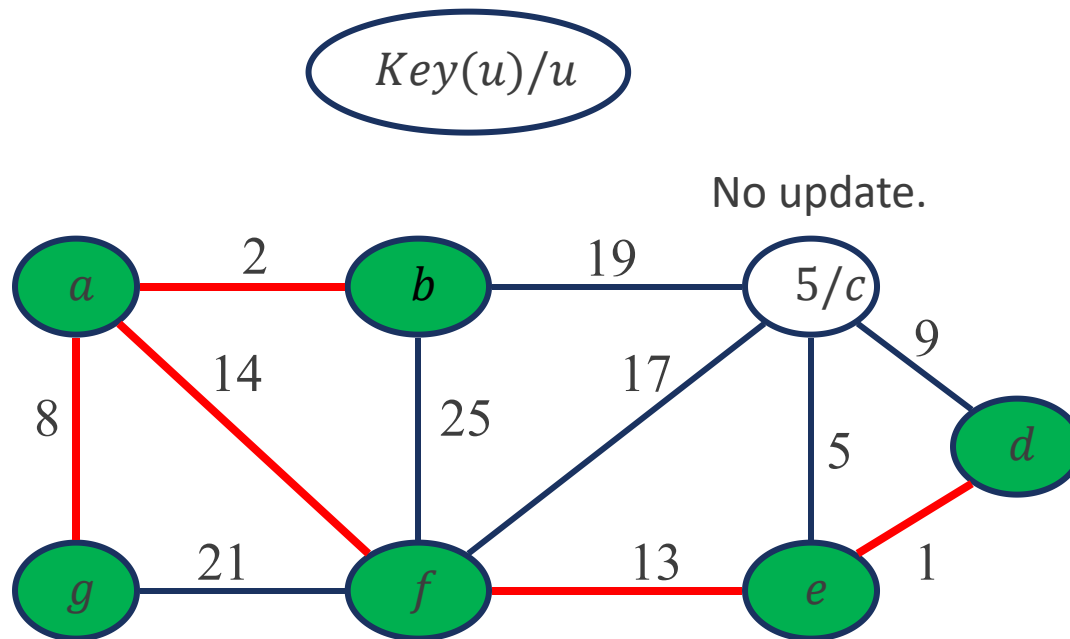


Example (Prim算法示例)



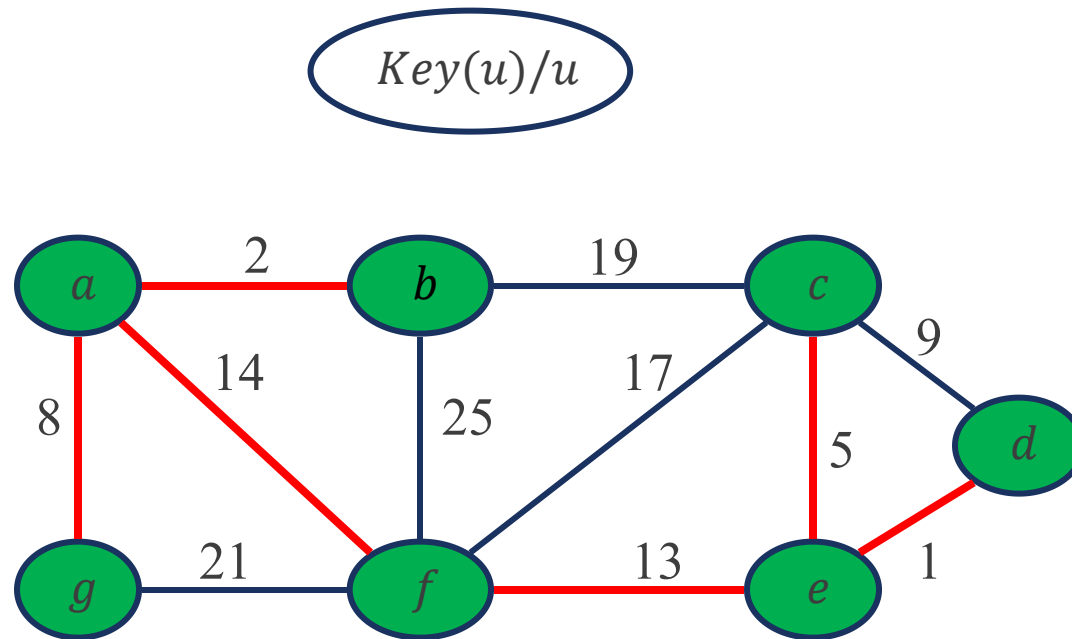


Example (Prim算法示例)





Example (Prim算法示例)





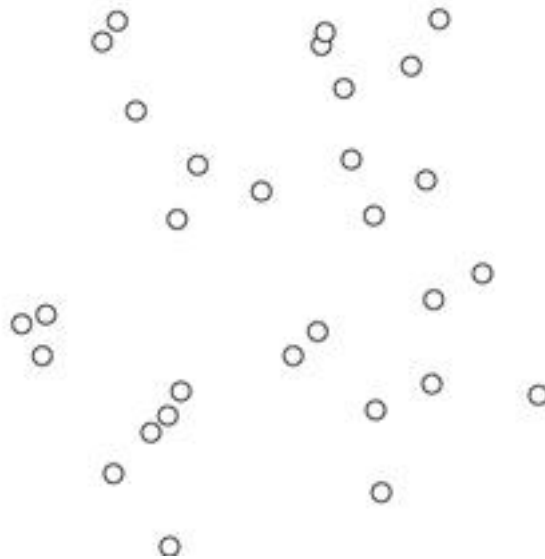
Kruskal's vs. Prim's

- Kruskal's algorithm: $O(|E| \lg |E|)$.
- Prim's algorithm: $O(|E| \lg |V|)$.
- Just compare $|E|$ and $|V|$.
 - For a sparse graph (close to null graph, $O(|E|) = O(1)$), Kruskal's algorithm is faster. Kruskal适合稀疏的图
 - For a dense graph (close to complete graph, $O(|E|) = O(|V|^2)$), Prim's algorithm is faster. Prim适合稠密的图

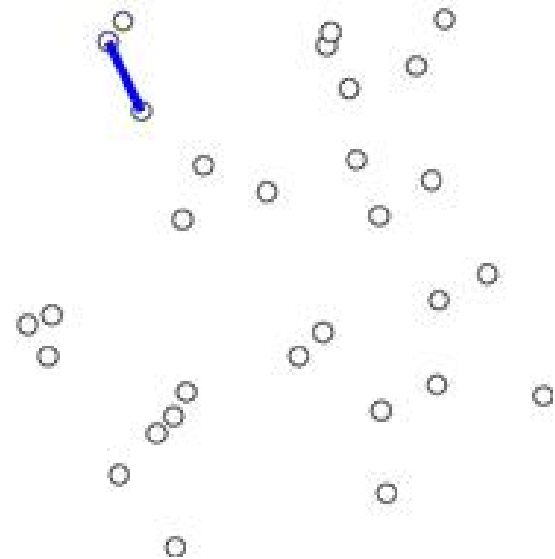


Kruskal's vs. Prim's

Complete graph



Kruskal's algorithm



Prim's algorithm



Classroom Exercise 课堂练习

- Another method to get a MST is called **reverse-delete algorithm**. We sort the edge weights in decreasing order first. Then, remove the edge if the graph is not disconnected. 将所有的边按照权重从大到小的排序。从头开始删除边。如果删除掉这条边不会导致图不联通的话，就删除掉这条边。
- Try to prove the correctness of this algorithm. 试着证明这个算法的正确性



Classroom Exercise

Proof:

- Prove by contradiction: Assume that the tree T obtained from reverse-delete algorithm is not a MST.
- Then, there exists a MST T' that is different from T .
- Pick the edge e that in T but not in T' . Adding e into T' will form a cycle.
- e being maintained in T means that there is another e' whose weight is larger than e in this cycle. e 是从大到小排序的边，并且删除掉 e 会导致整个图不联通。原来从图中删除边的方式，决定了删除掉的边肯定是在一个圈(cycle)中，并且这个删掉的边肯定是cycle中权重最大的边，因此在 T' 中的这个边 e' 肯定是原来删除掉的边，所以他的权重比 e 还大
- Replacing e' with e will obtain a better tree that contradicts with that T' is a MST.



Classroom Exercise (证明方法)

First, we show that the algorithm produces a spanning tree. This is because the graph is given connected at the beginning and, when deleting edges in the nonincreasing order, only the most expensive edge in any cycle is deleted, which does eliminate the cycles but not disconnect the graph, resulting in a connected graph containing no cycle at the end.

- To show that the obtained spanning tree is an MST, consider any edge removed by the algorithm. It can be observed that it must lie on some cycle (otherwise removing it would disconnect the graph) and it must be the most expensive one on it (otherwise retaining it would violate the cycle property). Hence, the “Reverse Delete” algorithm produces an MST.

Wang, Yingqiang, Xiang, Xiaoli, WILKES, D. Mitchell. A divide-and-conquer approach for minimum spanning tree-based clustering. IEEE Transactions on Knowledge and Data Engineering, 2009, 21.7: 945-958.



THE SHORTEST PATHS PROBLEM

最短路径问题



Shortest-Paths Problem

The shortest-paths problem:

- 本章首先讲了Bellman-ford算法，可以处理带负回路的图
- 单源最短路径算法也是产生了以源点 s 为根的树，这棵树可以称为最短路径树。从源点 s 到 v 的在树上的路径，就是图上最短路径
- The algorithms in this chapter differ in **how many times they relax** each edge and the order in which they relax edges. In Dijkstra's algorithm and the shortest-paths algorithm for directed acyclic graphs, each edge is **relaxed exactly once**. In the **Bellman-Ford algorithm**, each edge is relaxed many times.



Shortest-Paths Problem

The shortest-paths problem:

- Given a **weighted, directed** graph $G = (V, E)$, with weights $w(u, v)$ for each edge $(u, v) \in E$, the weight of path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges: 一条路径的长度就是路径上边权重的加和

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

- The **shortest-path weight** from u to v can be defined as follows: 最短路径就是从 u 到 v 的所有可能路径上最短的

$$\delta(u, v) = \begin{cases} \min_p \{w(p) : u \xrightarrow{p} v\} & \text{if there is one path } p \text{ from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

- A **shortest path** from vertex u to vertex v is then defined as **any path** p with weight $w(p) = \delta(u, v)$.



Variants of Shortest-Paths Problem

- **The single-source (单个源点) shortest-paths problem:** Given a graph $G = (V, E)$, we want to find a shortest path from a given source vertex $s \in V$ to each vertex $v \in V$. 单源最短路径问题，如果边不带权重的话，BFS就可以算出来
- **Single-pair (单个点对) shortest-path problem:** Given vertices u and v , find a shortest path from u to v . 给了一个起点 u 和终点 v ，找到其最短路径
- **All-pairs (所有点对) shortest-paths problem:** Find a shortest path from u to v for every pair of vertices u and v . 找图上所有点之间的最短路径



Optimal Substructure of a Shortest Path

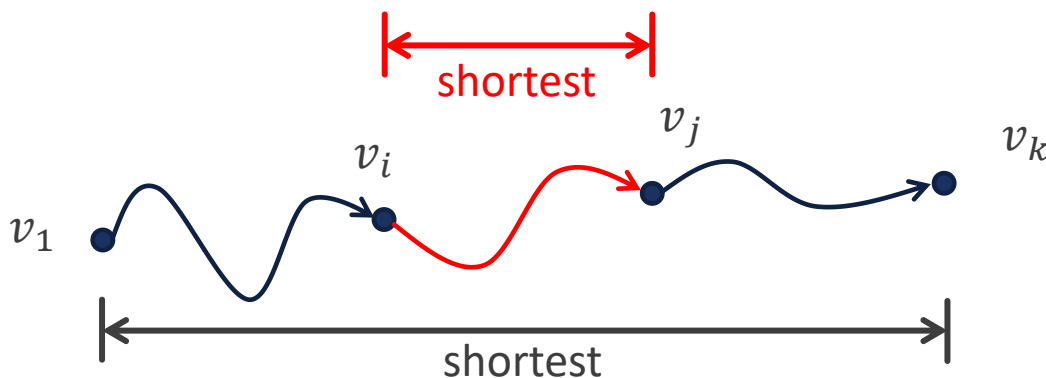
最短路径问题具有最优子结构

Theorem 8.7

Given a weighted, directed graph $G = (V, E)$ with weights $w(u, v)$ for each edge $(u, v) \in E$, let $p = v_1, v_2, \dots, v_k$ be a shortest path from vertex v_1 to vertex v_k .

For any i and j such that $1 \leq i \leq j \leq k$, let $p_{ij} = v_i, v_{i+1}, \dots, v_j$ be the subpath of p from vertex v_i to vertex v_j .

对于最短路径上的任意两个节点 v_i, v_j , 他们之间的路径也是最短的。
Then, p_{ij} is a shortest path from v_i to v_j .



证明方法
用反证法
Cut-and-paste



Relaxation 松弛技术

- The algorithms in this chapter use the technique of **relaxation** (松弛).
- For each vertex $v \in V$, we maintain an attribute $d[v]$, which is an **upper bound** on the weight of a shortest path from source s to v . $d[v]$ 记录从源点 s 到 v 点的距离的上界
- We call $d[v]$ a **shortest-path estimate**. It is updated to get smaller as the algorithm goes on. $d[v]$ 是最短距离的估计
 - Just similar to the $d[v]$ in BFS, but can be updated more than once to get close to $\delta(s, v)$. 最短路径计算过程中, $d[v]$ 会更新多次, 不同于BFS里面 $d[v]$ 只会更新一次



Relaxation

- We initialize the shortest-path estimates $d[v]$ and predecessors $\pi[v]$ by the following $O(V)$ -time procedure.
 - Exactly same as how BFS does.

InitializeSingleSource(G, s)

1 **for** each vertex $v \in V$ **do**

2 $d[v] \leftarrow \infty$

3 $\pi[v] \leftarrow NIL$ 最开始d都是无穷大

4 $d[s] \leftarrow 0$



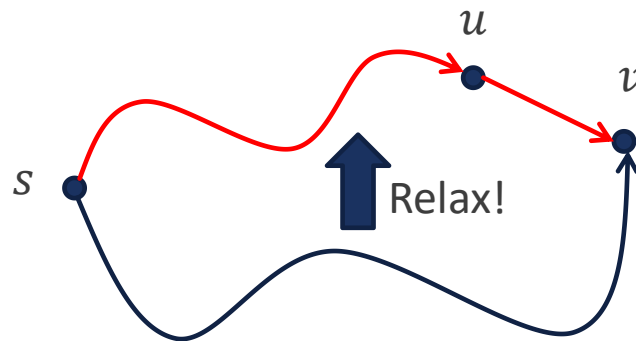
Relaxation

松弛其实是一个逐渐变紧的过程，将 $d[v]$ 不断变小，以接近其真正的最短距离。每次都判断下，是不是通过 u ，能有更短的距离，如果是，那么更新 $d[v] \leftarrow d[u] + w(u, v)$

- The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and, if so, updating $d[v]$ and $\pi[v]$.

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

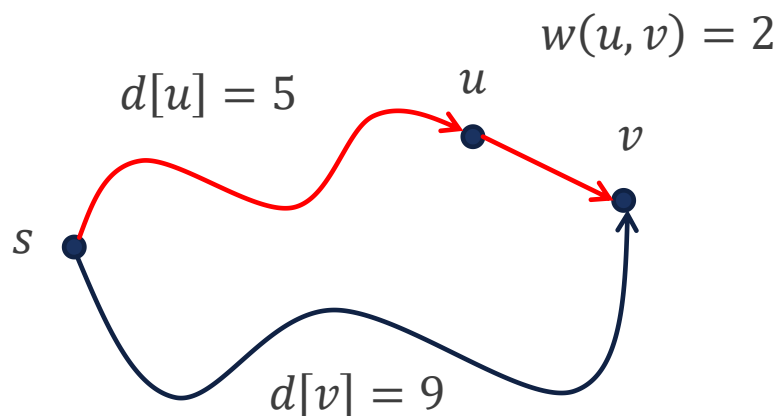
Relaxation in one sentence:
check if going through u is faster



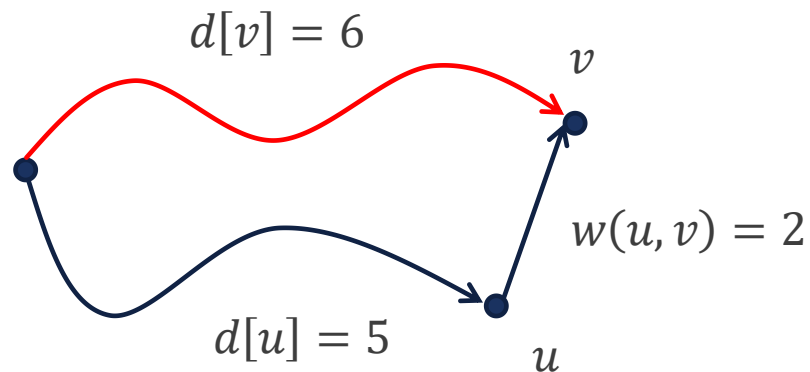
When $d[v] > d[u] + w(u, v)$



Example 松弛举例



Relax!



Don't relax



Properties of Shortest Paths and Relaxation

- **Triangle inequality (三角不等式):** For any edge $(u, v) \in E$,
$$\delta(s, v) \leq \delta(s, u) + w(u, v).$$
- **Upper-bound property (上界性质):** We always have $d[v] \geq \delta(s, v)$ for all vertices $v \in V$, and once $d[v]$ achieves the value $\delta(s, v)$, it never changes.
- **No-path property (无路径性质):** If there is no path from s to v , then $d[v] = \delta(s, v) = \infty$.
- **Convergence property (收敛性质):** If $s \sim u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and $d[u] = \delta(s, u)$, then after $\text{Relax}(u, v, w)$, we have $d[v] = \delta(s, v)$.



THE SHORTEST PATHS PROBLEM

单源最短路径问题: BELLMAN-FORD ALGORITHM



Single-Source Shortest-Paths Problem

Bellman-Ford algorithm:

- Given a weighted, directed graph $G = (V, E)$ with source s and weights $w(u, v)$ for each edge $(u, v) \in E$, where $w(u, v)$ can be negative.
- The Bellman-Ford algorithm does two things:
 - Calculate $\delta(s, v)$ for all $v \in V$.
 - Return a boolean value indicating whether or not there is a negative-weight cycle that is reachable from the source. 负回路是最短距离的克星, 千万不能有
- If there is such a cycle, the algorithm indicates that no solution exists. If there is no such cycle, the algorithm produces the shortest paths and their weights. 如果有负回路, 说明没有最短距离, 否则就有最短距离



Pseudocode

Relax(u, v, w)

```
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

BellmanFord(G, w, s)

```
1 InitializeSingleSource( $G, s$ )  
2 for  $i \leftarrow 1$  to  $|V| - 1$  do  
3   for each edge  $(u, v) \in E$  do  
4     Relax( $u, v, w$ )  
5 for each edge  $(u, v) \in E$  do  
6   if  $d[v] > d[u] + w(u, v)$  then return False  
7 return True
```

源节点 s 属于 V

Do $|V| - 1$ times

Each time relax all edges

After $|V| - 1$ times, there still exists an edge can be relaxed, it means negative-weight cycle exists.

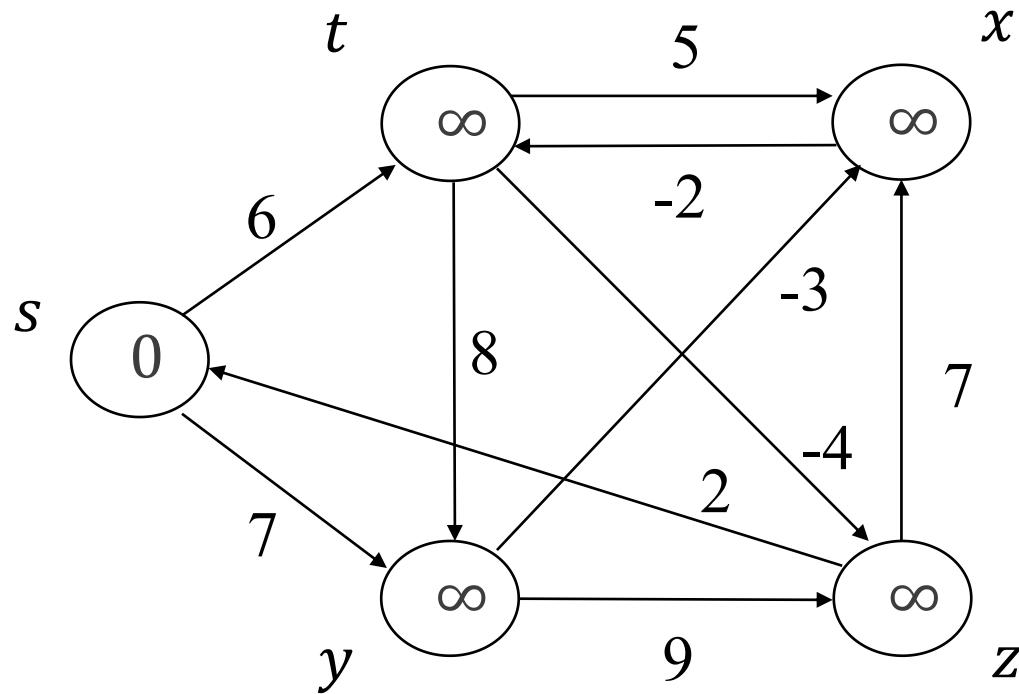
Total running time:
 $O(|V||E|)$

如果还能Relax,有负回路



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

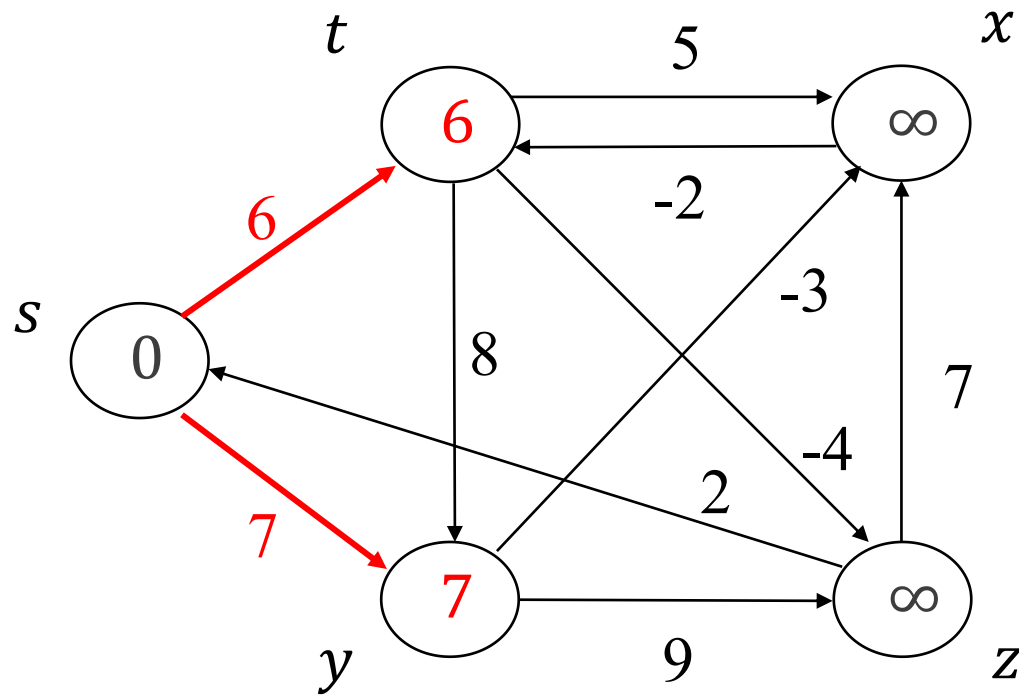


Initialization



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

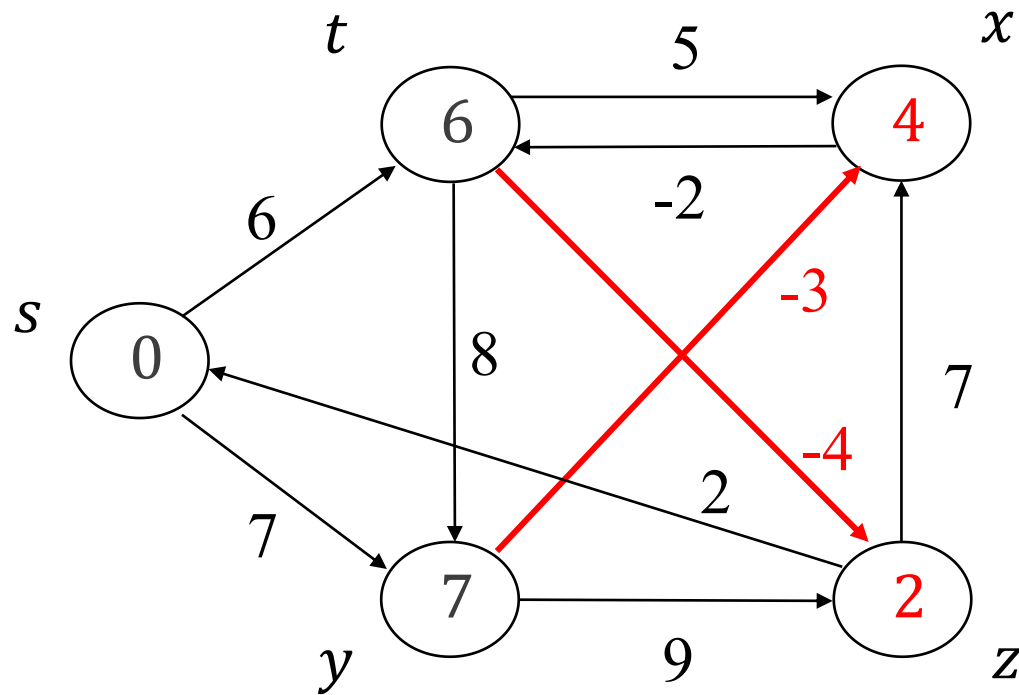


Round 1



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

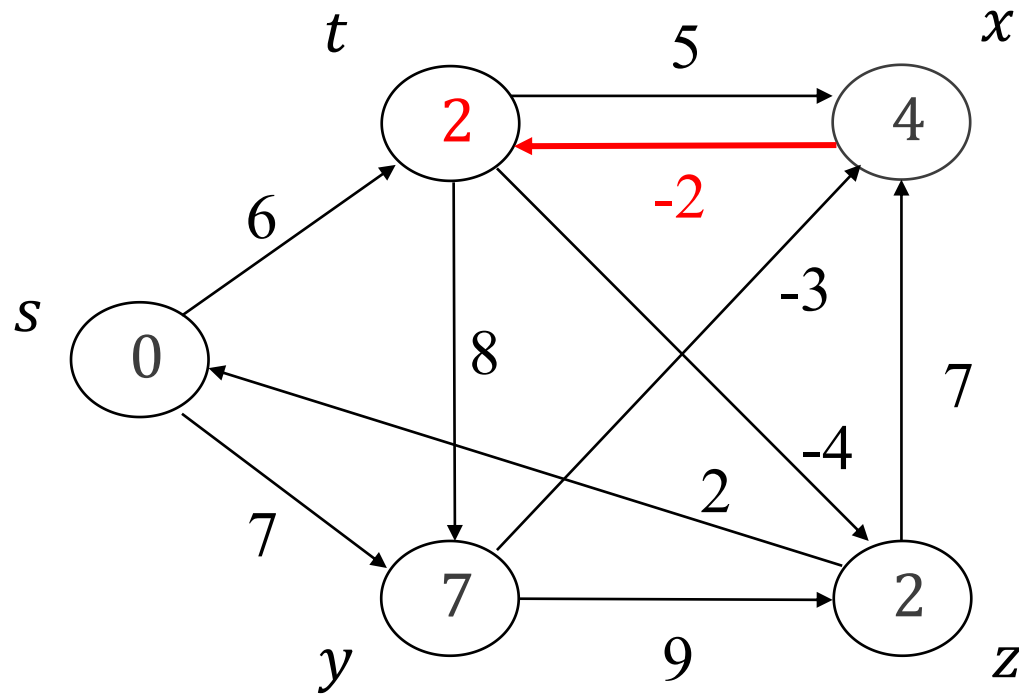


Round 2



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

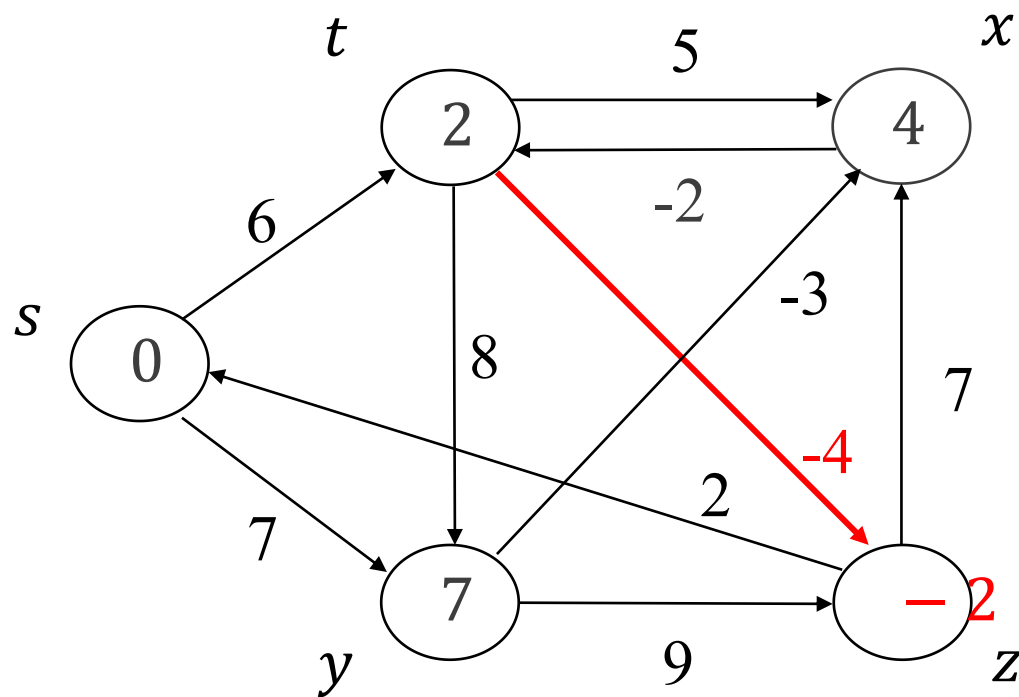


Round 3



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

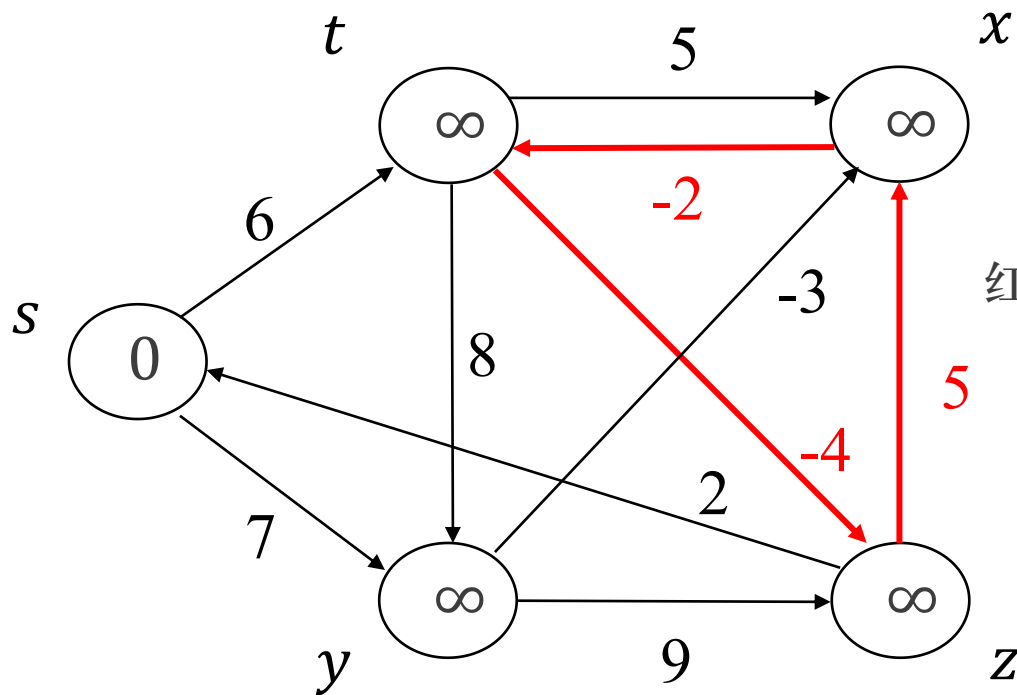


Round 4: No negative-weight cycle
之后判断不能再Relax了return TRUE



Example 有负回路的情况

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```



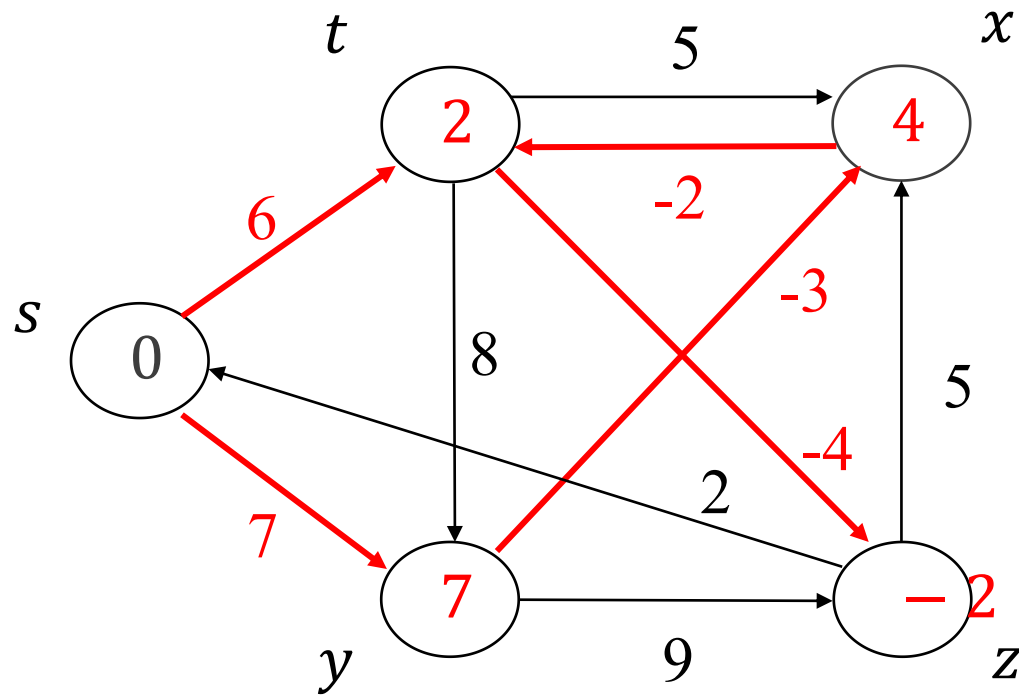
红色边上权重和小于0

Now, if we indeed have a negative-weight cycle...



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

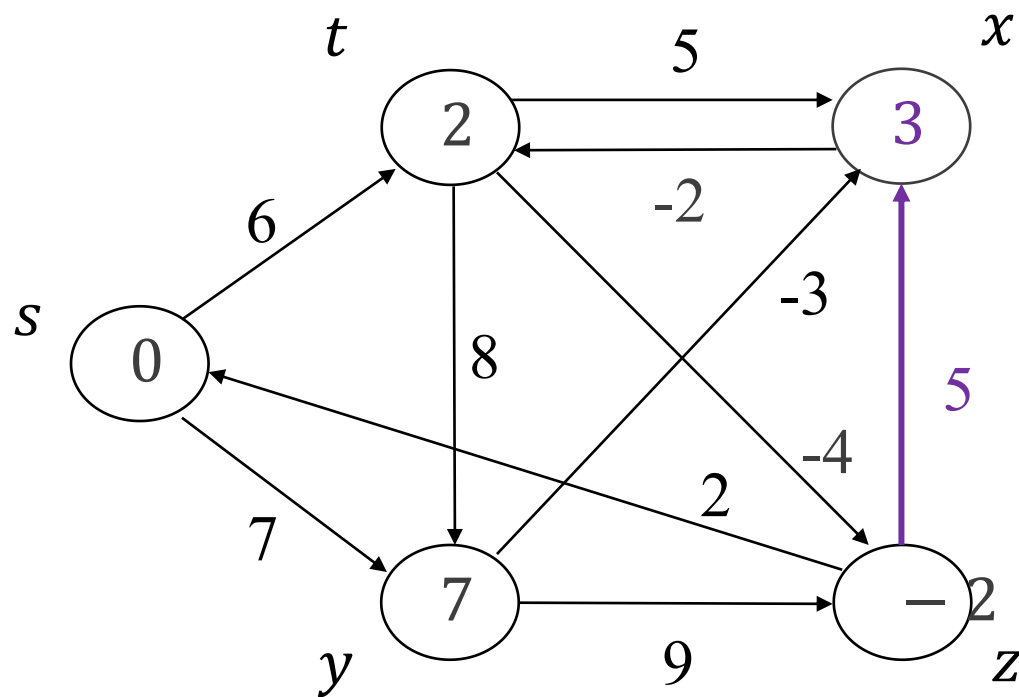


Round 1-4 are same



Example

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```



如果存在负回路, 所有
最小距离将永无止尽的
变小,
所以压根就没法算s到
所有点的最短路径距离
另外一方面, 如果不存在
负回路, $|V|-1$ 次肯定
够

We can still relax on an edge, return FALSE



THE SHORTEST PATHS PROBLEM

最短路径问题: DIJKSTRA'S ALGORITHM



Single-Source Shortest-Paths Problem

- Although Bellman-Ford algorithm is useful for negative-weight cycle detection, it is too slow ($O(|E||V|)$).
- If we know that there's no negative weight on the graph, we may have a better algorithm. 如果图中没有负权重



Single-Source Shortest-Paths Problem

Search results for "dijkstra":
dijkstra
dijkstra algorithm
dijkstra pronunciation
dijkstra python

Dijkstra's (/ˈdaɪkstrə/) (戴克斯特拉, 迪杰斯特拉) algorithm

- Maintains a set S of vertices whose final shortest-path weights from the source s have already been determined. 用 S 集合存储最短路径已经确定的节点
- Repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u to S , and relaxes all edges leaving u . 不断的将其他具有最短距离评估的节点加进 S , 并且更新从 u 出发的路径信息



Edsger Dijkstra
(1930-2002)

图灵奖得主26岁时提出这个算法



Dijkstra's Algorithm

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

Dijkstra(G, w, s)

1 InitializeSingleSource(G, s)

2 $S \leftarrow \emptyset$ S maintains the vertices with
determined shortest-path weight

3 $Q \leftarrow V$

4 **while** $Q \neq \emptyset$ **do**

5 $u \leftarrow \text{ExtractMin}(Q)$ Select $v \in Q$ with minimal $d[v]$

6 $S \leftarrow S \cup \{u\}$

7 **for each** vertex $v \in \text{Adj}[u]$ **do**

8 Relax(u, v, w)

优先队列

Relax all v 's neighbors

Total running time

取决于优先队列的实现

数组: $O(|V|^2)$

最小堆: $O(E \lg V)$

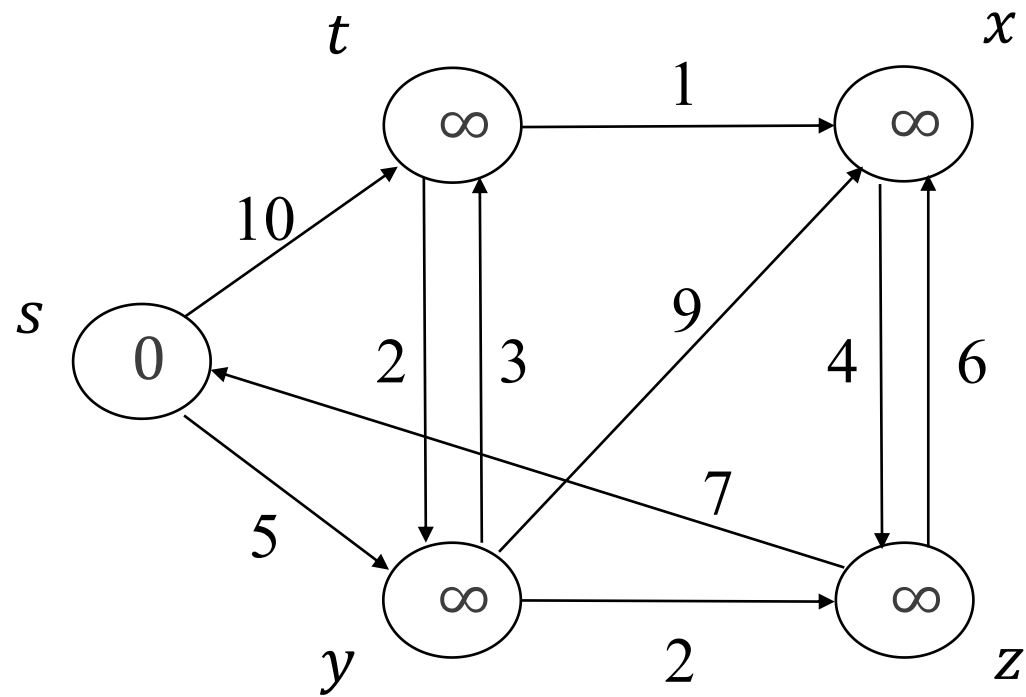
斐波那契堆: $O(V \lg V + E)$



Example

$$S = \{\}$$

Q	s	t	x	y	z
	0	∞	∞	∞	∞



Initialization



Example

```

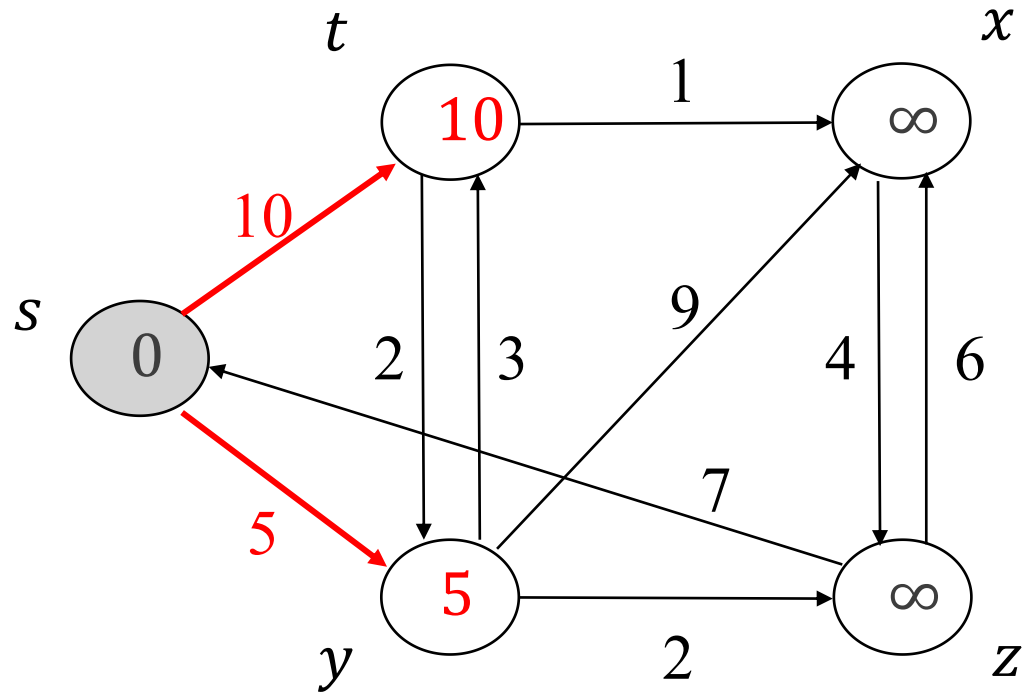
Relax( $u, v, w$ )
1 if  $d[v] > d[u] + w(u, v)$  then
2    $d[v] \leftarrow d[u] + w(u, v)$ 
3    $\pi[v] \leftarrow u$ 
  
```

```

Dijkstra( $G, w, s$ )
1 InitializeSingleSource( $G, s$ )
2  $S \leftarrow \emptyset$ 
3  $Q \leftarrow V$ 
4 while  $Q \neq \emptyset$  do
5    $u \leftarrow \text{ExtractMin}(Q)$ 
6    $S \leftarrow S \cup \{u\}$ 
7   for each vertex  $v \in \text{Adj}[u]$  do
8     Relax( $u, v, w$ )
  
```

$S = \{s\}$

Q	t	x	y	z
	10	∞	5	∞



$\text{ExtractMin}(Q) = s$



Example

```

Relax( $u, v, w$ )
1 if  $d[v] > d[u] + w(u, v)$  then
2    $d[v] \leftarrow d[u] + w(u, v)$ 
3    $\pi[v] \leftarrow u$ 
  
```

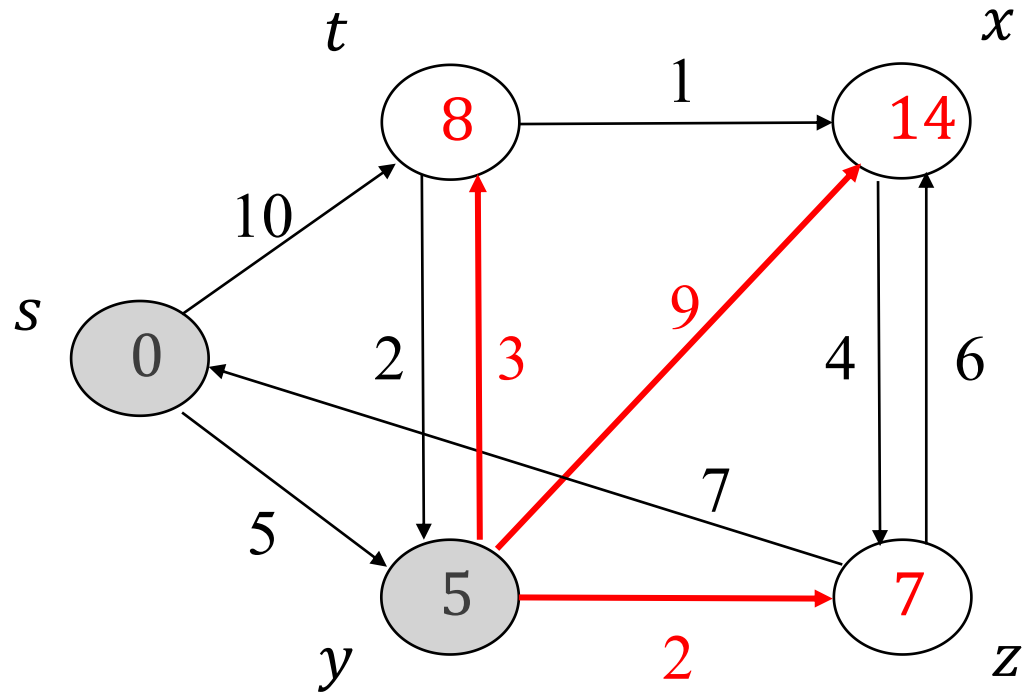
```

Dijkstra( $G, w, s$ )
1 InitializeSingleSource( $G, s$ )
2  $S \leftarrow \emptyset$ 
3  $Q \leftarrow V$ 
4 while  $Q \neq \emptyset$  do
5    $u \leftarrow \text{ExtractMin}(Q)$ 
6    $S \leftarrow S \cup \{u\}$ 
7   for each vertex  $v \in \text{Adj}[u]$  do
8     Relax( $u, v, w$ )
  
```

$S = \{s, y\}$

Q

t	x	z
8	14	7



$\text{ExtractMin}(Q) = y$



Example

```

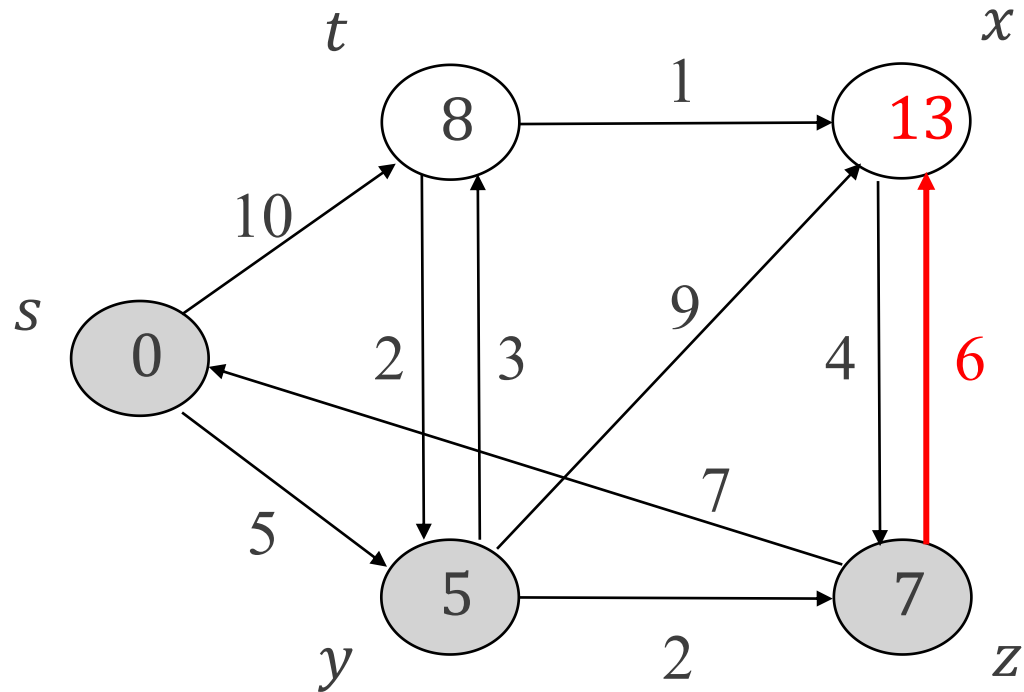
Relax( $u, v, w$ )
1 if  $d[v] > d[u] + w(u, v)$  then
2    $d[v] \leftarrow d[u] + w(u, v)$ 
3    $\pi[v] \leftarrow u$ 
  
```

```

Dijkstra( $G, w, s$ )
1 InitializeSingleSource( $G, s$ )
2  $S \leftarrow \emptyset$ 
3  $Q \leftarrow V$ 
4 while  $Q \neq \emptyset$  do
5    $u \leftarrow \text{ExtractMin}(Q)$ 
6    $S \leftarrow S \cup \{u\}$ 
7   for each vertex  $v \in \text{Adj}[u]$  do
8     Relax( $u, v, w$ )
  
```

$S = \{s, y, z\}$

Q	t	x
	8	13



$\text{ExtractMin}(Q) = z$



Example

Relax(u, v, w)

```

1 if  $d[v] > d[u] + w(u, v)$  then
2    $d[v] \leftarrow d[u] + w(u, v)$ 
3    $\pi[v] \leftarrow u$ 

```

Dijkstra(G, w, s)

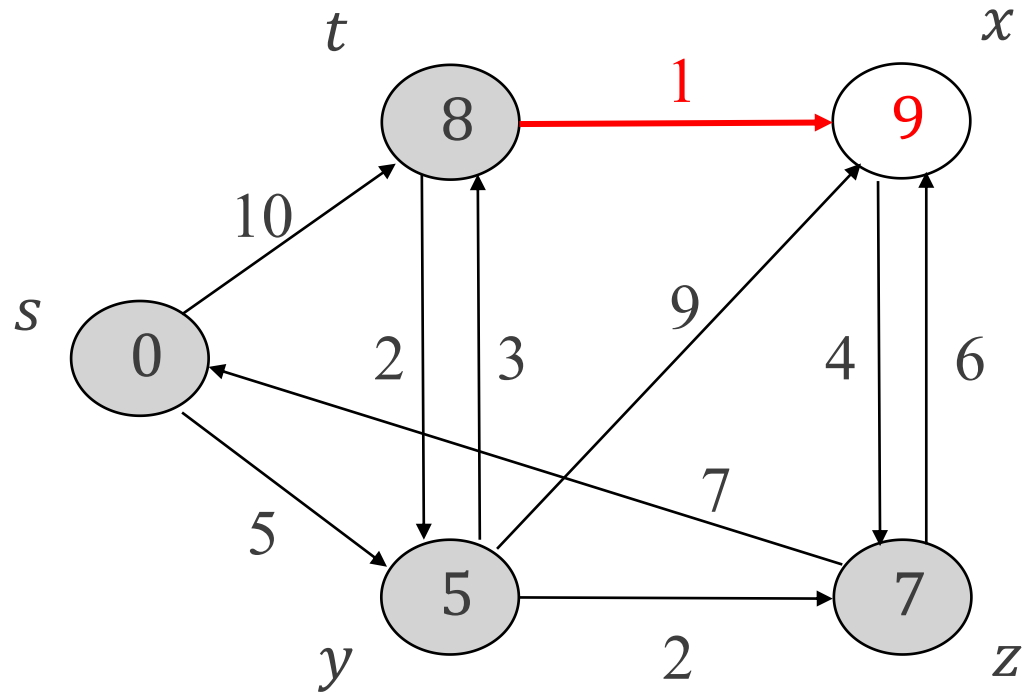
```

1 InitializeSingleSource( $G, s$ )
2  $S \leftarrow \emptyset$ 
3  $Q \leftarrow V$ 
4 while  $Q \neq \emptyset$  do
5    $u \leftarrow \text{ExtractMin}(Q)$ 
6    $S \leftarrow S \cup \{u\}$ 
7   for each vertex  $v \in \text{Adj}[u]$  do
8     Relax( $u, v, w$ )

```

$S = \{s, y, z, t\}$

Q x
9



ExtractMin(Q) = t



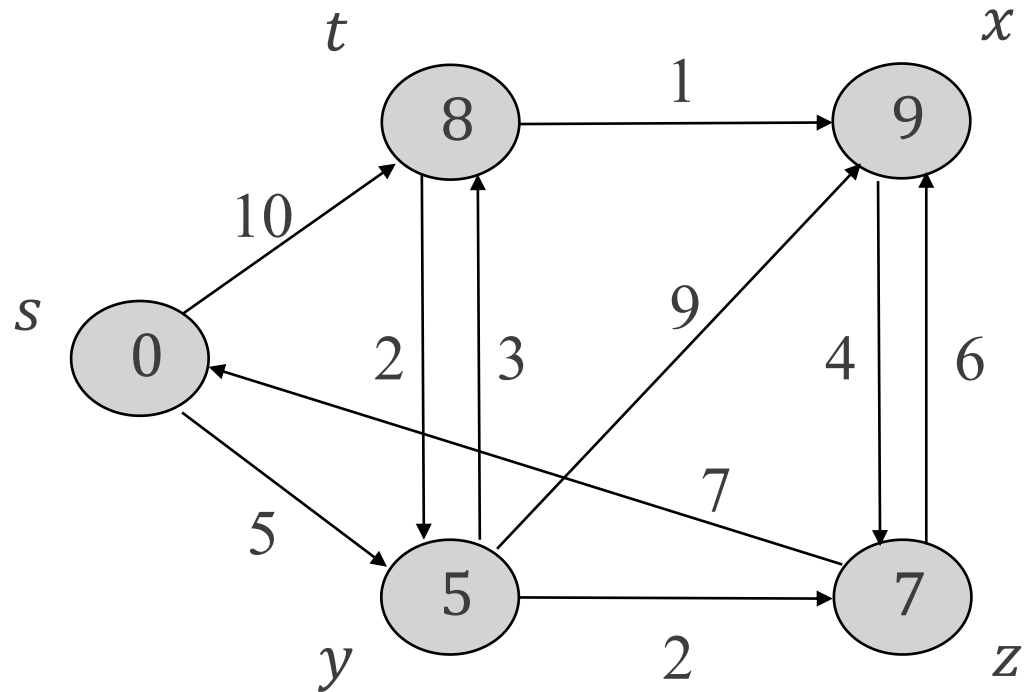
Example

Relax(u, v, w)

```
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

Dijkstra(G, w, s)

```
1 InitializeSingleSource( $G, s$ )  
2  $S \leftarrow \emptyset$   
3  $Q \leftarrow V$   
4 while  $Q \neq \emptyset$  do  
5    $u \leftarrow \text{ExtractMin}(Q)$   
6    $S \leftarrow S \cup \{u\}$   
7   for each vertex  $v \in \text{Adj}[u]$  do  
8     Relax( $u, v, w$ )
```



$S = \{s, y, z, t, x\}$

Q

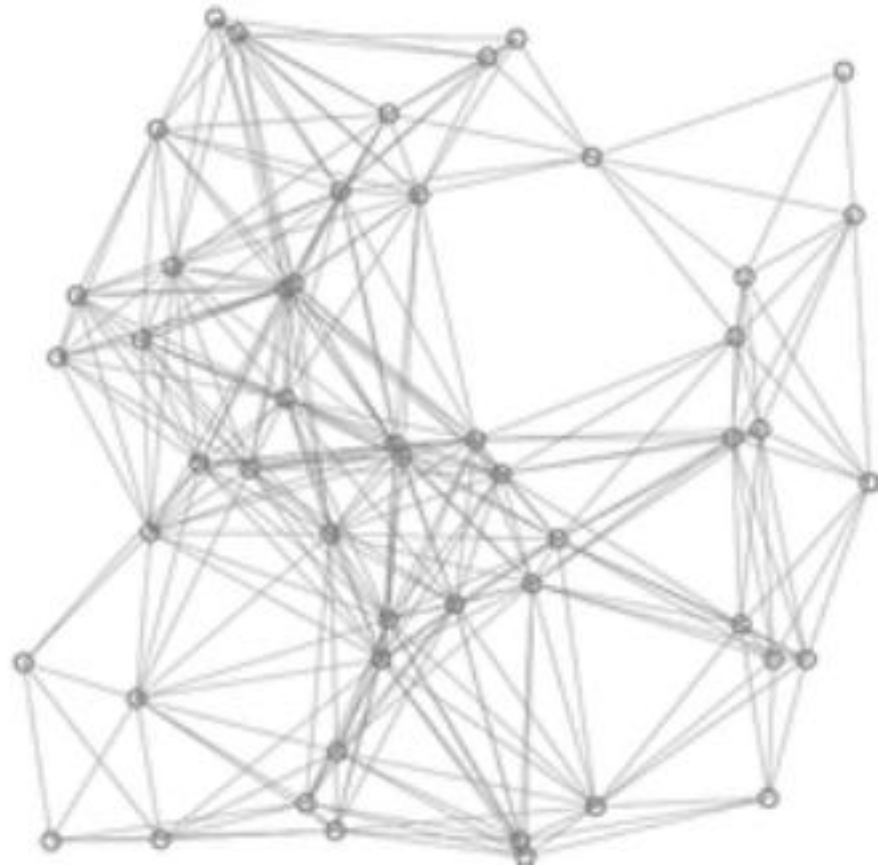
$\text{ExtractMin}(Q) = x$



Example

A demo of Dijkstra's algorithm based on Euclidean distance.

- Red lines (最短路径) are the shortest path covering, i.e., connecting u and $\pi[u]$.
- Blue lines (有松弛) indicate where relaxing happens, i.e., connecting v with a node u in Q , which gives a shorter path from the source to v .





Dijkstra's Algorithm vs. Prim's Algorithm

Dijkstra算法和Prim算法很像

Dij 中 $d[v]$ 存储到源点 s 的距离

Prim $key[v]$ 存储到树的距离

Dijkstra(G, w, s)

```
1 InitializeSingleSource( $G, s$ )
2  $S \leftarrow \emptyset$ 
3  $Q \leftarrow V$ 
4 while  $Q \neq \emptyset$  do
5    $u \leftarrow \text{ExtractMin}(Q)$ 
6    $S \leftarrow S \cup \{u\}$ 
7   for each vertex  $v \in \text{Adj}[u]$  do
8     Relax( $u, v, w$ )
```

Relax(u, v, w)

```
1 if  $d[v] > d[u] + w(u, v)$  then
2    $d[v] \leftarrow d[u] + w(u, v)$ 
3    $\pi[v] \leftarrow u$ 
```

PrimMST(G, w, r)

```
1 for each  $u \in V$  do
2    $key[u] \leftarrow \infty$ 
3    $\pi[u] \leftarrow \text{NIL}$ 
4  $key[r] \leftarrow 0$ 
5  $Q \leftarrow V$ 
6 while  $Q \neq \emptyset$  do
7    $u \leftarrow \text{ExtractMin}(Q)$ 
8   for each  $v \in \text{Adj}[u]$  do
9     if  $v \in Q$  and  $w(u, v) < key[v]$  then
10       $\pi[v] \leftarrow u$ 
11       $key[v] \leftarrow w(u, v)$ 
12      DecreaseKey( $Q, v, key$ )
```



Correctness of Dijkstra's Algorithm

Theorem 8.8

Dijkstra's algorithm, run on a weighted, directed graph $G = (V, E)$ with non-negative weight function w and source s , terminates with $d[u] = \delta(s, u)$ for all vertices $u \in V$. 算法运行结束后 $d[u]$ 就是最短路径的长度

- We use loop invariant to prove: At the start of each iteration of the while loop (Line 4-8), $d[v] = \delta(s, v)$ for each vertex $v \in S$.

用循环不变量证明，在每次循环开始的时候对于 S 中的每个顶点 v ， $d[v]$ 就是最短路径的长度 $\delta(s, v)$



Correctness of Dijkstra's Algorithm

```
Relax( $u, v, w$ )  
1 if  $d[v] > d[u] + w(u, v)$  then  
2    $d[v] \leftarrow d[u] + w(u, v)$   
3    $\pi[v] \leftarrow u$ 
```

```
Dijkstra( $G, w, s$ )  
1 InitializeSingleSource( $G, s$ )  
2  $S \leftarrow \emptyset$   
3  $Q \leftarrow V$   
4 while  $Q \neq \emptyset$  do  
5    $u \leftarrow \text{ExtractMin}(Q)$   
6    $S \leftarrow S \cup \{u\}$   
7   for each vertex  $v \in \text{Adj}[u]$  do  
8     Relax( $u, v, w$ )
```

Proof:

Initialization: Initially, $S = \emptyset$, and so the invariant is trivially true.

Maintenance: 归纳假设之前 S 里面的顶点都是最短距离了。
之后 We wish to show that after each iteration, $d[u] = \delta(s, u)$
for the vertex u just added to set S . 要证明在每次迭代之后刚刚
加进 S 的节点 u 的 $d[u]$ 也是最短距离

■ If there is no shortest path from s to u , $d[u] = \delta(s, u) = \infty$.

$u \leftarrow \text{ExtractMin}(Q)$ u 是最小优先队列中最小的。

分三种情况证明

1 没有最短路径

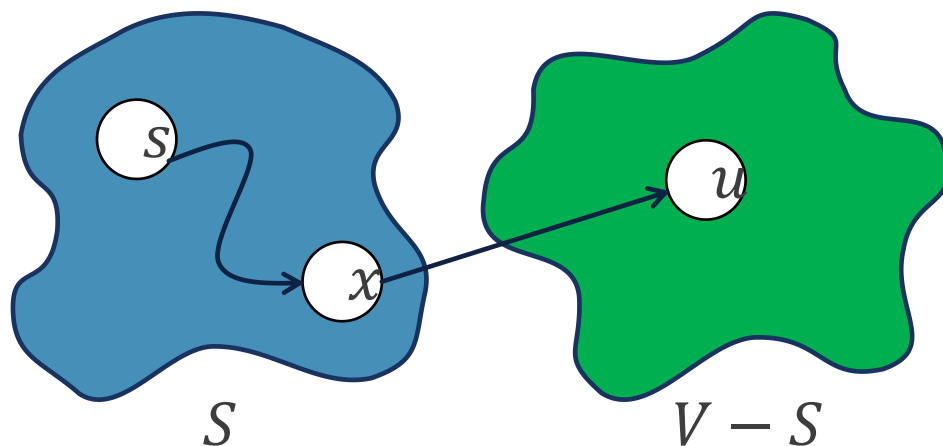
2 u 直接和大 S 中顶点
相连

3 u 不直接和大 S 中顶
点相连



Correctness of Dijkstra's Algorithm

- Proof (cont'd):** 假设有一条最短路径 p 从 s 到 u
如果 u 直接和 S 集合中元素相连接, 那么 $d[u]$ 就是最短距离
- Now we consider when there is a shortest path p from s to u .
 - If in the path p , u is connected to a vertex $x \in S$, without any other vertex $y \in V - s$ between x and u , $d[u] = \delta(s, u)$ by the convergence property.
 - Because $x \in S$, we know $d[x] = \delta(s, x)$.



115页 Convergence property: If $s \rightsquigarrow u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and $d[u] = \delta(s, u)$, then after $\text{Relax}(u, v, w)$, we have $d[v] \leq \delta(s, v)$. 具体证明请见 introduction to algorithms 24.14

$S \rightsquigarrow x \rightarrow v$ 指 s 途经 x 到达 v



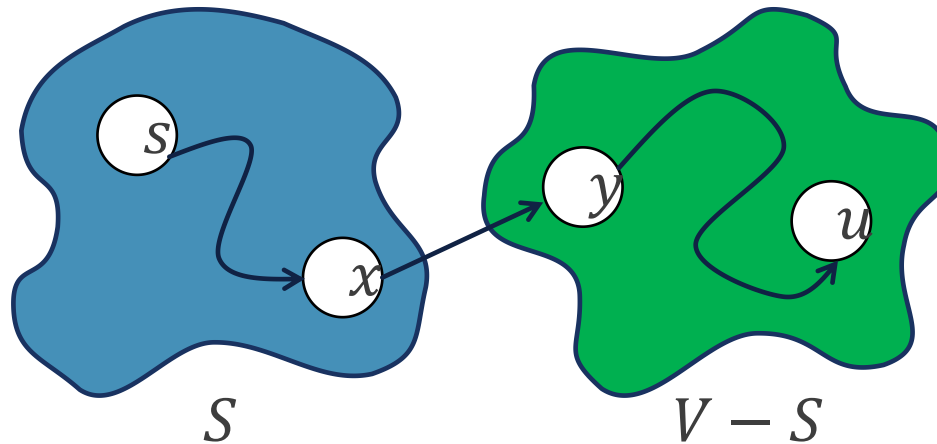
Correctness of Dijkstra's Algorithm

假设有一条最短路径 p 从 s 到 u

如果 u 不直接和 S 集中元素相连接，而是通过 y 再和 S 集合元素相连。

Proof (cont'd): 同样的 $d[y]$ 也是最短距离

- If u is not directly connected to x , there is a vertex $y \in V - S$ between x and u , we can assume y is the one connected to x .
- Again, by the convergence property we have $d[y] = \delta(s, y)$.





Correctness of Dijkstra's Algorithm

假设有一条最短路径 p 从 s 到 u

如果 u 不直接和 S 集合中元素相连接, 而是通过 y 再和 S 集合元素相连, 这个时候 $d[y]$ 也是最短距离

并且 y 的最短距离 $\leq u$ 的最短距离

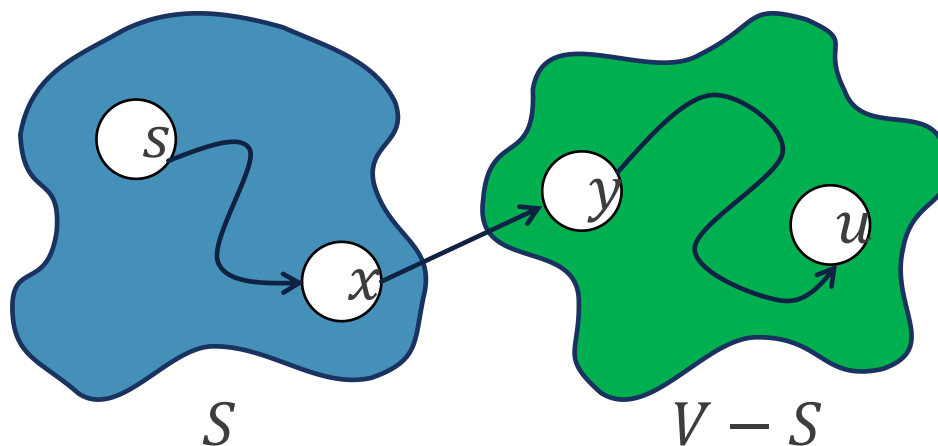
Proof (cont'd):

- Because y appears before u in the shortest path p , we have $\delta(s, y) \leq \delta(s, u)$.

- By the upper bound property, we have $\delta(s, u) \leq d[u]$, then we get:

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u].$$

这里证明
 $d[y] \leq d[u]$.





Correctness of Dijkstra's Algorithm

假设有一条最短路径 p 从 s 到 u

如果 u 不直接和 S 集合中元素相连接，而是通过 y 再和 S 集合元素相连，这个时候 $d[y]$ 也是最短距离，并且 $d[y] \leq d[u]$

Proof (cont'd):

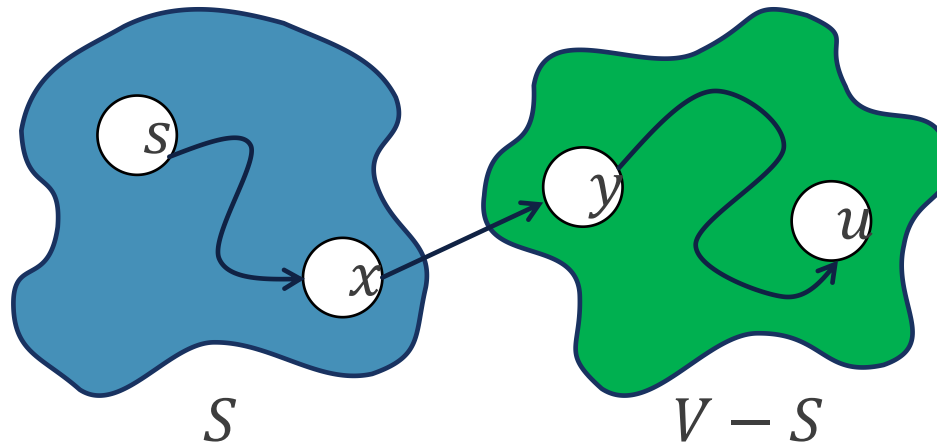
- However, u is chosen by Dijkstra's algorithm, which means:

$$d[u] \leq d[y].$$

u 是从最小优先队列里面抽的

- We have both $d[y] \leq d[u]$ and $d[u] \leq d[y]$, we get:

$$d[y] = \delta(s, y) = \delta(s, u) = d[u].$$





Correctness of Dijkstra's Algorithm

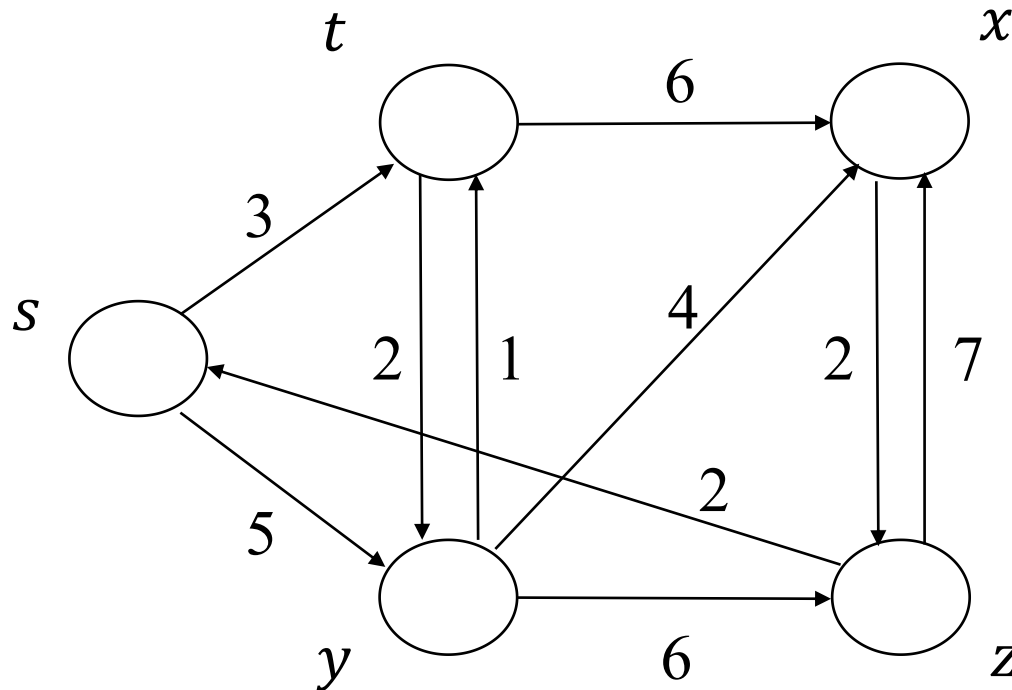
Proof (cont'd):

Termination: At termination, $Q = \emptyset$ and $S = V$. Thus, $d[u] = \delta(s, u)$ for all vertices $u \in V$.



Classroom Exercise

Run Dijkstra's algorithm on the following graph, starting from s .

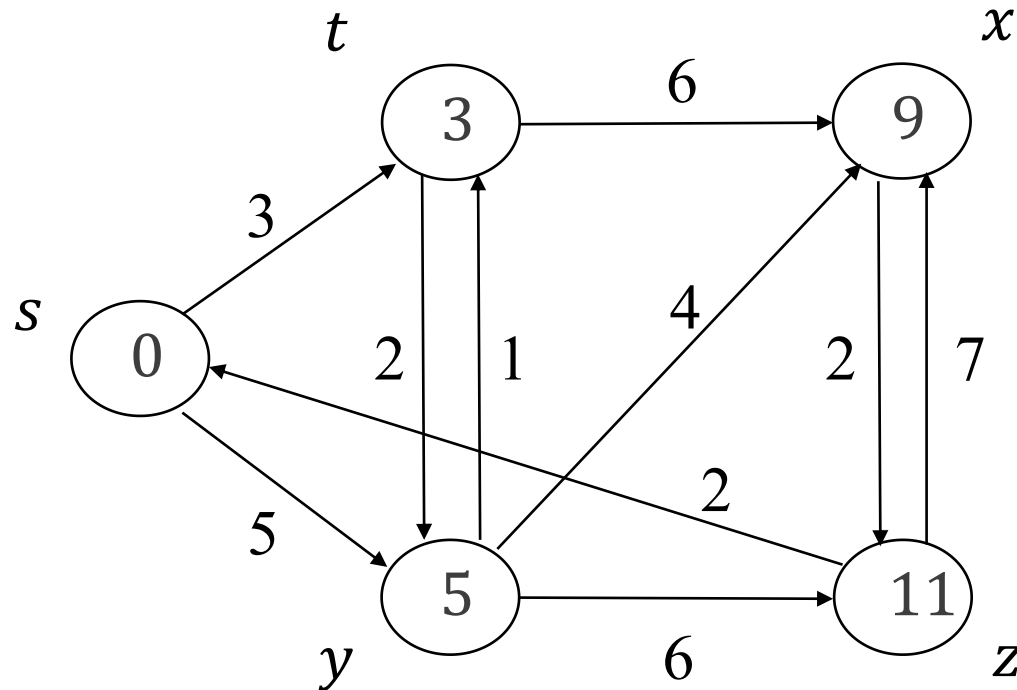


```
Dijkstra( $G, w, s$ )  
1 InitializeSingleSource( $G, s$ )  
2  $S \leftarrow \emptyset$   
3  $Q \leftarrow V$   
4 while  $Q \neq \emptyset$  do  
5    $u \leftarrow \text{ExtractMin}(Q)$   
6    $S \leftarrow S \cup \{u\}$   
7   for each vertex  $v \in \text{Adj}[u]$  do  
8     Relax( $u, v, w$ )
```



Classroom Exercise

Solution:



```
Dijkstra( $G, w, s$ )  
1 InitializeSingleSource( $G, s$ )  
2  $S \leftarrow \emptyset$   
3  $Q \leftarrow V$   
4 while  $Q \neq \emptyset$  do  
5    $u \leftarrow \text{ExtractMin}(Q)$   
6    $S \leftarrow S \cup \{u\}$   
7   for each vertex  $v \in \text{Adj}[u]$  do  
8     Relax( $u, v, w$ )
```



THE SHORTEST PATHS PROBLEM

ALL-PAIRS SHORTEST-PATHS PROBLEM



All-Pairs Shortest-Paths Problem

- We can solve an all-pairs shortest-paths problem by running a single-source shortest-paths algorithm $|V|$ times, once for each vertex as the source. 对于求解所有节点对之间的最短路径，显然我们可以对所有的节点跑Dijkstra算法，就可以了
- Can we do better? Can we get the shortest paths for all pairs of vertices at the same time? 肯定是有更快的算法的
- Yes, we have proved that the shortest path problem has optimal substructure property, dynamic programming is ready to go! 最短路径问题有最优子结构性质，可以直接用动态规划方法



All-Pairs Shortest-Paths Problem

- For convenience, we assume that the vertices are numbered $1, 2, \dots, |V|$, so that the input is an $|V| \times |V|$ matrix W representing the edge weights of an $|V|$ -vertex directed graph $G = (V, E)$. That is, $W = (w_{ij})$, where

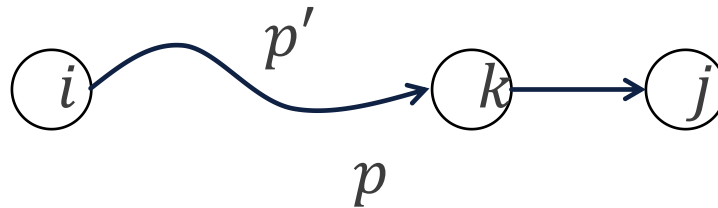
$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ w(i, j) & \text{if } i \neq j \text{ and } (i, j) \in E \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

- Negative-weight edges are allowed, but we assume for the time being that the input graph contains **no negative-weight cycles**.



Dynamic Programming Solution 动态规划解法

- Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most m edges.
- Assuming that there are no negative-weight cycles, m is finite.
 - If $i = j$, then p has weight 0 because of no edges.
 - If vertices i and j are distinct, then we decompose path p into



where path p' now contains at most $m - 1$ edges.

- By Theorem 8.7 (最优子结构), p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.



Dynamic Programming Solution

- Let $l_{ij}^{(m)}$ be the minimum weight of any path from vertex i to vertex j that contains **at most m edges**. 从*i*到*j*经过最多*m*条边的距离
- When $m = 0$, there is a shortest path from i to j with no edges if and only if $i = j$. Thus,

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } i \neq j \end{cases}$$

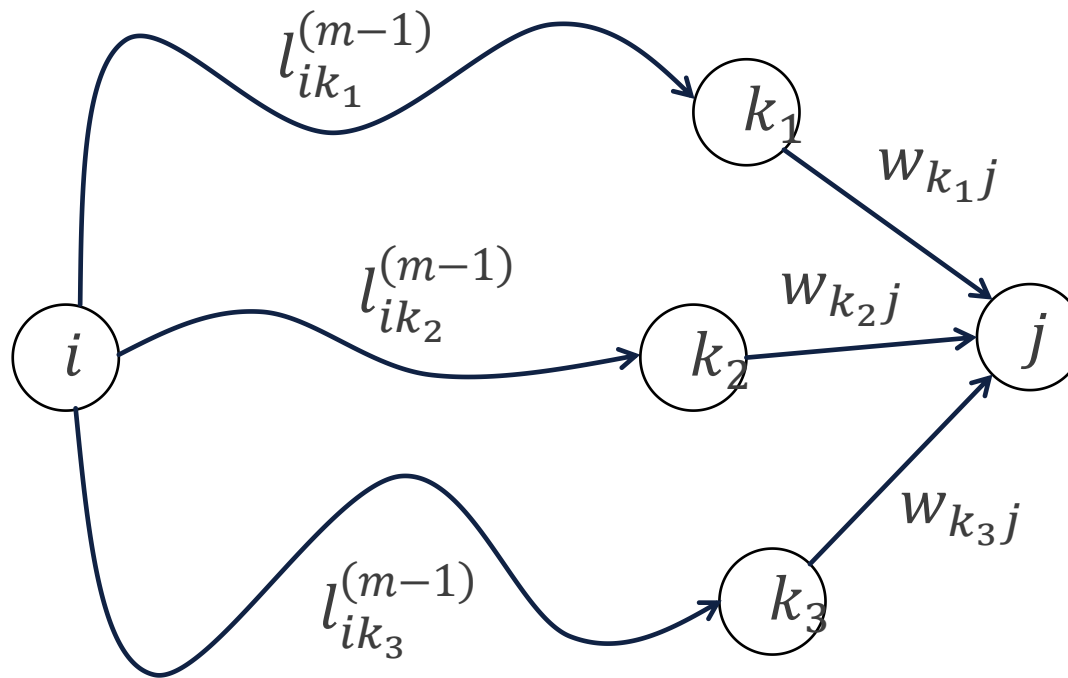
- When $m \geq 1$, we check all j 's neighbor k and select the minimal one:

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{ l_{ik}^{(m-1)} + w_{kj} \}$$



Dynamic Programming Solution

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{ l_{ik}^{(m-1)} + w_{kj} \}$$





Dynamic Programming Solution

- If a shortest path contains more than $|V| - 1$ vertices, it must contain a cycle.
 - If the cycle weight is positive, we can remove it to get a shorter path.
 - If the cycle weight is negative, we can go through this cycle infinite number of times to obtain $\delta(i, j) = -\infty$.
- Therefore, we assume that the graph contains no negative-weight cycles. 假设图中没有负权重的圈
- For every pair of vertices i and j for which $\delta(i, j) < \infty$, there is a shortest path from i to j that is simple and thus contains at most $|V| - 1$ edges.
- Finally: $\delta(i, j) = l_{ij}^{(|V|-1)}$.



Dynamic Programming Solution

- Let matrix $L^{(m)} = (l_{ij}^{(m)})$, we iteratively calculate
 $L^{(1)}, L^{(2)}, \dots, L^{(|V|-1)}$
- The matrix $L^{(|V|-1)}$ contains the shortest path distance of all vertex pairs.



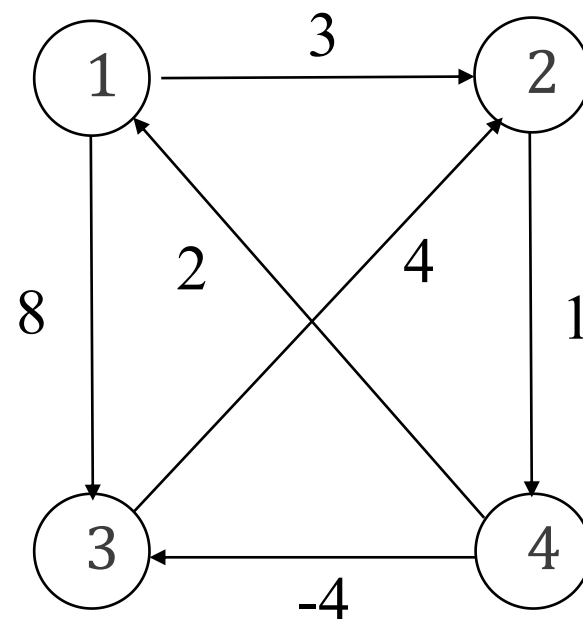
Example

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$

$$L^{(0)} = \begin{bmatrix} 0 & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty \\ \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Initialization:

Shortest path distance with 0 edges.



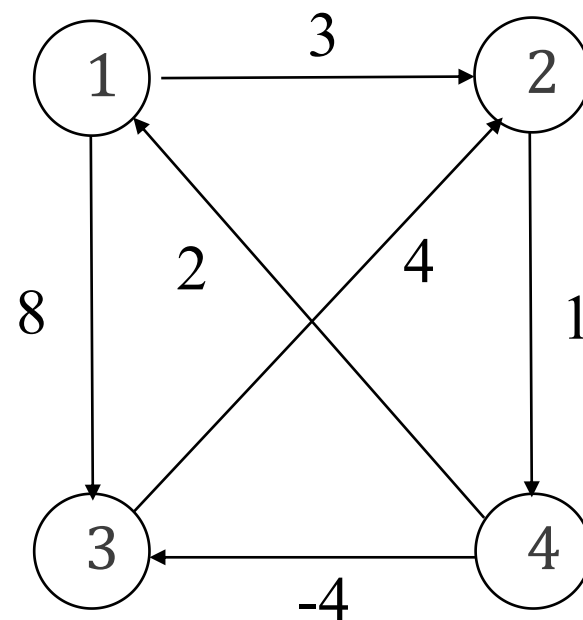


Example

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$

$$L^{(1)} = \begin{bmatrix} 0 & 3 & 8 & \infty \\ \infty & 0 & \infty & 1 \\ \infty & 4 & 0 & \infty \\ 2 & \infty & -4 & 0 \end{bmatrix}$$

Shortest path distance within 1 edges.





Example

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$

$$\begin{aligned} l_{11}^{(1)} + w_{12} &= 0 + 3 = 3 \\ l_{12}^{(1)} + w_{22} &= 3 + 0 = 3 \\ l_{13}^{(1)} + w_{32} &= 8 + 4 = 12 \\ l_{14}^{(1)} + w_{42} &= \infty + \infty = \infty \end{aligned}$$

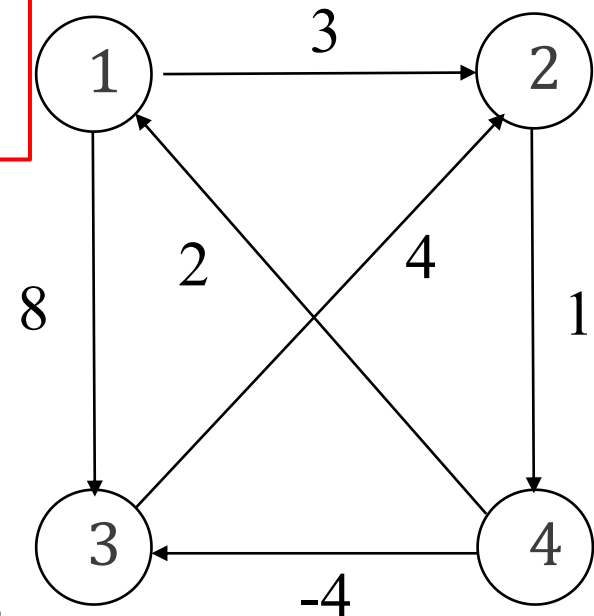
$$\begin{aligned} l_{11}^{(1)} + w_{14} &= 0 + \infty = \infty \\ l_{12}^{(1)} + w_{24} &= 3 + 1 = 4 \\ l_{13}^{(1)} + w_{34} &= 8 + \infty = \infty \\ l_{14}^{(1)} + w_{44} &= \infty + 0 = \infty \end{aligned}$$

$$L^{(2)} = \begin{bmatrix} 0 & \mathbf{3} & 8 & \mathbf{4} \\ \mathbf{3} & 0 & -3 & 1 \\ \infty & 4 & 0 & \mathbf{5} \\ 2 & \mathbf{0} & -4 & 0 \end{bmatrix}$$

$l_{12}^{(2)}$ points to the value 3 in the first row, second column.

$l_{14}^{(2)}$ points to the value 4 in the first row, fourth column.

Shortest path distance within 2 edges.



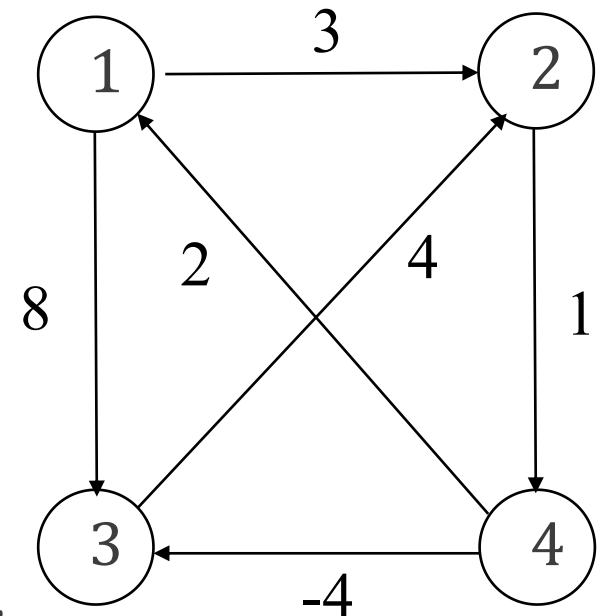


Example

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$

$$L^{(3)} = \begin{bmatrix} 0 & 3 & 0 & 4 \\ 3 & 0 & -3 & 1 \\ 7 & 4 & 0 & 5 \\ 2 & 0 & -4 & 0 \end{bmatrix}$$

Shortest path distance within 3 edges.





Dynamic Programming Solution

- Now look at the recursive equation, what is the computational complexity for this DP algorithm?

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{ l_{ik}^{(m-1)} + w_{kj} \}$$

- $O(|V|^4)$. Can we improve?



THE SHORTEST PATHS PROBLEM

FLOYD-WARSHALL ALGORITHM



Floyd-Warshall Algorithm



Robert W. Floyd
(1936-2001)

Turing Award in 1978



Stephen Warshall
(1935-2006)

- **Floyd-Warshall algorithm (弗洛伊德算法)** is an example of dynamic programming, and was published by Robert Floyd in 1962 and also by Stephen Warshall in 1962.



Floyd-Warshall Algorithm

- Can we use the idea of 0/1 knapsack problem? Determine at each step that we take item i or not. 能不能借鉴0/1背包的想法，确定在每一步要不要用item i
- First, we number all the vertices: $V = \{1, 2, \dots, |V|\}$.
- Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which we can only go through vertices are in the set $\{1, 2, \dots, k\}$.

$d_{ij}^{(k)}$ 定义为从 i 到 j 只利用 $\{1, 2, \dots, k\}$.作为中间节点的最短距离

- “Go through” does not include vertex i and j . 中间节点不包含 i, j 本身
- When $k = 0$, $d_{ij}^{(0)} = w_{ij}$, because the path is directly from vertex i to vertex j , without going through any vertex.



Floyd-Warshall Algorithm

- We have know all shortest paths when we can go though vertices $\{1, \dots, k-1\}$.
- Given a new vertex k to let you use it to go though, we may decide:
 - Don't go though it: $d_{ij}^{(k)} = d_{ij}^{(k-1)}$.
 - Go though it: $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.
- The recursive equation is:

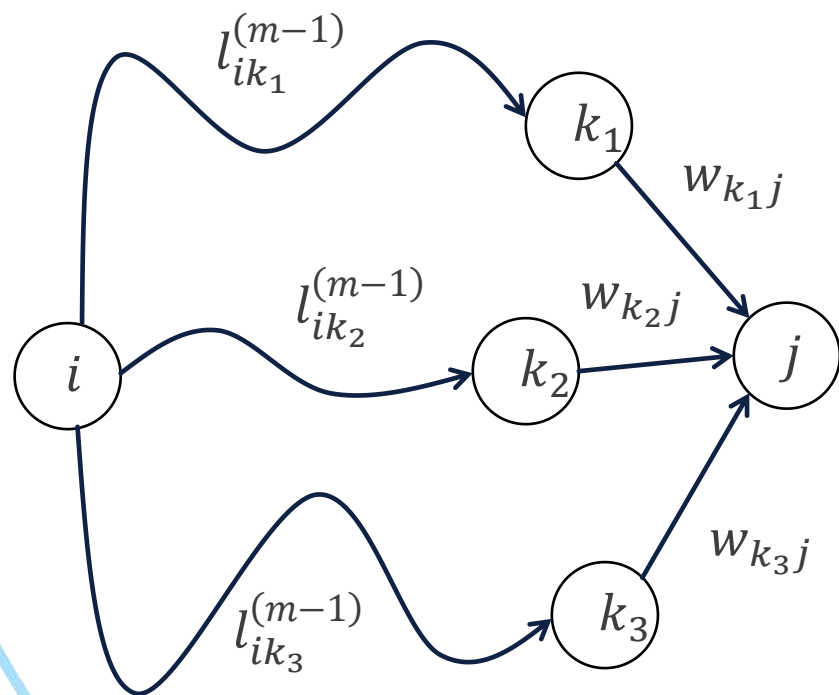
判断是不是经过k这个顶点

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 \\ \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & \text{if } k \geq 1 \end{cases}$$

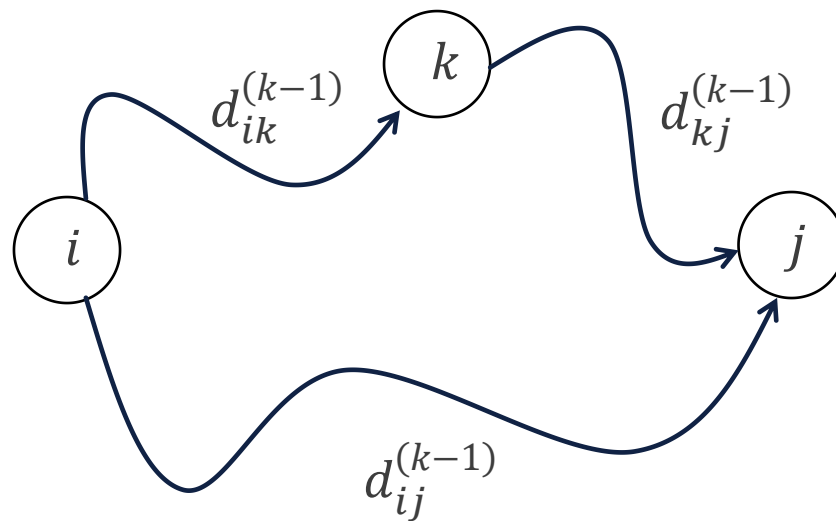


Floyd-Warshall Algorithm

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$



$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$





Pseudocode

FloydWarshall(W)

1 $D^{(0)} \leftarrow W$

2 **for** $k \leftarrow 1$ **to** $|V|$ **do**

3 **for** $i \leftarrow 1$ **to** $|V|$ **do**

4 **for** $j \leftarrow 1$ **to** $|V|$ **do**

5 **if** $d_{ij}^{(k-1)} < d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ **then** $d_{ij}^{(k)} \leftarrow d_{ij}^{(k-1)}$

6 **else** $d_{ij}^{(k)} = d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$

7 **return** $D^{(|V|)}$

Total running time: $O(|V|^3)$

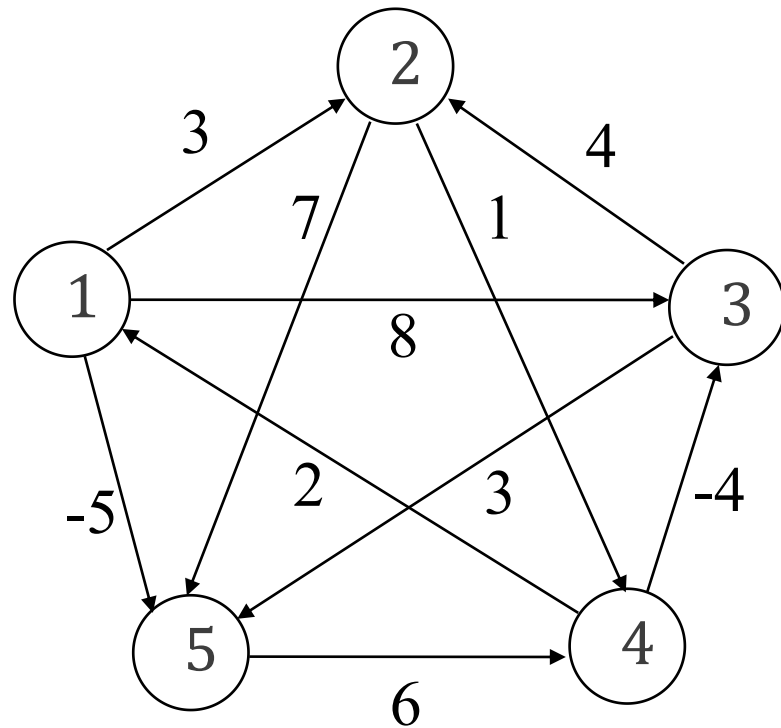


Example

$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

$$D^{(0)} = \begin{bmatrix} 0 & 3 & 8 & \infty & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & 3 \\ 2 & \infty & -4 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

Initialization





Example

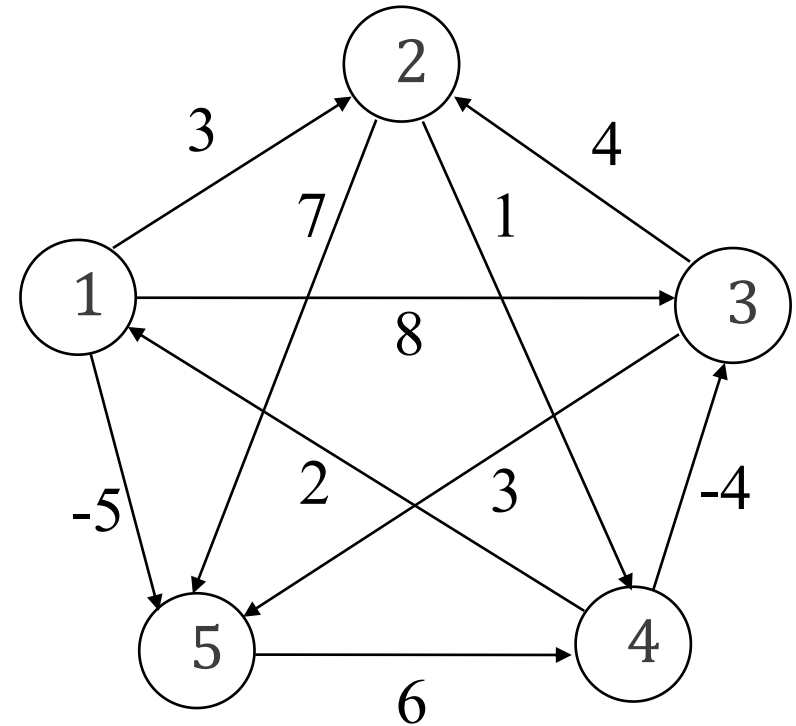
$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

$$\begin{aligned} d_{42}^{(0)} &= \infty \\ d_{41}^{(0)} + d_{12}^{(0)} &= 5 \end{aligned}$$

$$\begin{aligned} d_{45}^{(0)} &= \infty \\ d_{41}^{(0)} + d_{15}^{(0)} &= 3 \end{aligned}$$

$$D^{(1)} = \begin{bmatrix} 0 & 3 & 8 & \infty & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & 3 \\ 2 & \mathbf{5} & -4 & 0 & \mathbf{-3} \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

Only go through $\{v_1\}$





Example

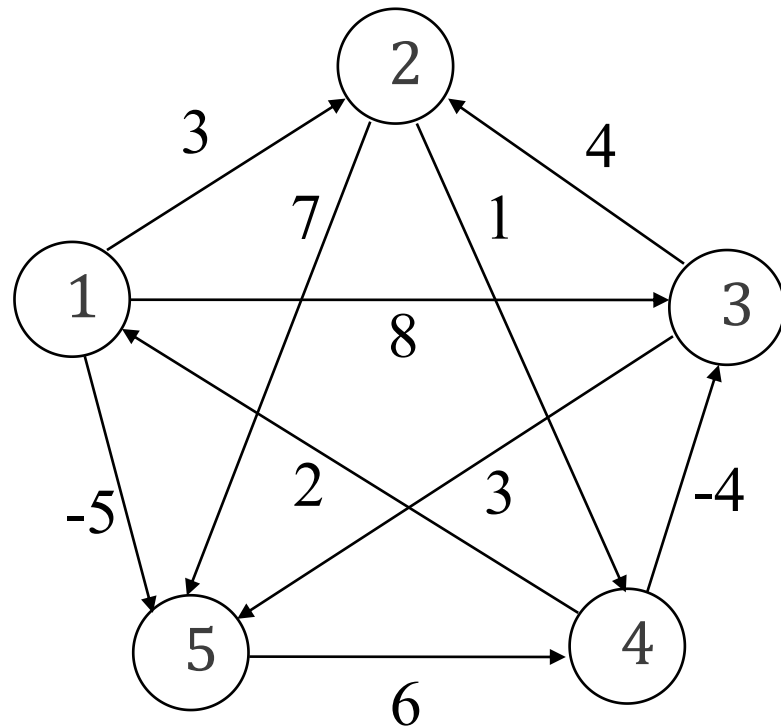
$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

$$\begin{aligned} d_{34}^{(1)} &= \infty \\ d_{32}^{(1)} + d_{24}^{(1)} &= 5 \end{aligned}$$

$$\begin{aligned} d_{14}^{(1)} &= \infty \\ d_{12}^{(1)} + d_{24}^{(1)} &= 4 \end{aligned}$$

$$D^{(2)} = \begin{bmatrix} 0 & 3 & 8 & 4 & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 3 \\ 2 & 5 & -4 & 0 & -3 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

Only go through $\{v_1, v_2\}$





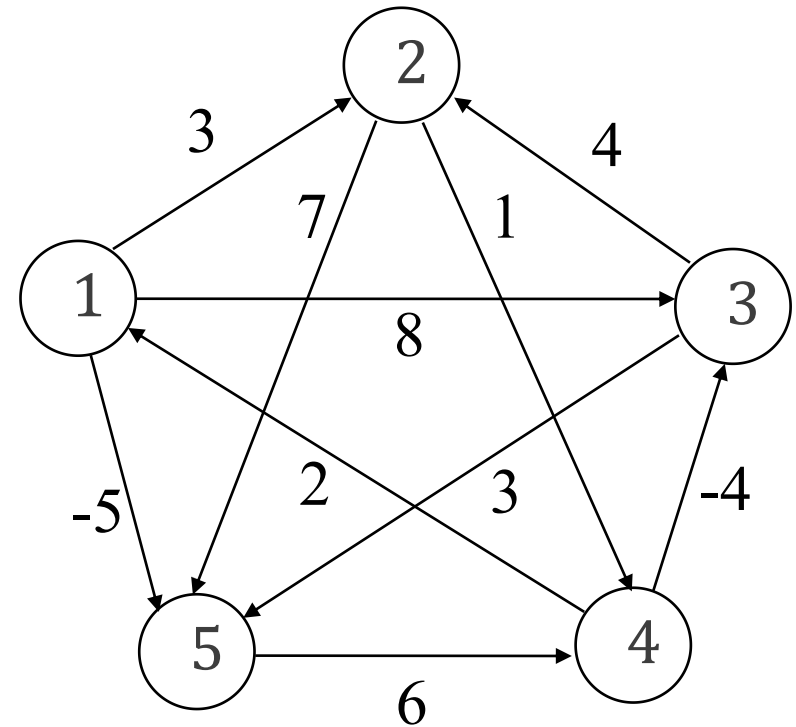
Example

$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

$$\begin{aligned} d_{42}^{(2)} &= 5 \\ d_{43}^{(2)} + d_{32}^{(2)} &= 0 \end{aligned}$$

$$D^{(3)} = \begin{bmatrix} 0 & 3 & 8 & 4 & -5 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 3 \\ 2 & 0 & -4 & 0 & -3 \\ \infty & \infty & \infty & 6 & 0 \end{bmatrix}$$

Only go through $\{v_1, v_2, v_3\}$



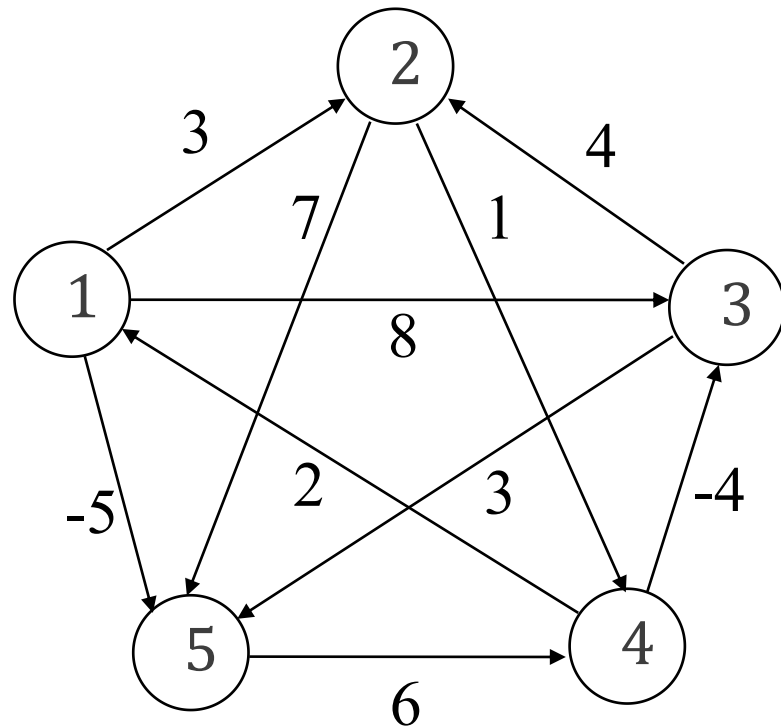


Example

$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

$$D^{(4)} = \begin{bmatrix} 0 & 3 & 0 & 4 & -5 \\ 3 & 0 & -3 & 1 & -2 \\ 7 & 4 & 0 & 5 & 2 \\ 2 & 0 & -4 & 0 & -3 \\ 8 & 6 & 2 & 6 & 0 \end{bmatrix}$$

Only go though $\{v_1, v_2, v_3, v_4\}$

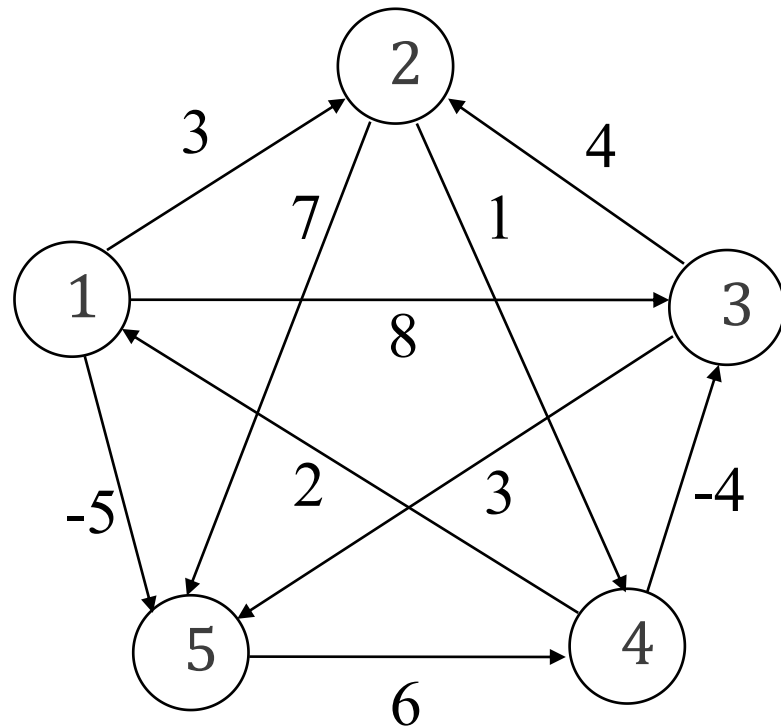




Example

$$D^{(5)} = \begin{bmatrix} 0 & \mathbf{1} & \mathbf{-3} & \mathbf{1} & -5 \\ 3 & 0 & -3 & 1 & -2 \\ 7 & 4 & 0 & 5 & 2 \\ 2 & 0 & -4 & 0 & -3 \\ 8 & 6 & 2 & 6 & 0 \end{bmatrix}$$

Only go through $\{v_1, v_2, v_3, v_4, v_5\}$



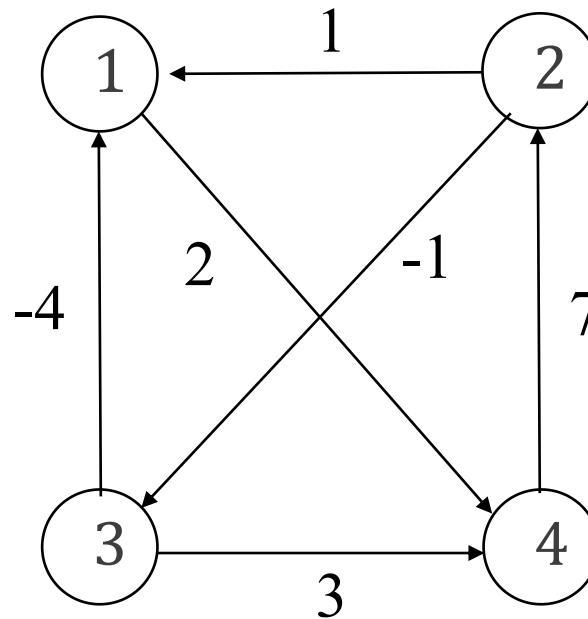


Classroom Exercise

$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

Run Floyd-Warshall algorithm on the following graph.

写出不同k的
矩阵，一共
有5个矩阵





Classroom Exercise 课堂练习

$$d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$$

Solution:

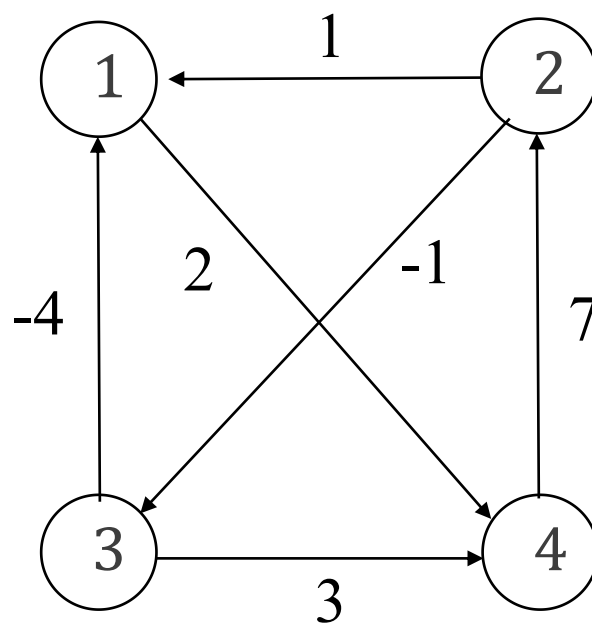
$$D^{(0)} = \begin{bmatrix} 0 & \infty & \infty & 2 \\ 1 & 0 & -1 & \infty \\ -4 & \infty & 0 & 3 \\ \infty & 7 & \infty & 0 \end{bmatrix}$$

$$D^{(1)} = \begin{bmatrix} 0 & \infty & \infty & 2 \\ 1 & 0 & -1 & \mathbf{3} \\ -4 & \infty & 0 & \mathbf{-2} \\ \infty & 7 & \infty & 0 \end{bmatrix}$$

$$D^{(2)} = \begin{bmatrix} 0 & \infty & \infty & 2 \\ 1 & 0 & -1 & 3 \\ -4 & \infty & 0 & -2 \\ \mathbf{8} & 7 & \mathbf{6} & 0 \end{bmatrix}$$

$$D^{(3)} = \begin{bmatrix} 0 & \infty & \infty & 2 \\ \mathbf{-5} & 0 & -1 & \mathbf{-3} \\ -4 & \infty & 0 & -2 \\ \mathbf{2} & 7 & 6 & 0 \end{bmatrix}$$

$$D^{(4)} = \begin{bmatrix} 0 & \mathbf{9} & \mathbf{8} & 2 \\ \mathbf{-5} & 0 & -1 & \mathbf{-3} \\ -4 & \mathbf{5} & 0 & -2 \\ 2 & 7 & 6 & 0 \end{bmatrix}$$





Johnson's Algorithm for Sparse Graph

- 相比Floyd-Warshall 算法快，并能处理负回路情况
- 综合采用了Dijkstra和Bellman-Ford算法
 - Dijkstra: 处理非负权重的单源最短路径算法 $O(E \lg V)$
 - Bellman-ford能够检测负回路的单源最短路径算法 $O(VE)$
- Only positive weight edges 如果图里面没有负数边，就对每个节点运行Dijkstra算法，在采用Fibonacci堆的情况下，复杂度 $O(V^2 \lg V + VE)$
- 有负数边，但是没有负回路, compute a new set of nonnegative edge weights that allows us to use the same method
 - 对边权重进行修改，使其非负



Johnson's Algorithm

- 修改原图G为G'，G'增加一个虚拟顶点S，其到达其他节点的权重为0，其他顶点和边的权重和原图G相同。之后对G'运行Bellman-ford算法，找出S到其他节点的路径，或者检测有没有负回路

- 如果没有负回路，修改G'每条边的权重如下

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v)$$

- 其中 $h(u)$ 是指在G'上，从虚拟顶点s到达u的最短距离 $\delta(s, u)$
- 之后对G'运行Dijkstra算法，即可获得最短路径



Johnson's Algorithm 伪代码

JOHNSON(G)

```
1  compute  $G'$ , where  $V[G'] = V[G] \cup \{s\}$ ,  
    $E[G'] = E[G] \cup \{(s, v) : v \in V[G]\}$ , and  
    $w(s, v) = 0$  for all  $v \in V[G]$   
2  if BELLMAN-FORD( $G'$ ,  $w$ ,  $s$ ) = FALSE  
3      then print "the input graph contains a negative-weight cycle"  
4  else for each vertex  $v \in V[G']$   
5      do set  $h(v)$  to the value of  $\delta(s, v)$   
           computed by the Bellman-Ford algorithm  
6      for each edge  $(u, v) \in E[G']$   
7          do  $\hat{w}(u, v) \leftarrow w(u, v) + h(u) - h(v)$   
8      for each vertex  $u \in V[G]$   
9          do run DIJKSTRA( $G$ ,  $\hat{w}$ ,  $u$ ) to compute  $\hat{\delta}(u, v)$  for all  $v \in V[G]$   
10         for each vertex  $v \in V[G]$   
11             do  $d_{uv} \leftarrow \hat{\delta}(u, v) + h(v) - h(u)$   
12  return  $D$ 
```



Conclusion

After this lecture, you should know:

- What is the difference between BFS and DFS.
- How to prove the correctness of BFS.
- How to obtain a minimal spanning tree.
- How to prove the correctness of Kruskal's and Prim's algorithm.
- What is relaxation.
- How to solve single-source shortest-paths problem.
- How to solve all-pairs shortest-paths problem.



Homework (课后作业)

■ Page 135-137

8.2 8.5 8.6 8.7

8.8 8.9 8.10 8.19

8.22 8.27



编程作业

- Leetcode 100 (Same Tree)
- Leetcode 207 (Course Schedule)
- Leetcode 1584 (Min Cost to Connect All Points)



有问题欢迎随时跟我讨论



Experiment

- There are n different types of items.
- There are m seller, each of them sells one set of items at a certain price (price is for **the set**). Each of these m people sells either nothing or all the items in the set he has.
- Information of these m sellers is represented by matrix M of size $m \times n$.

$$M_{ij} = \begin{cases} 1 & \text{seller } i \text{ sells item } j \\ 0 & \text{otherwise} \end{cases}$$

- The seller i will sell set of all his items at price C_i .
- You want to buy at least one item of each type (it is guaranteed that it is always possible to do so). **Use graph algorithms** to find the minimal cost needed to pay to achieve it.



Experiment

■ Input:

- The first line contains two integers n, m separated by space.
- The $(i + 1)$ th ($1 \leq i \leq m$) line contains $(n + 1)$ integers $M_{i,1}, M_{i,2}, M_{i,3}, \dots, M_{i,n}, C_i$.

■ Output: Minimal cost.

■ Sample:

```
3 3
0 1 1 2
1 0 0 3
1 1 1 9
```

Input

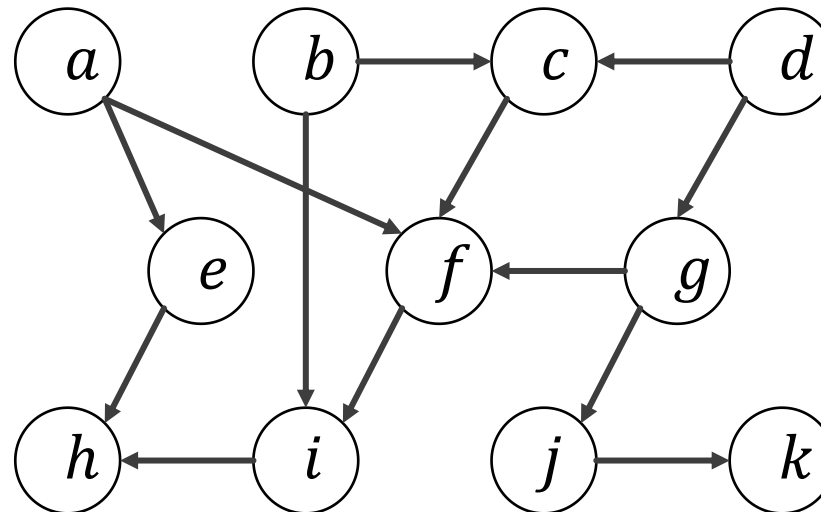
```
5
```

Output



Classroom Exercise (课堂练习)

Show the ordering of vertices produced by TopologicalSort when it is run on the following DAG.





Dynamic Programming Solution

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$

SLOW-ALL-PAIRS-SHORTEST-PATHS (W)

```
1   $n \leftarrow \text{rows}[W]$ 
2   $L^{(1)} \leftarrow W$ 
3  for  $m \leftarrow 2$  to  $n - 1$ 
4      do  $L^{(m)} \leftarrow \text{EXTEND-SHORTEST-PATHS}(L^{(m-1)}, W)$ 
5  return  $L^{(n-1)}$ 
```

EXTEND-SHORTEST-PATHS (L, W)

```
1   $n \leftarrow \text{rows}[L]$ 
2  let  $L' = (l'_{ij})$  be an  $n \times n$  matrix
3  for  $i \leftarrow 1$  to  $n$ 
4      do for  $j \leftarrow 1$  to  $n$ 
5          do  $l'_{ij} \leftarrow \infty$ 
6              for  $k \leftarrow 1$  to  $n$ 
7                  do  $l'_{ij} \leftarrow \min(l'_{ij}, l_{ik} + w_{kj})$ 
8  return  $L'$ 
```



Faster Algorithm

$$l_{ij}^{(m)} = \min_{1 \leq k \leq |V|} \{l_{ik}^{(m-1)} + w_{kj}\}$$

$$L^{(1)} = L^{(0)} \cdot W = W,$$

$$L^{(2)} = L^{(1)} \cdot W = W^2,$$

$$L^{(3)} = L^{(2)} \cdot W = W^3,$$

$\vdots$

$$L^{(n-1)} = L^{(n-2)} \cdot W = W^{n-1}.$$

SLOW-ALL-PAIRS-SHORTEST-PATHS (W)

1 $n \leftarrow \text{rows}[W]$

2 $L^{(1)} \leftarrow W$

3 **for** $m \leftarrow 2$ **to** $n - 1$

4 **do** $L^{(m)} \leftarrow \text{EXTEND-SHORTEST-PATHS}(L^{(m-1)}, W)$

5 **return** $L^{(n-1)}$



Faster Algorithm

由于矩阵乘法满足结合律，我们可以用二分法计算矩阵乘

$$L^{(1)} = W,$$

$$L^{(2)} = W^2 = W \cdot W,$$

$$L^{(4)} = W^4 = W^2 \cdot W^2$$

$$L^{(8)} = W^8 = W^4 \cdot W^4,$$

$$\vdots$$

- 时间复杂度从 $O(V^4)$ 降低为 $O(v^3 \lg v)$